

Magnate Backend API Documentation

None

08 - Roberta Williams - Equipo de backend

2026

Tabla de Contenidos

1. Welcome to Magnate backend documentation	3
2. Integration	4
2.1 REST API	4
2.2 WebSockets	27
2.3 Game	95
2.4 Actions	102
2.5 Responses	125
2.6 Fantasy events	138
2.7 Cheats	154
3. Technical reference	156
3.1 Core Game Logic (games.py)	156
3.2 Exceptions	225
3.3 Agent Heuristics: Probabilities and Dynamic Synergies	232

1. Welcome to Magnate backend documentation

[PDF version of the documentation](#)

2. Integration

2.1 REST API

This section documents the REST API endpoints available in the Magnate backend. All endpoints (except registration and login) require a valid JWT access token in the `Authorization` header.

2.1.1 Authentication

Register

Bases: `APIView`

Endpoint for registering a new user.

POST `/auth/register/`

• Request body

```
{ "username": "mario", "password": "Segura123!", "password2": "Segura123!" }
```

• Responses

201: User created successfully. Returns user data and JWT tokens. 400: Validation error (passwords don't match etc).

• Source code in `magnate/views.py`

```
class RegisterView(APIView):
    """
    Endpoint for registering a new user.

    POST /auth/register/

    Request body:
    {
        "username": "mario",
        "password": "Segura123!",
        "password2": "Segura123!"
    }

    Responses:
        201: User created successfully. Returns user data and JWT tokens.
        400: Validation error (passwords don't match etc).
    """

    permission_classes = [AllowAny]

    def post(self, request) -> Response:
        """
        Handles the user registration request.

        Args:
```

```

    request (Request): The HTTP request object containing registration data.

Returns:
    Response: The HTTP response containing registration results or errors.
    """

serializer = RegisterSerializer(data=request.data)
if serializer.is_valid():
    user = serializer.save()
    assert isinstance(user, CustomUser)
    tokens = get_tokens_for_user(user)
    return Response({
        'message': 'correctly registered user',
        'user': UserProfileSerializer(user).data,
        'tokens': tokens,
    }, status=status.HTTP_201_CREATED)
return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

```

2.1.2 `post(request)`

Handles the user registration request.

Parameters:

Name	Type	Description	Default
<code>request</code>	<code>Request</code>	The HTTP request object containing registration data.	<i>required</i>

Returns:

Name	Type	Description
<code>Response</code>	<code>Response</code>	The HTTP response containing registration results or errors.

• Source code in `magnate/views.py`

```

def post(self, request) -> Response:
    """
    Handles the user registration request.

    Args:
        request (Request): The HTTP request object containing registration data.

    Returns:
        Response: The HTTP response containing registration results or errors.
    """

    serializer = RegisterSerializer(data=request.data)
    if serializer.is_valid():
        user = serializer.save()
        assert isinstance(user, CustomUser)
        tokens = get_tokens_for_user(user)
        return Response({
            'message': 'correctly registered user',
            'user': UserProfileSerializer(user).data,
            'tokens': tokens,

```

```
    }, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

Login

Bases: `APIView`

Endpoint for authenticating an existing user.

POST /auth/login/

• Request body

```
{ "username": "mario", "password": "Segura123!" }
```

• Responses

200: Login successful. Returns user data and JWT tokens. 400: Missing or malformed fields. 401: Invalid credentials. 403: Bots are not allowed to log in.

• Source code in `magnate/views.py`

```
class LoginView(APIView):
    """
    Endpoint for authenticating an existing user.

    POST /auth/login/

    Request body:
    {
        "username": "mario",
        "password": "Segura123!"
    }

    Responses:
        200: Login successful. Returns user data and JWT tokens.
        400: Missing or malformed fields.
        401: Invalid credentials.
        403: Bots are not allowed to log in.
    """

    permission_classes = [AllowAny]

    def post(self, request) -> Response:
        """
        Handles the user login request.

        Args:
            request (Request): The HTTP request object containing login credentials.

        Returns:
            Response: The HTTP response containing login results or errors.
```

```

"""
serializer = LoginSerializer(data=request.data)
if not serializer.is_valid():
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

data: dict = serializer.validated_data # type: ignore

auth_user = authenticate(
    username=data['username'],
    password=data['password'],
)
if auth_user is None:
    return Response(
        {'error': 'bad credentials'},
        status=status.HTTP_401_UNAUTHORIZED,
    )

assert isinstance(auth_user, CustomUser) ### can't be bot

tokens = get_tokens_for_user(auth_user)
return Response({
    'message': 'succesful login',
    'user':    UserProfileSerializer(auth_user).data,
    'tokens':  tokens,
}, status=status.HTTP_200_OK)

```

2.1.3 `post(request)`

Handles the user login request.

Parameters:

Name	Type	Description	Default
<code>request</code>	<code>Request</code>	The HTTP request object containing login credentials.	<i>required</i>

Returns:

Name	Type	Description
<code>Response</code>	<code>Response</code>	The HTTP response containing login results or errors.

• Source code in `magnate/views.py`

```

def post(self, request) -> Response:
    """
    Handles the user login request.

    Args:
        request (Request): The HTTP request object containing login credentials.

    Returns:
        Response: The HTTP response containing login results or errors.
    """
    serializer = LoginSerializer(data=request.data)
    if not serializer.is_valid():

```

```

        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

data: dict = serializer.validated_data # type: ignore

auth_user = authenticate(
    username=data['username'],
    password=data['password'],
)
if auth_user is None:
    return Response(
        {'error': 'bad credentials'},
        status=status.HTTP_401_UNAUTHORIZED,
    )

assert isinstance(auth_user, CustomUser) ### can't be bot

tokens = get_tokens_for_user(auth_user)
return Response({
    'message': 'succesful login',
    'user':    UserProfileSerializer(auth_user).data,
    'tokens':  tokens,
}, status=status.HTTP_200_OK)

```

2.1.4 User Information

Profile

Bases: `APIView`

Endpoint for retrieving the authenticated user's profile.

GET /user/info/

• Headers

Authorization: Bearer

• Responses

200: Returns the user's profile data. 401: Missing or invalid access token.

• Source code in `magnate/views.py`

```

class ProfileView(APIView):
    """
    Endpoint for retrieving the authenticated user's profile.

    GET /user/info/

    Headers:

```



```
Authorization: Bearer <access_token>

Responses:
    200: Returns the user's profile data.
    401: Missing or invalid access token.
"""

permission_classes = [IsAuthenticated]

def get(self, request) -> Response:
    """
    Handles the request to retrieve user profile data.

    Args:
        request (Request): The HTTP request object.

    Returns:
        Response: The HTTP response containing the user profile data.
    """
    return Response(UserProfileSerializer(request.user).data)
```

2.1.5 `get(request)`

Handles the request to retrieve user profile data.

Parameters:

Name	Type	Description	Default
<code>request</code>	<code>Request</code>	The HTTP request object.	<i>required</i>

Returns:

Name	Type	Description
<code>Response</code>	<code>Response</code>	The HTTP response containing the user profile data.

• Source code in `magnate/views.py`

```
def get(self, request) -> Response:
    """
    Handles the request to retrieve user profile data.

    Args:
        request (Request): The HTTP request object.

    Returns:
        Response: The HTTP response containing the user profile data.
    """
    return Response(UserProfileSerializer(request.user).data)
```

User Played Games

Bases: `APIView`

Returns a list of IDs for all games played by the authenticated user.

GET /user/games-played/

- **Headers**
Authorization: Bearer

- **Responses**
200: Returns a list of game IDs (e.g., {"games": [1, 5, 12]}). 401: Missing or invalid token.

• **Source code in** `magnate/views.py`

```
class GetGamesPlayedView(APIView):
    """
    Returns a list of IDs for all games played by the authenticated user.

    GET /user/games-played/

    Headers:
        Authorization: Bearer <access_token>

    Responses:
        200: Returns a list of game IDs (e.g., {"games": [1, 5, 12]}).
        401: Missing or invalid token.
    """
    permission_classes = [IsAuthenticated]

    def get(self, request) -> Response:
        """
        Handles the request to list all games played by the authenticated user.

        Args:
            request (Request): The HTTP request object.

        Returns:
            Response: The HTTP response containing the list of game IDs.
        """
        game_ids = list(request.user.played_games.filter(finished = True, has_bots = False).values_list('id', flat=True))

        return Response({'games': game_ids}, status=status.HTTP_200_OK)
```

2.1.6 `get(request)`

Handles the request to list all games played by the authenticated user.

Parameters:

Name	Type	Description	Default
<code>request</code>	<code>Request</code>	The HTTP request object.	<i>required</i>

Returns:

Name	Type	Description
------	------	-------------

Response	Response	The HTTP response containing the list of game IDs.
----------	----------	--

- **Source code in** `magnate/views.py`

```
def get(self, request) -> Response:
    """
    Handles the request to list all games played by the authenticated user.

    Args:
        request (Request): The HTTP request object.

    Returns:
        Response: The HTTP response containing the list of game IDs.
    """
    game_ids = list(request.user.played_games.filter(finished = True, has_bots = False).values_list('id', flat=True))

    return Response({'games': game_ids}, status=status.HTTP_200_OK)
```

User Name and PieceBases: `APIView`

Returns the username and active piece of the requested user.

GET /info/user-name-piece//

- **Responses**

200: Returns username and piece. 404: User not found.

- **Source code in** `magnate/views.py`

```
class UserNamePieceView(APIView):
    """
    Returns the username and active piece of the requested user.

    GET /info/user-name-piece/<pk>/

    Responses:
        200: Returns username and piece.
        404: User not found.
    """

    permission_classes = [AllowAny]

    def get(self, request, pk) -> Response:
        """
        Handles the request to get a specific user's username and piece.

        Args:
            request (Request): The HTTP request object.
```

```
pk (int): The primary key of the user.

Returns:
    Response: The HTTP response containing the user's name and piece.
"""
user = get_object_or_404(CustomUser, pk=pk)
return Response({
    'username': user.username,
    'piece':    user.user_piece.custom_id,
}, status=status.HTTP_200_OK)
```

2.1.7 `get(request, pk)`

Handles the request to get a specific user's username and piece.

Parameters:

Name	Type	Description	Default
<code>request</code>	<code>Request</code>	The HTTP request object.	<i>required</i>
<code>pk</code>	<code>int</code>	The primary key of the user.	<i>required</i>

Returns:

Name	Type	Description
<code>Response</code>	<code>Response</code>	The HTTP response containing the user's name and piece.

• Source code in `magnate/views.py`

```
def get(self, request, pk) -> Response:
    """
    Handles the request to get a specific user's username and piece.

    Args:
        request (Request): The HTTP request object.
        pk (int): The primary key of the user.

    Returns:
        Response: The HTTP response containing the user's name and piece.
    """
    user = get_object_or_404(CustomUser, pk=pk)
    return Response({
        'username': user.username,
        'piece':    user.user_piece.custom_id,
    }, status=status.HTTP_200_OK)
```

2.1.8 Shop & Customization

Shop Items

Bases: `APIView`

Returns the full list of available shop items. Each item includes an 'owned' flag for the requesting user.

GET `/shop/items/`

- **Headers**
Authorization: Bearer

- **Responses**
200: List of all items with id, itemType, price and owned flag. 401: Missing or invalid token.

• **Source code in** `magnate/views.py`

```
class ShopItemListView(APIView):
    """
    Returns the full list of available shop items.
    Each item includes an 'owned' flag for the requesting user.

    GET /shop/items/

    Headers:
        Authorization: Bearer <access_token>

    Responses:
        200: List of all items with id, itemType, price and owned flag.
        401: Missing or invalid token.
    """

    permission_classes = [IsAuthenticated]

    def get(self, request) -> Response:
        """
        Handles the request to list available shop items.

        Args:
            request (Request): The HTTP request object.

        Returns:
            Response: The HTTP response containing the list of items.
        """
        items = Item.objects.all()
        serializer = ItemSerializer(items, many=True, context={'request': request})
        return Response(serializer.data, status=status.HTTP_200_OK)
```

2.1.9 `get(request)`

Handles the request to list available shop items.

Parameters:

Name	Type	Description	Default
<code>request</code>	<code>Request</code>	The HTTP request object.	<i>required</i>

Returns:

Name	Type	Description
Response	Response	The HTTP response containing the list of items.

- **Source code in** `magnate/views.py`

```
def get(self, request) -> Response:
    """
    Handles the request to list available shop items.

    Args:
        request (Request): The HTTP request object.

    Returns:
        Response: The HTTP response containing the list of items.
    """
    items = Item.objects.all()
    serializer = ItemSerializer(items, many=True, context={'request': request})
    return Response(serializer.data, status=status.HTTP_200_OK)
```

Buy ItemBases: `APIView`

Purchases an item for the authenticated user. Deducts the item price from the user's points and adds it to owned_items.

POST /shop/buy/

- **Headers**

Authorization: Bearer

- **Request body**

{ "item_id": 1 }

- **Responses**

200: Purchase successful. Returns updated points and item data. 400: Item not found, already owned, or insufficient points. 401: Missing or invalid token.

- **Source code in** `magnate/views.py`

```
class BuyItemView(APIView):
    """
    Purchases an item for the authenticated user.
    Deducts the item price from the user's points and adds it to owned_items.
```

```

POST /shop/buy/

Headers:
    Authorization: Bearer <access_token>

Request body:
    { "item_id": 1 }

Responses:
    200: Purchase successful. Returns updated points and item data.
    400: Item not found, already owned, or insufficient points.
    401: Missing or invalid token.
"""

permission_classes = [IsAuthenticated]

def post(self, request) -> Response:
    """
    Handles the request to purchase a shop item.

    Args:
        request (Request): The HTTP request object containing the item identifier.

    Returns:
        Response: The HTTP response indicating purchase success or failure.
    """
    serializer = PurchaseSerializer(data=request.data, context={'request': request})
    if not serializer.is_valid():
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    data: dict = serializer.validated_data # type: ignore
    item = Item.objects.get(id=data['custom_id'])

    user: CustomUser = request.user # type: ignore
    user.points -= item.price # money check already done in serializer
    user.save()
    user.owned_items.add(item)

    return Response({
        'message': 'Purchase successful.',
        'item': ItemSerializer(item, context={'request': request}).data,
        'points_remaining': user.points,
    }, status=status.HTTP_200_OK)

```

2.1.10 `post(request)`

Handles the request to purchase a shop item.

Parameters:

Name	Type	Description	Default
<code>request</code>	<code>Request</code>	The HTTP request object containing the item identifier.	<i>required</i>

Returns:

Name	Type	Description
Response	Response	The HTTP response indicating purchase success or failure.

- **Source code in** `magnate/views.py`

```
def post(self, request) -> Response:
    """
    Handles the request to purchase a shop item.

    Args:
        request (Request): The HTTP request object containing the item identifier.

    Returns:
        Response: The HTTP response indicating purchase success or failure.
    """
    serializer = PurchaseSerializer(data=request.data, context={'request': request})
    if not serializer.is_valid():
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    data: dict = serializer.validated_data # type: ignore
    item = Item.objects.get(id=data['custom_id'])

    user: CustomUser = request.user # type: ignore
    user.points -= item.price # money check already done in serializer
    user.save()
    user.owned_items.add(item)

    return Response({
        'message': 'Purchase successful.',
        'item': ItemSerializer(item, context={'request': request}).data,
        'points_remaining': user.points,
    }, status=status.HTTP_200_OK)
```

User Owned PiecesBases: `APIView`

Returns the list of pieces owned by the authenticated user.

GET /shop/user-pieces/

- **Headers**

Authorization: Bearer

- **Responses**

200: List of owned pieces with id, itemType, price and owned flag (always true). 401: Missing or invalid token.

• **Source code in** `magnate/views.py`

```
class UserPiecesView(APIView):
    """
    Returns the list of pieces owned by the authenticated user.

    GET /shop/user-pieces/

    Headers:
        Authorization: Bearer <access_token>

    Responses:
        200: List of owned pieces with id, itemType, price and owned flag (always true).
        401: Missing or invalid token.
    """

    permission_classes = [IsAuthenticated]

    def get(self, request) -> Response:
        """
        Handles the request to list pieces owned by the user.

        Args:
            request (Request): The HTTP request object.

        Returns:
            Response: The HTTP response containing the list of owned pieces.
        """
        user: CustomUser = request.user # type: ignore
        pieces = user.owned_items.filter(itemType='piece')
        serializer = ItemSerializer(pieces, many=True, context={'request': request})
        return Response(serializer.data, status=status.HTTP_200_OK)
```

2.1.11 `get(request)`

Handles the request to list pieces owned by the user.

Parameters:

Name	Type	Description	Default
<code>request</code>	<code>Request</code>	The HTTP request object. <i>required</i>	

Returns:

Name	Type	Description
<code>Response</code>	<code>Response</code>	The HTTP response containing the list of owned pieces.

• **Source code in** `magnate/views.py`

```
def get(self, request) -> Response:
    """
    Handles the request to list pieces owned by the user.

    Args:
        request (Request): The HTTP request object.
```

Returns:

```

    Response: The HTTP response containing the list of owned pieces.
    """
    user: CustomUser = request.user # type: ignore
    pieces = user.owned_items.filter(itemType='piece')
    serializer = ItemSerializer(pieces, many=True, context={'request': request})
    return Response(serializer.data, status=status.HTTP_200_OK)

```

User Owned Emojis

Bases: `APIView`

Returns the list of emojis owned by the authenticated user.

GET /shop/user-emojis/

• Headers

Authorization: Bearer

• Responses

200: List of owned emojis with id, itemType, price and owned flag (always true). 401: Missing or invalid token.

• Source code in `magnate/views.py`

```

class UserEmojisView(APIView):
    """
    Returns the list of emojis owned by the authenticated user.

    GET /shop/user-emojis/

    Headers:
        Authorization: Bearer <access_token>

    Responses:
        200: List of owned emojis with id, itemType, price and owned flag (always true).
        401: Missing or invalid token.
    """

    permission_classes = [IsAuthenticated]

    def get(self, request) -> Response:
        """
        Handles the request to list emojis owned by the user.

        Args:
            request (Request): The HTTP request object.

        Returns:
            Response: The HTTP response containing the list of owned emojis.
        """
        user: CustomUser = request.user # type: ignore

```

```
emojis = user.owned_items.filter(itemType='emoji')
serializer = ItemSerializer(emojis, many=True, context={'request': request})
return Response(serializer.data, status=status.HTTP_200_OK)
```

2.1.12 `get(request)`

Handles the request to list emojis owned by the user.

Parameters:

Name	Type	Description	Default
<code>request</code>	Request	The HTTP request object.	<i>required</i>

Returns:

Name	Type	Description
<code>Response</code>	Response	The HTTP response containing the list of owned emojis.

• Source code in `magnate/views.py`

```
def get(self, request) -> Response:
    """
    Handles the request to list emojis owned by the user.

    Args:
        request (Request): The HTTP request object.

    Returns:
        Response: The HTTP response containing the list of owned emojis.
    """
    user: CustomUser = request.user # type: ignore
    emojis = user.owned_items.filter(itemType='emoji')
    serializer = ItemSerializer(emojis, many=True, context={'request': request})
    return Response(serializer.data, status=status.HTTP_200_OK)
```

Change Active Piece

Bases: `APIView`

Changes the user's active piece to one they own.

POST `/user/change-piece/`

• Headers

Authorization: Bearer

• Request body

```
{ "custom_id": 1 }
```

• Responses

200: Piece changed successfully. Returns the new user_piece value. 400: Validation error (item not found, not owned, not a piece). 401: Missing or invalid token.

• Source code in `magnate/views.py`

```
class ChangeUserPieceView(APIView):
    """
    Changes the user's active piece to one they own.

    POST /user/change-piece/

    Headers:
        Authorization: Bearer <access_token>

    Request body:
        { "custom_id": 1 }

    Responses:
        200: Piece changed successfully. Returns the new user_piece value.
        400: Validation error (item not found, not owned, not a piece).
        401: Missing or invalid token.
    """

    permission_classes = [IsAuthenticated]

    def post(self, request) -> Response:
        """
        Handles the request to change the user's active piece.

        Args:
            request (Request): The HTTP request object containing the piece identifier.

        Returns:
            Response: The HTTP response indicating success or failure.
        """
        serializer = ChangePieceSerializer(data=request.data, context={'request': request})
        if not serializer.is_valid():
            return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

        data: dict = serializer.validated_data # type: ignore
        user: CustomUser = request.user # type: ignore
        item = Item.objects.get(custom_id=data['custom_id'])
        user.user_piece = item
        user.save()

        return Response({
            'message': 'Piece changed successfully.',
            'user_piece': user.user_piece.custom_id,
        }, status=status.HTTP_200_OK)
```

2.1.13 `post(request)`

Handles the request to change the user's active piece.

Parameters:

Name	Type	Description	Default
request	Request	The HTTP request object containing the piece identifier.	<i>required</i>

Returns:

Name	Type	Description
Response	Response	The HTTP response indicating success or failure.

- **Source code in** `magnate/views.py`

```
def post(self, request) -> Response:
    """
    Handles the request to change the user's active piece.

    Args:
        request (Request): The HTTP request object containing the piece identifier.

    Returns:
        Response: The HTTP response indicating success or failure.
    """
    serializer = ChangePieceSerializer(data=request.data, context={'request': request})
    if not serializer.is_valid():
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    data: dict = serializer.validated_data # type: ignore
    user: CustomUser = request.user # type: ignore
    item = Item.objects.get(custom_id=data['custom_id'])
    user.user_piece = item
    user.save()

    return Response({
        'message': 'Piece changed successfully.',
        'user_piece': user.user_piece.custom_id,
    }, status=status.HTTP_200_OK)
```

2.1.14 Lobby & Matchmaking

Get Private Code

Bases: `APIView`

Generates a unique 6-character alphanumeric private room code.

GET /lobby/get-private-code

- **Headers**

Authorization: Bearer

• Responses

200: Returns a unique private room code. 401: Missing or invalid token.

• Source code in `magnate/views.py`

```
class GetPrivateCodeView(APIView):
    """
    Generates a unique 6-character alphanumeric private room code.

    GET /lobby/get-private-code

    Headers:
        Authorization: Bearer <access_token>

    Responses:
        200: Returns a unique private room code.
        401: Missing or invalid token.
    """

    permission_classes = [IsAuthenticated]

    def get(self, request) -> Response:
        """
        Handles the request to generate a unique private room code.

        Args:
            request (Request): The HTTP request object.

        Returns:
            Response: The HTTP response containing the generated code.
        """
        code = self._generate_unique_code()
        return Response({
            'code': code,
            'message': 'Private room code generated successfully.'
        }, status=status.HTTP_200_OK)

#Existe una pequeña y despreciable posibilidad de que lleguen
#2 peticiones muy seguidas y por mala suerte les dé el mismo número
#aleatorio. En fin, srand() es mi pastor nada me falta.
@staticmethod
def _generate_unique_code() -> str:
    """
    Generates a unique 6-character alphanumeric code in uppercase.
    Ensures no existing PrivateRoom is currently using this code in the database.

    Args:
        None

    Returns:
        str: The generated 6-character unique code.
    """
    characters = string.ascii_uppercase + string.digits
    while True:
        code = ''.join(random.choices(characters, k=6))
```

```
if not PrivateRoom.objects.filter(room_code=code).exists():
    return code
```

2.1.15 `get(request)`

Handles the request to generate a unique private room code.

Parameters:

Name	Type	Description	Default
<code>request</code>	<code>Request</code>	The HTTP request object. <i>required</i>	

Returns:

Name	Type	Description
<code>Response</code>	<code>Response</code>	The HTTP response containing the generated code.

• Source code in `magnate/views.py`

```
def get(self, request) -> Response:
    """
    Handles the request to generate a unique private room code.

    Args:
        request (Request): The HTTP request object.

    Returns:
        Response: The HTTP response containing the generated code.
    """
    code = self._generate_unique_code()
    return Response({
        'code': code,
        'message': 'Private room code generated successfully.'
    }, status=status.HTTP_200_OK)
```

Check Room Code

Bases: `APIView`

Checks if a private room with the given code exists.

GET `/lobby/check-code/`

• Headers

Authorization: Bearer

• Responses

200: Returns a boolean indicating if the room exists (`{'exists': true/false}`). 401: Missing or invalid token.

• Source code in `magnate/views.py`

```
class CheckPrivateRoomView(APIView):
    """
    Checks if a private room with the given code exists.

    GET /lobby/check-code/<room_code>/

    Headers:
        Authorization: Bearer <access_token>

    Responses:
        200: Returns a boolean indicating if the room exists ({'exists': true/false}).
        401: Missing or invalid token.
    """
    permission_classes = [IsAuthenticated]

    def get(self, request, room_code) -> Response:
        """
        Handles the request to check if a private room exists.

        Args:
            request (Request): The HTTP request object.
            room_code (str): The code of the room to check.

        Returns:
            Response: The HTTP response containing the existence check result.
        """
        exists = PrivateRoom.objects.filter(room_code=room_code).exists()
        return Response({'exists': exists}, status=status.HTTP_200_OK)
```

2.1.16 `get(request, room_code)`

Handles the request to check if a private room exists.

Parameters:

Name	Type	Description	Default
<code>request</code>	<code>Request</code>	The HTTP request object.	<i>required</i>
<code>room_code</code>	<code>str</code>	The code of the room to check.	<i>required</i>

Returns:

Name	Type	Description
<code>Response</code>	<code>Response</code>	The HTTP response containing the existence check result.

• Source code in `magnate/views.py`

```
def get(self, request, room_code) -> Response:
    """
    Handles the request to check if a private room exists.

    Args:
        request (Request): The HTTP request object.
        room_code (str): The code of the room to check.
```


Returns:

```

    Response: The HTTP response containing the existence check result.
    """
    exists = PrivateRoom.objects.filter(room_code=room_code).exists()
    return Response({'exists': exists}, status=status.HTTP_200_OK)

```

2.1.17 Game Summary

Game Summary

Bases: `APIView`

Returns a summary of a completed game, including start/end times and final money for each player.

GET /game/summary//

• Headers

Authorization: Bearer

• Responses

200: Returns game summary with start_date, end_date, and final_money dict. 401: Missing or invalid token.

403: User did not participate in this game. 404: Game summary not found.

• Source code in `magnate/views.py`

```

class GetGameSummaryView(APIView):
    """
    Returns a summary of a completed game, including start/end times and final money for each player.

    GET /game/summary/<game_id>/

    Headers:
        Authorization: Bearer <access_token>

    Responses:
        200: Returns game summary with start_date, end_date, and final_money dict.
        401: Missing or invalid token.
        403: User did not participate in this game.
        404: Game summary not found.
    """
    permission_classes = [IsAuthenticated]

    def get(self, request, game_id) -> Response:
        """
        Handles the request to retrieve a summary for a specific game.

        Args:
            request (Request): The HTTP request object.
            game_id (int): The identifier of the game.

        Returns:

```

```

        Response: The HTTP response containing the game summary or an error.
    """
    summary = get_object_or_404(GameSummary, game_id=game_id)

    if not request.user.played_games.filter(id=game_id).exists():
        return Response(
            {'error': 'No tienes permiso para ver el resumen de una partida en la que no participaste.'},
            status=status.HTTP_403_FORBIDDEN
        )

    serializer = GameSummarySerializer(summary)
    return Response(serializer.data, status=status.HTTP_200_OK)

```

2.1.18 `get(request, game_id)`

Handles the request to retrieve a summary for a specific game.

Parameters:

Name	Type	Description	Default
<code>request</code>	<code>Request</code>	The HTTP request object.	<i>required</i>
<code>game_id</code>	<code>int</code>	The identifier of the game.	<i>required</i>

Returns:

Name	Type	Description
<code>Response</code>	<code>Response</code>	The HTTP response containing the game summary or an error.

• Source code in `magnate/views.py`

```

def get(self, request, game_id) -> Response:
    """
    Handles the request to retrieve a summary for a specific game.

    Args:
        request (Request): The HTTP request object.
        game_id (int): The identifier of the game.

    Returns:
        Response: The HTTP response containing the game summary or an error.
    """
    summary = get_object_or_404(GameSummary, game_id=game_id)

    if not request.user.played_games.filter(id=game_id).exists():
        return Response(
            {'error': 'No tienes permiso para ver el resumen de una partida en la que no participaste.'},
            status=status.HTTP_403_FORBIDDEN
        )

    serializer = GameSummarySerializer(summary)
    return Response(serializer.data, status=status.HTTP_200_OK)

```

2.2 WebSockets

2.2.1 PublicQueueConsumer

Bases: AsyncWebsocketConsumer

WebSocket consumer for the public matchmaking queue.

Handles automatic matchmaking: players connect, wait until enough players are in the queue, and are then redirected to a newly created game.

How to Connect

Endpoint: `ws://<host>/ws/queue/`

Authentication required: Yes — the user must be authenticated via session or token before connecting.

Unauthenticated connections are closed with code `4002`.

Connection Lifecycle

1. Client opens the WebSocket.
2. Server adds the user to the matchmaking queue.
3. If enough players are queued (`NUM_PUBLIC_GAME_PLAYERS`), the server automatically creates a game and sends a `match_found` event to every matched player.
4. Each matched client receives the `game_id` and should navigate to the game screen, then connect to `GameConsumer`.

Messages: Client → Server

Cancel Queue

Voluntarily leave the matchmaking queue.

```
{ "action": "cancel" }
```

Messages: Server → Client

Match Found

Sent when matchmaking succeeds. The connection is closed with code `4001` immediately after this message.

```
{
  "action": "match_found",
  "game_id": 42
}
```

Error

Sent when an invalid message is received.

```
{
  "action": "error",
}
```

```
"message": "Datos invalidos."
}
```

Close Codes

Code Meaning

4001 Match found — user was moved to a game. Connect to `GameConsumer`.

4002 Unauthorized — user was not authenticated.

4000 User cancelled the queue.

Example Flow (JavaScript)

```
const socket = new WebSocket("ws://localhost:8000/ws/queue/");

socket.onmessage = (event) => {
  const data = JSON.parse(event.data);
  if (data.action === "match_found") {
    // Navigate to game with data.game_id
    connectToGame(data.game_id);
  }
};

// To leave the queue manually:
socket.send(JSON.stringify({ action: "cancel" }));
```

• Source code in `magnate/consumers.py`

```
class PublicQueueConsumer(AsyncWebsocketConsumer):
    """
    WebSocket consumer for the public matchmaking queue.

    Handles automatic matchmaking: players connect, wait until enough players
    are in the queue, and are then redirected to a newly created game.

    ---
    ## How to Connect

    **Endpoint:** ``ws://<host>/ws/queue/``

    **Authentication required:** Yes — the user must be authenticated via session
    or token before connecting. Unauthenticated connections are closed with code ``4002``.

    ---
    ## Connection Lifecycle

    1. Client opens the WebSocket.
    2. Server adds the user to the matchmaking queue.
    3. If enough players are queued (``NUM_PUBLIC_GAME_PLAYERS``), the server
       automatically creates a game and sends a ``match_found`` event to every
       matched player.
    4. Each matched client receives the ``game_id`` and should navigate to the
       game screen, then connect to ``GameConsumer``.

    ---
    ## Messages: Client → Server
```

```

### Cancel Queue
Voluntarily leave the matchmaking queue.

```json
{ "action": "cancel" }
```

---
## Messages: Server → Client

### Match Found
Sent when matchmaking succeeds. The connection is closed with code ``4001``
immediately after this message.

```json
{
 "action": "match_found",
 "game_id": 42
}
```

### Error
Sent when an invalid message is received.

```json
{
 "action": "error",
 "message": "Datos invalidos."
}
```

---
## Close Codes

Code	Meaning
4001	Match found – user was moved to a game. Connect to ``GameConsumer``.
4002	Unauthorized – user was not authenticated.
4000	User cancelled the queue.

---
## Example Flow (JavaScript)

```js
const socket = new WebSocket("ws://localhost:8000/ws/queue/");

socket.onmessage = (event) => {
 const data = JSON.parse(event.data);
 if (data.action === "match_found") {
 // Navigate to game with data.game_id
 connectToGame(data.game_id);
 }
};

// To leave the queue manually:
socket.send(JSON.stringify({ action: "cancel" }));
```

```

```

QUEUE_GROUP = "public_queue"

async def connect(self):
    """
    Handles initial WebSocket connection.
    Validates authentication, accepts the connection, adds the user to the
    public queue group, and triggers matchmaking logic.

    Args:
        None

    Returns:
        None
    """
    scope_user = self.scope.get('user')
    if scope_user is None or getattr(scope_user, 'is_anonymous', True):
        await self.close(code=4002)
        return

    self.user = await database_sync_to_async(CustomUser.objects.get)(pk=scope_user.pk)
    await self.accept()

    # join queue
    await self.channel_layer.group_add(self.QUEUE_GROUP, self.channel_name)

    await self.add_user_to_queue(self.user, self.channel_name)

    # checking
    match_result = await self.matchmaking_logic()

    if match_result:
        game_id, players_channels = match_result
        for channel in players_channels:
            await self.channel_layer.send(
                channel,
                {
                    'type': 'match_found_event',
                    'game_id': game_id,
                }
            )

async def disconnect(self, close_code):
    """
    Handles WebSocket disconnection.
    Removes the user from the public queue group and the database queue
    unless they were disconnected because a match was found.

    Args:
        close_code (int): The WebSocket close code.

    Returns:
        None
    """
    await self.channel_layer.group_discard(self.QUEUE_GROUP, self.channel_name)

    if close_code == 4001:
        print(f"User {self.user} found a match and is leaving the queue.")
    elif close_code == 4002:

```

```

        return
    else:
        print(f"User {self.user} left the queue.")
        await self.remove_user_from_queue(self.user)

async def receive(self, text_data):
    """
    Processes incoming messages from the client.
    Currently only supports the 'cancel' action to leave the queue.

    Args:
        text_data (str): The raw JSON string received.

    Returns:
        None
    """
    try:
        data = json.loads(text_data)
        if data.get('action') == 'cancel':
            await self.close(code=4000)
    except json.JSONDecodeError:
        await self.send_error("Datos invalidos.")

async def match_found_event(self, event):
    """
    Handler for the 'match_found_event' broadcast.
    Sends the game ID to the client.

    Args:
        event (dict): The event payload containing 'game_id'.

    Returns:
        None
    """
    await self.send(text_data=json.dumps({
        'action': 'match_found',
        'game_id': event['game_id']
    }))

async def send_error(self, message):
    """
    Utility method to send an error message to the client.

    Args:
        message (str): The error message to be sent.

    Returns:
        None
    """
    await self.send(text_data=json.dumps({
        'action': 'error',
        'message': message
    }))

# ----- DB access methods -----#
@database_sync_to_async
def add_user_to_queue(self, user, channel_name):
    """

```

Adds a user to the PublicQueuePosition table if they are not already present.

Args:

user (CustomUser): The user to add.
channel_name (str): The specific channel name for this connection.

Returns:

None

```
"""
existing_position = PublicQueuePosition.objects.filter(user=user).first()
if existing_position:
    # User already in queue, don't add again
    return None

# Create queue position for the user
PublicQueuePosition.objects.create(
    user=user,
    channel = channel_name,
    date_time=timezone.now()

)
```

@database_sync_to_async

def remove_user_from_queue(self, user):

"""

Removes a user's entry from the PublicQueuePosition table.

Args:

user (CustomUser): The user to remove.

Returns:

None

```
"""
existing_position = PublicQueuePosition.objects.filter(user=user).first()
if existing_position:
    existing_position.delete()
else:
    # User not in queue
    return None
```

TODO: be aware of race conditions -> new users while executing the method

@database_sync_to_async

def matchmaking_logic(self):

"""

Checks if there are enough players in the queue. If so, creates a new game, assigns starting positions and funds, randomizes the turn order, and removes the users from the public queue.

Args:

None

Returns:

tuple | None: A tuple containing the new game's primary key (`game\_id`) and a list of player channel names if a match is successfully made.
Returns `None` if there are not enough players in the queue.

"""

with transaction.atomic():


```

players = list(PublicQueuePosition.objects.select_for_update().order_by('date_time')[:NUM_PUBLIC_GAME_PLA

if len(players) < NUM_PUBLIC_GAME_PLAYERS:
    return None

player_channels = [player.channel for player in players]
users = [player.user for player in players]

# create with compulsory sh -> TODO: change to random
game = Game.objects.create(
    datetime=timezone.now(),
    active_turn_player=users[0],
    active_phase_player=users[0],
    phase=GameManager.ROLL_THE_DICES
)

# initialize money and positions (see then what the optimal money)
game.money = {str(u.pk): 3000 for u in users}
game.positions = {str(u.pk): 0 for u in users}

game.players.set(users)
game.ordered_players = [u.pk for u in users]
game.ordered_players = random.sample(game.ordered_players, len(game.ordered_players)) #random order of p

#task = kick_out_callback.apply_async(args=[game.pk, users[0].pk], countdown=50) #necessary for first tur
#game.kick_out_task_id = task.id
game.save()

for user in users:
    user.active_game = game
    user.played_games.add(game)
    user.save()

PublicQueuePosition.objects.filter(pk__in=[p.pk for p in players]).delete()

return (game.pk, player_channels)

```

```
add_user_to_queue(user, channel_name)
```

Adds a user to the PublicQueuePosition table if they are not already present.

Parameters:

| Name | Type | Description | Default |
|---------------------------|-------------------------|--|-----------------|
| <code>user</code> | <code>CustomUser</code> | The user to add. | <i>required</i> |
| <code>channel_name</code> | <code>str</code> | The specific channel name for this connection. | <i>required</i> |

Returns:

Type Description

None

• **Source code in** `magnate/consumers.py`

```
@database_sync_to_async
def add_user_to_queue(self, user, channel_name):
    """
    Adds a user to the PublicQueuePosition table if they are not already present.

    Args:
        user (CustomUser): The user to add.
        channel_name (str): The specific channel name for this connection.

    Returns:
        None
    """
    existing_position = PublicQueuePosition.objects.filter(user=user).first()
    if existing_position:
        # User already in queue, don't add again
        return None

    # Create queue position for the user
    PublicQueuePosition.objects.create(
        user=user,
        channel = channel_name,
        date_time=timezone.now()

    )
```

`connect()` `async`

Handles initial WebSocket connection. Validates authentication, accepts the connection, adds the user to the public queue group, and triggers matchmaking logic.

Returns:

Type Description

None

• **Source code in** `magnate/consumers.py`

```
async def connect(self):
    """
    Handles initial WebSocket connection.
    Validates authentication, accepts the connection, adds the user to the
    public queue group, and triggers matchmaking logic.

    Args:
        None

    Returns:
        None
    """
    scope_user = self.scope.get('user')
    if scope_user is None or getattr(scope_user, 'is_anonymous', True):
        await self.close(code=4002)
        return
```

```

self.user = await database_sync_to_async(CustomUser.objects.get)(pk=scope_user.pk)
await self.accept()

# join queue
await self.channel_layer.group_add(self.QUEUE_GROUP, self.channel_name)

await self.add_user_to_queue(self.user, self.channel_name)

# checking
match_result = await self.matchmaking_logic()

if match_result:
    game_id, players_channels = match_result
    for channel in players_channels:
        await self.channel_layer.send(
            channel,
            {
                'type': 'match_found_event',
                'game_id': game_id,
            }
        )

```

`disconnect(close_code)` `async`

Handles WebSocket disconnection. Removes the user from the public queue group and the database queue unless they were disconnected because a match was found.

Parameters:

| Name | Type | Description | Default |
|-------------------------|------------------|---|---------|
| <code>close_code</code> | <code>int</code> | The WebSocket close code. <i>required</i> | |

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```

async def disconnect(self, close_code):
    """
    Handles WebSocket disconnection.
    Removes the user from the public queue group and the database queue
    unless they were disconnected because a match was found.

    Args:
        close_code (int): The WebSocket close code.

    Returns:
        None
    """
    await self.channel_layer.group_discard(self.QUEUE_GROUP, self.channel_name)

    if close_code == 4001:
        print(f"User {self.user} found a match and is leaving the queue.")
    elif close_code == 4002:
        return

```

```

else:
    print(f"User {self.user} left the queue.")
    await self.remove_user_from_queue(self.user)

```

```
match_found_event(event) async
```

Handler for the 'match_found_event' broadcast. Sends the game ID to the client.

Parameters:

| Name | Type | Description | Default |
|-------|------|---|---------|
| event | dict | The event payload containing 'game_id'. <i>required</i> | |

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```

async def match_found_event(self, event):
    """
    Handler for the 'match_found_event' broadcast.
    Sends the game ID to the client.

    Args:
        event (dict): The event payload containing 'game_id'.

    Returns:
        None
    """
    await self.send(text_data=json.dumps({
        'action': 'match_found',
        'game_id': event['game_id']
    }))

```

```
matchmaking_logic()
```

Checks if there are enough players in the queue. If so, creates a new game, assigns starting positions and funds, randomizes the turn order, and removes the users from the public queue.

Returns:

Type Description

tuple | None: A tuple containing the new game's primary key (`game_id`) and a list of player channel names if a match is successfully made. Returns `None` if there are not enough players in the queue.

• Source code in `magnate/consumers.py`

```

@database_sync_to_async
def matchmaking_logic(self):
    """
    Checks if there are enough players in the queue. If so, creates a new game,
    assigns starting positions and funds, randomizes the turn order, and removes

```

the users from the public queue.

Args:

None

Returns:

tuple | None: A tuple containing the new game's primary key (`game\_id`) and a list of player channel names if a match is successfully made. Returns `None` if there are not enough players in the queue.

"""

```
with transaction.atomic():

    players = list(PublicQueuePosition.objects.select_for_update().order_by('date_time')[:NUM_PUBLIC_GAME_PLAYERS])

    if len(players) < NUM_PUBLIC_GAME_PLAYERS:
        return None

    player_channels = [player.channel for player in players]
    users = [player.user for player in players]

    # create with compulsory sh -> TODO: change to random
    game = Game.objects.create(
        datetime=timezone.now(),
        active_turn_player=users[0],
        active_phase_player=users[0],
        phase=GameManager.ROLL_THE_DICES
    )

    # initialize money and positions (see then what the optimal money)
    game.money = {str(u.pk): 3000 for u in users}
    game.positions = {str(u.pk): 0 for u in users}

    game.players.set(users)
    game.ordered_players = [u.pk for u in users]
    game.ordered_players = random.sample(game.ordered_players, len(game.ordered_players)) #random order of players

    #task = kick_out_callback.apply_async(args=[game.pk, users[0].pk], countdown=50) #necessary for first turn
    #game.kick_out_task_id = task.id
    game.save()

    for user in users:
        user.active_game = game
        user.played_games.add(game)
        user.save()

    PublicQueuePosition.objects.filter(pk__in=[p.pk for p in players]).delete()

    return (game.pk, player_channels)
```

receive(text_data) async

Processes incoming messages from the client. Currently only supports the 'cancel' action to leave the queue.

Parameters:

| Name | Type | Description | Default |
|-----------|------|-------------------------------|-----------------|
| text_data | str | The raw JSON string received. | <i>required</i> |

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```
async def receive(self, text_data):
    """
    Processes incoming messages from the client.
    Currently only supports the 'cancel' action to leave the queue.

    Args:
        text_data (str): The raw JSON string received.

    Returns:
        None
    """
    try:
        data = json.loads(text_data)
        if data.get('action') == 'cancel':
            await self.close(code=4000)
    except json.JSONDecodeError:
        await self.send_error("Datos invalidos.")
```

`remove_user_from_queue(user)`

Removes a user's entry from the PublicQueuePosition table.

Parameters:

| Name | Type | Description | Default |
|------|------------|---------------------|-----------------|
| user | CustomUser | The user to remove. | <i>required</i> |

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```
@database_sync_to_async
def remove_user_from_queue(self, user):
    """
    Removes a user's entry from the PublicQueuePosition table.

    Args:
        user (CustomUser): The user to remove.

    Returns:
        None
```

```

"""
existing_position = PublicQueuePosition.objects.filter(user=user).first()
if existing_position:
    existing_position.delete()
else:
    # User not in queue
    return None

```

`send_error(message)` `async`

Utility method to send an error message to the client.

Parameters:

| Name | Type | Description | Default |
|----------------------|------------------|-------------------------------|-----------------|
| <code>message</code> | <code>str</code> | The error message to be sent. | <i>required</i> |

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```

async def send_error(self, message):
    """
    Utility method to send an error message to the client.

    Args:
        message (str): The error message to be sent.

    Returns:
        None
    """
    await self.send(text_data=json.dumps({
        'action': 'error',
        'message': message
    }))

```

2.2.2 PrivateRoomConsumer

Bases: `AsyncWebsocketConsumer`

WebSocket consumer for private game room lobbies.

Manages the pre-game lobby where a host invites friends, players set their ready status, and the host starts the game when all players are ready. Supports bot-filling up to the configured target player count.

How to Connect

Endpoint: `ws://<host>/ws/room/<room_code>/`

Authentication required: Yes. Unauthenticated users are rejected with code `4002`.

The `room_code` must correspond to an existing `PrivateRoom`. Attempting to join a non-existent or full room will result in an error message followed by close code `4003`.

Connection Lifecycle

1. Client opens the WebSocket with a valid `room_code`.
2. Server validates room existence, capacity, and that the user is not already in the room.
3. On success, the server broadcasts a `joined` lobby update to all members.
4. Players toggle ready status. The host can change settings (bot level, target player count).
5. When all players are ready, the host sends `start_game`.
6. Server creates the game (filling remaining slots with bots if needed) and broadcasts a `game_start` event to everyone.
7. Each client receives the `game_id` and connects to `GameConsumer`.

Messages: Client → Server

All messages are JSON objects with a `command` field.

Toggle Ready Status

```
{ "command": "ready_status", "is_ready": true }
```

Start Game (*host only*)

Requires all players to be ready and at least `MIN_PRIVATE_GAME_PLAYERS` in the room. Bots are added to reach `target_players` if needed.

```
{ "command": "start_game" }
```

Send Chat Message

```
{ "command": "chat_message", "message": "Hello!" }
```

Update Room Settings (*host only*)

Change the target number of players and/or the bot difficulty level.

```
{
  "command": "update_settings",
  "bot_level": "easy",
  "target_players": 4
}
```


Messages: Server → Client

Player Joined

Player Joined

Broadcast to all members when someone connects. Includes current room settings.

```
{
  "action": "joined",
  "user": "alice",
  "owner": "alice",
  "is_owner": true,
  "bot_level": "easy",
  "target_players": 4,
  "players": [
    { "username": "alice", "ready_to_play": false }
  ]
}
```

Player Left

Broadcast to remaining members when someone disconnects. `owner` may change if the previous owner left (host migration).

```
{
  "action": "player_left",
  "user_left": "bob",
  "owner": "alice",
  "is_owner": true,
  "players": [
    { "username": "alice", "ready_to_play": false }
  ]
}
```

Ready Status Update

Broadcast to all members when any player changes their ready status.

```
{
  "action": "ready_status",
  "user": "bob",
  "is_ready": true,
  "owner": "alice",
  "is_owner": false
}
```

Settings Changed

Broadcast to all members when the host updates room settings.

```
{
  "action": "settings_changed",
  "bot_level": "hard",
  "target_players": 4
}
```

Game Start

Broadcast to all members when the game has been created. The connection is closed with code `4001` immediately after.

```
{
  "action": "game_start",
  "game_id": 42
}
```

Chat Message

Broadcast to all members.

```
{
  "action": "chat_message",
  "user": "alice",
  "message": "Good luck!"
}
```

Error

Sent only to the client that triggered the error.

```
{
  "action": "error",
  "message": "Solo el host puede iniciar una partida."
}
```

Close Codes

Code Meaning

4001 Game started — connect to `GameConsumer` with the received `game_id`.

4002 Unauthorized.

4003 Room not found, full, or user already in room.

Notes

- `is_owner` in lobby events is computed per-recipient: `true` only for the current room host.
- If the host disconnects, ownership is automatically transferred to the next oldest player.
- If the last player leaves, the room is deleted.
- Bots are created at game start to fill slots up to `target_players`; they are not visible in the lobby.

Example Flow (JavaScript)

```
const socket = new WebSocket("ws://localhost:8000/ws/room/ABC123/");

socket.onmessage = (event) => {
  const data = JSON.parse(event.data);

  switch (data.action) {
    case "joined":
    case "player_left":
```

```

        updatePlayerList(data.players, data.owner);
        break;
    case "ready_status":
        updateReadyIndicator(data.user, data.is_ready);
        break;
    case "settings_changed":
        updateSettings(data.bot_level, data.target_players);
        break;
    case "game_start":
        connectToGame(data.game_id);
        break;
    case "error":
        showError(data.message);
        break;
}
};

// Mark yourself as ready:
socket.send(JSON.stringify({ command: "ready_status", is_ready: true }));

// Host starts the game:
socket.send(JSON.stringify({ command: "start_game" }));

```

• Source code in `magnate/consumers.py`

```

class PrivateRoomConsumer(AsyncWebsocketConsumer):
    """
    WebSocket consumer for private game room lobbies.

    Manages the pre-game lobby where a host invites friends, players set their
    ready status, and the host starts the game when all players are ready. Supports
    bot-filling up to the configured target player count.

    ---
    ## How to Connect

    **Endpoint:** ``ws://<host>/ws/room/<room_code>/``

    **Authentication required:** Yes. Unauthenticated users are rejected with code ``4002``.

    The ``room_code`` must correspond to an existing ``PrivateRoom``. Attempting to
    join a non-existent or full room will result in an error message followed by
    close code ``4003``.

    ---
    ## Connection Lifecycle

    1. Client opens the WebSocket with a valid ``room_code``.
    2. Server validates room existence, capacity, and that the user is not already in
       the room.
    3. On success, the server broadcasts a ``joined`` lobby update to all members.
    4. Players toggle ready status. The host can change settings (bot level, target
       player count).
    5. When all players are ready, the host sends ``start_game``.
    6. Server creates the game (filling remaining slots with bots if needed) and
       broadcasts a ``game_start`` event to everyone.
    7. Each client receives the ``game_id`` and connects to ``GameConsumer``.

    ---
    """

```

Messages: Client → Server

All messages are JSON objects with a ``command`` field.

Toggle Ready Status

```
```json
{ "command": "ready_status", "is_ready": true }
```
```

Start Game *(host only)*

Requires all players to be ready and at least ``MIN\_PRIVATE\_GAME\_PLAYERS`` in the room. Bots are added to reach ``target\_players`` if needed.

```
```json
{ "command": "start_game" }
```
```

Send Chat Message

```
```json
{ "command": "chat_message", "message": "Hello!" }
```
```

Update Room Settings *(host only)*

Change the target number of players and/or the bot difficulty level.

```
```json
{
 "command": "update_settings",
 "bot_level": "easy",
 "target_players": 4
}
```
```

Messages: Server → Client

Player Joined

Player Joined

Broadcast to all members when someone connects. Includes current room settings.

```
```json
{
 "action": "joined",
 "user": "alice",
 "owner": "alice",
 "is_owner": true,
 "bot_level": "easy",
 "target_players": 4,
 "players": [
 { "username": "alice", "ready_to_play": false }
]
}
```
```

Player Left

Broadcast to remaining members when someone disconnects. ``owner`` may change if the previous owner left (host migration).

```
```json
{
 "action": "player_left",
 "user_left": "bob",
 "owner": "alice",

```

```

 "is_owner": true,
 "players": [
 { "username": "alice", "ready_to_play": false }
]
 }
 ...

Ready Status Update
Broadcast to all members when any player changes their ready status.
```json
{
    "action": "ready_status",
    "user": "bob",
    "is_ready": true,
    "owner": "alice",
    "is_owner": false
}
...

### Settings Changed
Broadcast to all members when the host updates room settings.
```json
{
 "action": "settings_changed",
 "bot_level": "hard",
 "target_players": 4
}
...

Game Start
Broadcast to all members when the game has been created. The connection is
closed with code ``4001`` immediately after.
```json
{
    "action": "game_start",
    "game_id": 42
}
...

### Chat Message
Broadcast to all members.
```json
{
 "action": "chat_message",
 "user": "alice",
 "message": "Good luck!"
}
...

Error
Sent only to the client that triggered the error.
```json
{
    "action": "error",
    "message": "Solo el host puede iniciar una partida."
}
...

---
```

Close Codes

Code	Meaning
4001	Game started – connect to ``GameConsumer`` with the received ``game_id``.
4002	Unauthorized.
4003	Room not found, full, or user already in room.

Notes

- ``is\_owner`` in lobby events is computed per-recipient: ``true`` only for the current room host.
- If the host disconnects, ownership is automatically transferred to the next oldest player.
- If the last player leaves, the room is deleted.
- Bots are created at game start to fill slots up to ``target\_players``; they are not visible in the lobby.

Example Flow (JavaScript)

```js

```
const socket = new WebSocket("ws://localhost:8000/ws/room/ABC123/");
```

```
socket.onmessage = (event) => {
 const data = JSON.parse(event.data);

 switch (data.action) {
 case "joined":
 case "player_left":
 updatePlayerList(data.players, data.owner);
 break;
 case "ready_status":
 updateReadyIndicator(data.user, data.is_ready);
 break;
 case "settings_changed":
 updateSettings(data.bot_level, data.target_players);
 break;
 case "game_start":
 connectToGame(data.game_id);
 break;
 case "error":
 showError(data.message);
 break;
 }
};
```

```
// Mark yourself as ready:
socket.send(JSON.stringify({ command: "ready_status", is_ready: true }));
```

```
// Host starts the game:
socket.send(JSON.stringify({ command: "start_game" }));
```
```

"""

```
async def connect(self):
```

"""

Handles initial WebSocket connection to a private room lobby.
Validates authentication and room code, accepts the connection, and

notifies the group of the new player.

Args:

None

Returns:

None

"""

Triggered when user opens a new private room or joins an existing one.

scope_user = self.scope.get('user')

if scope_user is None or getattr(scope_user, 'is_anonymous', True):

await self.close(code=4002)

return

self.user = await database_sync_to_async(CustomUser.objects.get)(pk=scope_user.pk)

self.url = self.scope.get('url_route')

if self.url is None:

await self.close(code=4002)

return

self.room_code = self.url.get('kwargs').get('room_code')

self.room_group_name = f"lobby_{self.room_code}"

was_created = await self.get_or_create_room_db(self.room_code, self.user)

await self.accept()

if was_created:

```
    await self.send(text_data=json.dumps({
        'action': 'room_created',
        'room_code': self.room_code
    })))
```

can_join, message = await self.check_room(self.room_code)

Case of invalid code or full lobby -> reject connection

if not can_join:

Acept to send error message and close connection

await self.accept()

await self.send(text_data=json.dumps({'error': message}))

await self.close(code=4003)

return

await self.channel_layer.group_add(

self.room_group_name,

self.channel_name

)

room_data = await self.join_room_group_db(self.room_code, self.user)

if not room_data:

await self.close(code=4003)

return

Notify lobby members of new player and update player list

await self.channel_layer.group_send(

```

        self.room_group_name,
        {
            'type': 'lobby_update',
            'action': 'joined',
            'user': self.user.username,
            'players': room_data['players'],
            'owner': room_data['owner'],
            'bot_level': room_data['bot_level'],
            'target_players': room_data['target_players']
        }
    )

async def disconnect(self, close_code):
    """
    Handles WebSocket disconnection from the private lobby.
    Removes the user from the group and handles host migration or room deletion.

    Args:
        close_code (int): The WebSocket close code.

    Returns:
        None
    """
    # Triggered when user leaves the private room lobby.
    # If owner leaves -> change host to the second older player. Else -> just update lobby.
    if close_code == 4002:
        print(f"Unauthorized user attempted to connect and was rejected.")
        return

    await self.channel_layer.group_discard(
        self.room_group_name,
        self.channel_name
    )

    # DB operations -> if needed, rotate host, remove room if empty, etc. Return updated player list and new host
    room_data = await self.leave_room_and_update_host(self.room_code, self.user)

    if not room_data:
        return

    if self.user is None:
        return

    if room_data:
        await self.channel_layer.group_send(
            self.room_group_name,
            {
                'type': 'lobby_update',
                'action': 'player_left',
                'user_left': self.user.username,
                'owner': room_data["owner"],
                'players': room_data['players']
            }
        )

async def receive(self, text_data):
    """

```


Processes incoming messages from lobby clients (chat, ready status, start game, settings).

Args:

text_data (str): The raw JSON string received.

Returns:

None

"""

Chat messages or 'start_game' command / 'ready' status updates.

if self.user is None:

return

try:

data = json.loads(text_data)

command = data.get('command')

if not command:

await self.send_error("Comando invalido.")

return

if command == 'start_game':

is_owner = await self.is_owner(self.user, self.room_code)

if not is_owner:

await self.send_error("Solo el host puede iniciar una partida.")

return

target_players = await self.get_target_players(self.room_code)

if target_players < MIN_PRIVATE_GAME_PLAYERS:

await self.send_error(f"Se necesitan {MIN_PRIVATE_GAME_PLAYERS} jugadores para iniciar la partida")

return

all_ready = await self.check_all_ready(self.room_code)

if not all_ready:

await self.send_error("Todos los jugadores deben estar listos para iniciar la partida.")

return

game_pk = await self.create_private_game(self.room_code)

await self.channel_layer.group_send(

self.room_group_name,

{

'type': 'game_start',

'game_id': game_pk

})

elif command == 'ready_status':

is_ready = data.get('is_ready')

Update in db

await self.update_player_ready_status(self.room_code, self.user, is_ready)

owner = await self.get_room_owner(self.room_code)

await self.channel_layer.group_send(

```

        self.room_group_name,
        {
            'type': 'lobby_update',
            'action': 'ready_status',
            'user': self.user.username,
            'is_ready': is_ready,
            'owner': owner
        }
    )

    elif command == 'chat_message':
        message = data.get('message')
        await self.channel_layer.group_send(
            self.room_group_name,
            {
                'type': 'chat_event',
                'user': self.user.username,
                'message': message
            }
        )

    elif command == 'update_settings': # owner changes target users and bot level
        is_owner = await self.is_owner(self.user, self.room_code)
        if not is_owner:
            await self.send_error("Solo el host puede cambiar la configuración.")
            return

        bot_level = data.get('bot_level')
        target_players = data.get('target_players')

        await self.update_room_settings(self.room_code, bot_level, target_players)

        # Avisar a todos los de la sala del cambio
        await self.channel_layer.group_send(
            self.room_group_name,
            {
                'type': 'lobby_update',
                'action': 'settings_changed',
                'bot_level': bot_level,
                'target_players': target_players
            }
        )

    except json.JSONDecodeError:
        await self.send_error("Datos invalidos.")
        return

# ----- Handlers ----- #
async def lobby_update(self, event):
    """
    Handler for lobby status broadcasts (joined, left, ready, settings).
    Forwards the update to the connected WebSocket client.

    Args:
        event (dict): The event dictionary containing the action and relevant data.

    Returns:
        None

```

```

"""
if self.user is None:
    return

# Send lobby updates to frontend (new player joined, player left)
if event['action'] == 'joined':
    await self.send(text_data=json.dumps({
        'action': event['action'],
        'user': event['user'],
        'owner': event['owner'],
        'is_owner': (self.user.username == event['owner']),
        'players': event['players'],
        'bot_level': event.get('bot_level'),
        'target_players': event.get('target_players')
    }))
elif event['action'] == 'player_left':
    await self.send(text_data=json.dumps({
        'action': event['action'],
        'user_left': event['user_left'],
        'owner': event['owner'],
        'is_owner': (self.user.username == event['owner']),
        'players': event['players']
    }))
elif event['action'] == 'ready_status':
    await self.send(text_data=json.dumps({
        'action': event['action'],
        'user': event['user'],
        'is_ready': event['is_ready'],
        'owner': event['owner'],
        'is_owner': (self.user.username == event['owner'])
    }))
elif event['action'] == 'settings_changed':
    await self.send(text_data=json.dumps({
        'action': event['action'],
        'bot_level': event['bot_level'],
        'target_players': event['target_players']
    }))

@database_sync_to_async
def get_or_create_room_db(self, room_code, user):
    """
    Retrieves an existing room or creates a new one with the given code and owner.

    Args:
        room_code (str): The unique room identifier.
        user (CustomUser): The user creating or retrieving the room.

    Returns:
        bool: True if a new room was created, False if it already existed.
    """
    room = PrivateRoom.objects.filter(room_code=room_code).first()
    if not room:
        PrivateRoom.objects.create(
            room_code=room_code,
            owner=user
        )
        return True
    return False

```

```

@database_sync_to_async
def update_room_settings(self, room_code, bot_level, target_players):
    """
    Updates the target player count and bot difficulty level for a room.

    Args:
        room_code (str): The room identifier.
        bot_level (str): The difficulty level for bots.
        target_players (int): The desired total number of players.

    Returns:
        None
    """
    room = PrivateRoom.objects.get(room_code=room_code)
    if bot_level:
        room.bot_level = bot_level
    if target_players:
        room.target_players = target_players
    room.save()

@database_sync_to_async
def create_private_game(self, room_code):
    """
    Creates a game from a private room, filling empty slots with Bots according
    to the configured difficulty. Randomizes player order, initializes statistics,
    and deletes the lobby.

    Args:
        room_code (str): The unique string identifier for the private room.

    Returns:
        int: The primary key (`pk`) of the newly created Game instance.
    """
    import random
    from django.utils import timezone
    from .models import PrivateRoom, Game, CustomUser
    from .games import GameManager

    room = PrivateRoom.objects.get(room_code=room_code)
    real_users = list(room.players.all())
    users = real_users.copy()

    # fill with bots
    bots = False
    huecos = room.target_players - len(real_users)
    for i in range(huecos):
        bots = True
        bot_username = f"Bot_{room_code}_{i+1}"
        bot_user, _ = Bot.objects.get_or_create(
            username=bot_username
        )

        bot_user.bot_level = room.bot_level
        bot_user.save()

        users.append(bot_user)

```

```

game = Game.objects.create(
    datetime=tmezone.now(),
    active_turn_player=users[0], # Se ajusta abajo
    active_phase_player=users[0],
    phase=GameManager.ROLL_THE_DICES,
    has_bots = bots
)

game.money = {str(u.pk): 3000 for u in users}
game.positions = {str(u.pk): 0 for u in users}

game.players.set(users)
ordered_pks = [u.pk for u in users]
random.shuffle(ordered_pks)
game.ordered_players = ordered_pks

first_player = CustomUser.objects.get(pk=ordered_pks[0])
game.active_turn_player = first_player
game.active_phase_player = first_player

for user in users:
    user.active_game = game
    user.played_games.add(game)
    user.current_private_room = None

    user.save()
    PlayerGameStatistic.objects.get_or_create(user=user, game=game)

room.delete()

# TODO: Make this an option

game.save()

GameManager._set_kick_out_timer(game, first_player)

return game.pk

async def chat_event(self, event):
    """
    Handler for chat messages. Sends the message content to the client.

    Args:
        event (dict): The event dictionary containing 'user' and 'message'.

    Returns:
        None
    """
    # Send chat messages to frontend
    await self.send(text_data=json.dumps({
        'action': 'chat_message',
        'user': event['user'],
        'message': event['message']
    }))

async def game_start(self, event):
    """

```

Handler for the 'game_start' event. Notifies the client that the game has begun.

Args:

event (dict): The event dictionary containing 'game_id'.

Returns:

None

"""

```
await self.send(text_data=json.dumps({
    'action': 'game_start',
    'game_id': event['game_id']
}))
```

----- DB access methos -----

@database_sync_to_async

def check_room(self, room_code):

"""

Validates if a user can join a specific room.

Checks room existence, capacity, and if the user is already a member.

Args:

room_code (str): The room identifier.

Returns:

tuple[bool, str | None]: A tuple (can_join, error_message).

"""

if self.user is None:

return False, None

Check if room exists and if user can join (not full, not already in, etc). Create if doesn't exist and user

room = PrivateRoom.objects.filter(room_code=room_code).first()

if not room:

return False, "Sala no encontrada."

#Using relation players in users - private room

current_number_players = room.players.count()

if current_number_players >= MAX_PRIVATE_GAME_PLAYERS:

return False, "Sala llena."

user = CustomUser.objects.get(username=self.user.username)

current_private_room = user.current_private_room

if current_private_room== room:

return False, "Ya estás en esta sala."

return True, None

@database_sync_to_async

def join_room_group_db(self, room_code, user):

"""

Updates the database to reflect a user joining a private room and

retrieves the room's full current state and configuration.

Args:

room_code (str): The room identifier.
user (CustomUser): The user joining.

Returns:

dict: A dictionary containing 'players' (list of dicts -> username, ready_to_play, user_piece),
'owner' (str or None), 'bot_level' (str), and
'target_players' (int).

"""

```
current_user = CustomUser.objects.get(username=user.username)
room = PrivateRoom.objects.get(room_code=room_code)
```

```
current_user.current_private_room = room
current_user.ready_to_play = False
current_user.save()
```

```
if room.target_players < MAX_PRIVATE_GAME_PLAYERS:
    room.target_players += 1
    room.save()
```

```
players_data = list(room.players.values('username', 'ready_to_play', piece=models.F('user_piece__custom_id')))
```

```
return {
    'players': [{ 'username': p['username'], 'ready_to_play': p['ready_to_play'], 'user_piece': p['piece'] } for p in players_data],
    'owner': room.owner.username if room.owner else None,
    'bot_level': room.bot_level,
    'target_players': room.target_players
}
```

@database_sync_to_async

```
def leave_room_and_update_host(self, room_code, user):
    """
```

Updates the database when a user leaves a room.
Handles host migration to the next available player or room deletion if empty.

Args:

room_code (str): The room identifier.
user (CustomUser): The user leaving.

Returns:

dict | None: A dictionary with the new owner and player list (username, ready_to_play, user_piece), or None
"""

```
room = PrivateRoom.objects.filter(room_code=room_code).first()
if not room:
    return None
user_from_db = CustomUser.objects.get(username=user.username)
```

```
user_from_db.current_private_room = None
user_from_db.save()
```

```
new_owner = room.players.exclude(pk=user_from_db.pk).first()
```

```
# if no one left, delete
```

```
if new_owner is None:
    room.delete()
    return None
```

```

room.owner = new_owner
if room.target_players > MIN_PRIVATE_GAME_PLAYERS:
    room.target_players -= 1
room.save()

players_data = list(room.players.values('username', 'ready_to_play', piece=models.F('user_piece__custom_id')))

return {
    'owner': room.owner.username,
    'players': [{ 'username': p['username'], 'ready_to_play': p['ready_to_play'], 'user_piece': p['piece']} for p in players_data]
}

@database_sync_to_async
def update_player_ready_status(self, room_code, user, is_ready):
    """
    Updates the ready status of a user within a room.

    Args:
        room_code (str): The room identifier.
        user (CustomUser): The user updating their status.
        is_ready (bool): The new ready status.

    Returns:
        bool | None: True if updated, False if code mismatch, None if not in a room.
    """
    user_from_db = CustomUser.objects.get(username=user.username)

    if user_from_db.current_private_room is None:
        return None

    if user_from_db.current_private_room.room_code != room_code:
        return False

    user_from_db.ready_to_play = is_ready
    user_from_db.save()

@database_sync_to_async
def get_target_players(self, room_code):
    """
    Retrieves the current number of players in a private room.

    Args:
        room_code (str): The room identifier.

    Returns:
        int: The player count.
    """
    # Return the current number of players in the room
    room = PrivateRoom.objects.get(room_code=room_code)
    return room.target_players

@database_sync_to_async
def check_all_ready(self, room_code):
    """
    Checks if all players in a private room have set their status to ready.

    Args:
        room_code (str): The room identifier.

```



```

Returns:
    bool: True if everyone is ready, False otherwise.
"""
# Check if all players in the room are ready
room = PrivateRoom.objects.get(room_code=room_code)
if not room:
    return False

for player in room.players.all():
    if not player.ready_to_play:
        return False

return True

@database_sync_to_async
def is_owner(self, user, room_code):
    """
    Verifies if a specific user is the host (owner) of a private room.

    Args:
        user (CustomUser): The user to check.
        room_code (str): The room identifier.

    Returns:
        bool: True if the user is the owner, False otherwise.
    """
    # Check if the user is the host of the room
    room = PrivateRoom.objects.get(room_code=room_code)
    if not room:
        return False

    return room.owner == user

@database_sync_to_async
def get_room_owner(self, room_code):
    """
    Retrieves the username of the room's current owner.

    Args:
        room_code (str): The room identifier.

    Returns:
        str | None: The username of the owner, or None if not found.
    """
    # Return the username of the room's owner
    room = PrivateRoom.objects.get(room_code=room_code)
    if not room:
        return False

    if room.owner is None:
        return None

    return room.owner.username

async def send_error(self, message):
    """
    Sends an error message to the client.

```

```

Args:
    message (str): The error description.

Returns:
    None
"""
await self.send(text_data=json.dumps({
    'action': 'error',
    'message': message
})))

```

`chat_event(event)` `async`

Handler for chat messages. Sends the message content to the client.

Parameters:

Name	Type	Description	Default
<code>event</code>	<code>dict</code>	The event dictionary containing 'user' and 'message'. <i>required</i>	

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```

async def chat_event(self, event):
    """
    Handler for chat messages. Sends the message content to the client.

    Args:
        event (dict): The event dictionary containing 'user' and 'message'.

    Returns:
        None
    """
    # Send chat messages to frontend
    await self.send(text_data=json.dumps({
        'action': 'chat_message',
        'user': event['user'],
        'message': event['message']
    })))

```

`check_all_ready(room_code)`

Checks if all players in a private room have set their status to ready.

Parameters:

Name	Type	Description	Default
<code>room_code</code>	<code>str</code>	The room identifier. <i>required</i>	

Returns:**Name Type Description**

bool	True if everyone is ready, False otherwise.
------	---

- **Source code in** `magnate/consumers.py`

```
@database_sync_to_async
def check_all_ready(self, room_code):
    """
    Checks if all players in a private room have set their status to ready.

    Args:
        room_code (str): The room identifier.

    Returns:
        bool: True if everyone is ready, False otherwise.
    """
    # Check if all players in the room are ready
    room = PrivateRoom.objects.get(room_code=room_code)
    if not room:
        return False

    for player in room.players.all():
        if not player.ready_to_play:
            return False

    return True
```

<code>check_room(room_code)</code>

Validates if a user can join a specific room. Checks room existence, capacity, and if the user is already a member.

Parameters:

Name	Type	Description	Default
room_code	str	The room identifier.	<i>required</i>

Returns:**Type Description**

tuple[bool, str | None]: A tuple (can_join, error_message).

- **Source code in** `magnate/consumers.py`

```
@database_sync_to_async
def check_room(self, room_code):
    """
    Validates if a user can join a specific room.
    Checks room existence, capacity, and if the user is already a member.

    Args:
        room_code (str): The room identifier.

    Returns:
        tuple[bool, str | None]: A tuple (can_join, error_message).
```

```

"""
if self.user is None:
    return False, None

# Check if room exists and if user can join (not full, not already in, etc). Create if doesn't exist and user is
room = PrivateRoom.objects.filter(room_code=room_code).first()

if not room:
    return False, "Sala no encontrada."

#Using relation players in users - private room
current_number_players = room.players.count()

if current_number_players >= MAX_PRIVATE_GAME_PLAYERS:
    return False, "Sala llena."

user = CustomUser.objects.get(username=self.user.username)
current_private_room = user.current_private_room

if current_private_room== room:
    return False, "Ya estás en esta sala."

return True, None

```

`connect()` `async`

Handles initial WebSocket connection to a private room lobby. Validates authentication and room code, accepts the connection, and notifies the group of the new player.

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```

async def connect(self):
    """
    Handles initial WebSocket connection to a private room lobby.
    Validates authentication and room code, accepts the connection, and
    notifies the group of the new player.

    Args:
        None

    Returns:
        None
    """
    # Triggered when user opens a new private room or joins an existing one.
    scope_user = self.scope.get('user')
    if scope_user is None or getattr(scope_user, 'is_anonymous', True):
        await self.close(code=4002)
        return

    self.user = await database_sync_to_async(CustomUser.objects.get)(pk=scope_user.pk)

```

```

self.url = self.scope.get('url_route')
if self.url is None:
    await self.close(code=4002)
    return

self.room_code = self.url.get('kwargs').get('room_code')

self.room_group_name = f"lobby_{self.room_code}"

was_created = await self.get_or_create_room_db(self.room_code, self.user)

await self.accept()

if was_created:
    await self.send(text_data=json.dumps({
        'action': 'room_created',
        'room_code': self.room_code
    }))

can_join, message = await self.check_room(self.room_code)

# Case of invalid code or full lobby -> reject connection
if not can_join:
    # Accept to send error message and close connection
    await self.accept()
    await self.send(text_data=json.dumps({'error': message}))
    await self.close(code=4003)
    return

await self.channel_layer.group_add(
    self.room_group_name,
    self.channel_name
)

room_data = await self.join_room_group_db(self.room_code, self.user)
if not room_data:
    await self.close(code=4003)
    return

# Notify lobby members of new player and update player list
await self.channel_layer.group_send(
    self.room_group_name,
    {
        'type': 'lobby_update',
        'action': 'joined',
        'user': self.user.username,
        'players': room_data['players'],
        'owner': room_data['owner'],
        'bot_level': room_data['bot_level'],
        'target_players': room_data['target_players']
    }
)

```

```
create_private_game(room_code)
```

Creates a game from a private room, filling empty slots with Bots according to the configured difficulty. Randomizes player order, initializes statistics, and deletes the lobby.

Parameters:

Name	Type	Description	Default
room_code	str	The unique string identifier for the private room.	<i>required</i>

Returns:

Name Type Description

int		The primary key (<code>pk</code>) of the newly created Game instance.
-----	--	---

• Source code in `magnate/consumers.py`

```
@database_sync_to_async
def create_private_game(self, room_code):
    """
    Creates a game from a private room, filling empty slots with Bots according
    to the configured difficulty. Randomizes player order, initializes statistics,
    and deletes the lobby.

    Args:
        room_code (str): The unique string identifier for the private room.

    Returns:
        int: The primary key (`pk`) of the newly created Game instance.
    """
    import random
    from django.utils import timezone
    from .models import PrivateRoom, Game, CustomUser
    from .games import GameManager

    room = PrivateRoom.objects.get(room_code=room_code)
    real_users = list(room.players.all())
    users = real_users.copy()

    # fill with bots
    bots = False
    huecos = room.target_players - len(real_users)
    for i in range(huecos):
        bots = True
        bot_username = f"Bot_{room_code}_{i+1}"
        bot_user, _ = Bot.objects.get_or_create(
            username=bot_username
        )

        bot_user.bot_level = room.bot_level
        bot_user.save()

        users.append(bot_user)

    game = Game.objects.create(
        datetime=timezone.now(),
        active_turn_player=users[0], # Se ajusta abajo
        active_phase_player=users[0],
```

```

        phase=GameManager.ROLL_THE_DICES,
        has_bots = bots
    )

    game.money = {str(u.pk): 3000 for u in users}
    game.positions = {str(u.pk): 0 for u in users}

    game.players.set(users)
    ordered_pks = [u.pk for u in users]
    random.shuffle(ordered_pks)
    game.ordered_players = ordered_pks

    first_player = CustomUser.objects.get(pk=ordered_pks[0])
    game.active_turn_player = first_player
    game.active_phase_player = first_player

    for user in users:
        user.active_game = game
        user.played_games.add(game)
        user.current_private_room = None

        user.save()
        PlayerGameStatistic.objects.get_or_create(user=user, game=game)

    room.delete()

    # TODO: Make this an option

    game.save()

    GameManager._set_kick_out_timer(game, first_player)

    return game.pk

```

`disconnect(close_code)` `async`

Handles WebSocket disconnection from the private lobby. Removes the user from the group and handles host migration or room deletion.

Parameters:

Name	Type	Description	Default
<code>close_code</code>	<code>int</code>	The WebSocket close code. <i>required</i>	

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```

async def disconnect(self, close_code):
    """
    Handles WebSocket disconnection from the private lobby.
    Removes the user from the group and handles host migration or room deletion.

    Args:

```

```

        close_code (int): The WebSocket close code.

Returns:
    None
    """
    # Triggered when user leaves the private room lobby.
    # If owner leaves -> change host to the second older player. Else -> just update lobby.
    if close_code == 4002:
        print(f"Unauthorized user attempted to connect and was rejected.")
        return

    await self.channel_layer.group_discard(
        self.room_group_name,
        self.channel_name
    )

    # DB operations -> if needed, rotate host, remove room if empty, etc. Return updated player list and new host if
    room_data = await self.leave_room_and_update_host(self.room_code, self.user)

    if not room_data:
        return

    if self.user is None:
        return

    if room_data:
        await self.channel_layer.group_send(
            self.room_group_name,
            {
                'type': 'lobby_update',
                'action': 'player_left',
                'user_left': self.user.username,
                'owner': room_data["owner"],
                'players': room_data['players']
            }
        )

```

`game_start(event)` `async`

Handler for the 'game_start' event. Notifies the client that the game has begun.

Parameters:

Name	Type	Description	Default
event	dict	The event dictionary containing 'game_id'. <i>required</i>	

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```

async def game_start(self, event):
    """
    Handler for the 'game_start' event. Notifies the client that the game has begun.

```



```

Args:
    event (dict): The event dictionary containing 'game_id'.

Returns:
    None
"""
await self.send(text_data=json.dumps({
    'action': 'game_start',
    'game_id': event['game_id']
}))

```

```
get_or_create_room_db(room_code, user)
```

Retrieves an existing room or creates a new one with the given code and owner.

Parameters:

Name	Type	Description	Default
<code>room_code</code>	<code>str</code>	The unique room identifier.	<i>required</i>
<code>user</code>	<code>CustomUser</code>	The user creating or retrieving the room.	<i>required</i>

Returns:

Name Type Description

<code>bool</code>	True if a new room was created, False if it already existed.
-------------------	--

• Source code in `magnate/consumers.py`

```

@database_sync_to_async
def get_or_create_room_db(self, room_code, user):
    """
    Retrieves an existing room or creates a new one with the given code and owner.

    Args:
        room_code (str): The unique room identifier.
        user (CustomUser): The user creating or retrieving the room.

    Returns:
        bool: True if a new room was created, False if it already existed.
    """
    room = PrivateRoom.objects.filter(room_code=room_code).first()
    if not room:
        PrivateRoom.objects.create(
            room_code=room_code,
            owner=user
        )
    return True
return False

```

```
get_room_owner(room_code)
```

Retrieves the username of the room's current owner.

Parameters:

Name	Type	Description	Default
room_code	str	The room identifier.	<i>required</i>

Returns:**Type Description**

str | None: The username of the owner, or None if not found.

- Source code in `magnate/consumers.py`

```
@database_sync_to_async
def get_room_owner(self, room_code):
    """
    Retrieves the username of the room's current owner.

    Args:
        room_code (str): The room identifier.

    Returns:
        str | None: The username of the owner, or None if not found.
    """
    # Return the username of the room's owner
    room = PrivateRoom.objects.get(room_code=room_code)
    if not room:
        return False

    if room.owner is None:
        return None

    return room.owner.username
```

`get_target_players(room_code)`

Retrieves the current number of players in a private room.

Parameters:

Name	Type	Description	Default
room_code	str	The room identifier.	<i>required</i>

Returns:**Name Type Description**

int The player count.

- Source code in `magnate/consumers.py`

```
@database_sync_to_async
def get_target_players(self, room_code):
    """
    Retrieves the current number of players in a private room.

    Args:
        room_code (str): The room identifier.
```

Returns:

```
int: The player count.
"""
# Return the current number of players in the room
room = PrivateRoom.objects.get(room_code=room_code)
return room.target_players
```

`is_owner(user, room_code)`

Verifies if a specific user is the host (owner) of a private room.

Parameters:

Name	Type	Description	Default
<code>user</code>	<code>CustomUser</code>	The user to check.	<i>required</i>
<code>room_code</code>	<code>str</code>	The room identifier.	<i>required</i>

Returns:

Name Type Description

<code>bool</code>	True if the user is the owner, False otherwise.
-------------------	---

• Source code in `magnate/consumers.py`

```
@database_sync_to_async
def is_owner(self, user, room_code):
    """
    Verifies if a specific user is the host (owner) of a private room.

    Args:
        user (CustomUser): The user to check.
        room_code (str): The room identifier.

    Returns:
        bool: True if the user is the owner, False otherwise.
    """
    # Check if the user is the host of the room
    room = PrivateRoom.objects.get(room_code=room_code)
    if not room:
        return False

    return room.owner == user
```

`join_room_group_db(room_code, user)`

Updates the database to reflect a user joining a private room and retrieves the room's full current state and configuration.

Parameters:

Name	Type	Description	Default
room_code	str	The room identifier.	<i>required</i>
user	CustomUser	The user joining.	<i>required</i>

Returns:**Name Type Description**

dict		A dictionary containing 'players' (list of dicts -> username, ready_to_play, user_piece), 'owner' (str or None), 'bot_level' (str), and 'target_players' (int).
------	--	---

• Source code in `magnate/consumers.py`

```
@database_sync_to_async
def join_room_group_db(self, room_code, user):
    """
    Updates the database to reflect a user joining a private room and
    retrieves the room's full current state and configuration.

    Args:
        room_code (str): The room identifier.
        user (CustomUser): The user joining.

    Returns:
        dict: A dictionary containing 'players' (list of dicts -> username, ready_to_play, user_piece),
              'owner' (str or None), 'bot_level' (str), and
              'target_players' (int).
    """
    current_user = CustomUser.objects.get(username=user.username)
    room = PrivateRoom.objects.get(room_code=room_code)

    current_user.current_private_room = room
    current_user.ready_to_play = False
    current_user.save()

    if room.target_players < MAX_PRIVATE_GAME_PLAYERS:
        room.target_players += 1
        room.save()

    players_data = list(room.players.values('username', 'ready_to_play', piece=models.F('user_piece__custom_id')))

    return {
        'players': [{ 'username': p['username'], 'ready_to_play': p['ready_to_play'], 'user_piece': p['piece'] } for p
        'owner': room.owner.username if room.owner else None,
        'bot_level': room.bot_level,
        'target_players': room.target_players
    }
```

leave_room_and_update_host(room_code, user)

Updates the database when a user leaves a room. Handles host migration to the next available player or room deletion if empty.

Parameters:

Name	Type	Description	Default
room_code	str	The room identifier.	<i>required</i>
user	CustomUser	The user leaving.	<i>required</i>

Returns:**Type Description**

dict | None: A dictionary with the new owner and player list (username, ready_to_play, user_piece), or None if the room was deleted.

- **Source code in** `magnate/consumers.py`

```
@database_sync_to_async
def leave_room_and_update_host(self, room_code, user):
    """
    Updates the database when a user leaves a room.
    Handles host migration to the next available player or room deletion if empty.

    Args:
        room_code (str): The room identifier.
        user (CustomUser): The user leaving.

    Returns:
        dict | None: A dictionary with the new owner and player list (username, ready_to_play, user_piece), or None if
        the room was deleted.
    """
    room = PrivateRoom.objects.filter(room_code=room_code).first()
    if not room:
        return None
    user_from_db = CustomUser.objects.get(username=user.username)

    user_from_db.current_private_room = None
    user_from_db.save()

    new_owner = room.players.exclude(pk=user_from_db.pk).first()

    # if no one left, delete
    if new_owner is None:
        room.delete()
        return None

    room.owner = new_owner
    if room.target_players > MIN_PRIVATE_GAME_PLAYERS:
        room.target_players -= 1
    room.save()

    players_data = list(room.players.values('username', 'ready_to_play', piece=models.F('user_piece__custom_id')))

    return {
        'owner': room.owner.username,
        'players': [{ 'username': p['username'], 'ready_to_play': p['ready_to_play'], 'user_piece': p['piece'] } for p in players_data]
    }
```

lobby_update(event) `async`

Handler for lobby status broadcasts (joined, left, ready, settings). Forwards the update to the connected WebSocket client.

Parameters:

Name	Type	Description	Default
event	dict	The event dictionary containing the action and relevant data.	<i>required</i>

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```
async def lobby_update(self, event):
    """
    Handler for lobby status broadcasts (joined, left, ready, settings).
    Forwards the update to the connected WebSocket client.

    Args:
        event (dict): The event dictionary containing the action and relevant data.

    Returns:
        None
    """
    if self.user is None:
        return

    # Send lobby updates to frontend (new player joined, player left)
    if event['action'] == 'joined':
        await self.send(text_data=json.dumps({
            'action': event['action'],
            'user': event['user'],
            'owner': event['owner'],
            'is_owner': (self.user.username == event['owner']),
            'players': event['players'],
            'bot_level': event.get('bot_level'),
            'target_players': event.get('target_players')
        }))
    elif event['action'] == 'player_left':
        await self.send(text_data=json.dumps({
            'action': event['action'],
            'user_left': event['user_left'],
            'owner': event['owner'],
            'is_owner': (self.user.username == event['owner']),
            'players': event['players']
        }))
    elif event['action'] == 'ready_status':
        await self.send(text_data=json.dumps({
            'action': event['action'],
            'user': event['user'],
            'is_ready': event['is_ready'],
            'owner': event['owner'],
            'is_owner': (self.user.username == event['owner'])
        }))
    elif event['action'] == 'settings_changed':
```

```

        await self.send(text_data=json.dumps({
            'action': event['action'],
            'bot_level': event['bot_level'],
            'target_players': event['target_players']
        }))

```

```

receive(text_data)  async

```

Processes incoming messages from lobby clients (chat, ready status, start game, settings).

Parameters:

Name	Type	Description	Default
text_data	str	The raw JSON string received.	<i>required</i>

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```

async def receive(self, text_data):
    """
    Processes incoming messages from lobby clients (chat, ready status, start game, settings).

    Args:
        text_data (str): The raw JSON string received.

    Returns:
        None
    """
    # Chat messages or 'start_game' command / 'ready' status updates.
    if self.user is None:
        return

    try:
        data = json.loads(text_data)
        command = data.get('command')

        if not command:
            await self.send_error("Comando invalido.")
            return

        if command == 'start_game':
            is_owner = await self.is_owner(self.user, self.room_code)
            if not is_owner:
                await self.send_error("Solo el host puede iniciar una partida.")
                return

            target_players = await self.get_target_players(self.room_code)

            if target_players < MIN_PRIVATE_GAME_PLAYERS:
                await self.send_error(f"Se necesitan {MIN_PRIVATE_GAME_PLAYERS} jugadores para iniciar la partida.")
                return

```

```

all_ready = await self.check_all_ready(self.room_code)
if not all_ready:
    await self.send_error("Todos los jugadores deben estar listos para iniciar la partida.")
    return

game_pk = await self.create_private_game(self.room_code)

await self.channel_layer.group_send(
    self.room_group_name,
    {
        'type': 'game_start',
        'game_id': game_pk
    })

elif command == 'ready_status':
    is_ready = data.get('is_ready')

    # Update in db
    await self.update_player_ready_status(self.room_code, self.user, is_ready)

    owner = await self.get_room_owner(self.room_code)

    await self.channel_layer.group_send(
        self.room_group_name,
        {
            'type': 'lobby_update',
            'action': 'ready_status',
            'user': self.user.username,
            'is_ready': is_ready,
            'owner': owner
        }
    )

elif command == 'chat_message':
    message = data.get('message')
    await self.channel_layer.group_send(
        self.room_group_name,
        {
            'type': 'chat_event',
            'user': self.user.username,
            'message': message
        }
    )

elif command == 'update_settings': # owner changes target users and bot level
    is_owner = await self.is_owner(self.user, self.room_code)
    if not is_owner:
        await self.send_error("Solo el host puede cambiar la configuración.")
        return

    bot_level = data.get('bot_level')
    target_players = data.get('target_players')

    await self.update_room_settings(self.room_code, bot_level, target_players)

```



```

        # Avisar a todos los de la sala del cambio
        await self.channel_layer.group_send(
            self.room_group_name,
            {
                'type': 'lobby_update',
                'action': 'settings_changed',
                'bot_level': bot_level,
                'target_players': target_players
            }
        )

    except json.JSONDecodeError:
        await self.send_error("Datos invalidos.")
        return

```

`send_error(message)` `async`

Sends an error message to the client.

Parameters:

Name	Type	Description	Default
<code>message</code>	<code>str</code>	The error description. <i>required</i>	

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```

async def send_error(self, message):
    """
    Sends an error message to the client.

    Args:
        message (str): The error description.

    Returns:
        None
    """
    await self.send(text_data=json.dumps({
        'action': 'error',
        'message': message
    }))

```

`update_player_ready_status(room_code, user, is_ready)`

Updates the ready status of a user within a room.

Parameters:

Name	Type	Description	Default
room_code	str	The room identifier.	<i>required</i>
user	CustomUser	The user updating their status.	<i>required</i>
is_ready	bool	The new ready status.	<i>required</i>

Returns:**Type Description**

bool | None: True if updated, False if code mismatch, None if not in a room.

- **Source code in** `magnate/consumers.py`

```
@database_sync_to_async
def update_player_ready_status(self, room_code, user, is_ready):
    """
    Updates the ready status of a user within a room.

    Args:
        room_code (str): The room identifier.
        user (CustomUser): The user updating their status.
        is_ready (bool): The new ready status.

    Returns:
        bool | None: True if updated, False if code mismatch, None if not in a room.
    """
    user_from_db = CustomUser.objects.get(username=user.username)

    if user_from_db.current_private_room is None:
        return None

    if user_from_db.current_private_room.room_code != room_code:
        return False

    user_from_db.ready_to_play = is_ready
    user_from_db.save()
```

```
update_room_settings(room_code, bot_level, target_players)
```

Updates the target player count and bot difficulty level for a room.

Parameters:

Name	Type	Description	Default
room_code	str	The room identifier.	<i>required</i>
bot_level	str	The difficulty level for bots.	<i>required</i>
target_players	int	The desired total number of players.	<i>required</i>

Returns:**Type Description**

None

• **Source code in** `magnate/consumers.py`

```
@database_sync_to_async
def update_room_settings(self, room_code, bot_level, target_players):
    """
    Updates the target player count and bot difficulty level for a room.

    Args:
        room_code (str): The room identifier.
        bot_level (str): The difficulty level for bots.
        target_players (int): The desired total number of players.

    Returns:
        None
    """
    room = PrivateRoom.objects.get(room_code=room_code)
    if bot_level:
        room.bot_level = bot_level
    if target_players:
        room.target_players = target_players
    room.save()
```

2.2.3 GameConsumer

Bases: `AsyncWebsocketConsumer`

WebSocket consumer for an active game session.

This is the main gameplay socket. Once connected, the client receives the current game state, can send player actions, and receives broadcasts of all actions and their resolved outcomes in real time.

How to Connect

Endpoint: `ws://<host>/ws/game/<game_id>/`

Authentication required: Yes. Unauthenticated connections are closed with code `4002`.

The `game_id` must match an existing `Game` that the authenticated user belongs to. If the user is not a participant, the connection is rejected with code `4003`.

Connection Lifecycle

1. Client opens the WebSocket with the `game_id` received from `PublicQueueConsumer` or `PrivateRoomConsumer`.
2. Server validates that the user is a participant.
3. On success, the server immediately sends a `game_state` event to the connecting client with the full current game state.
4. The client sends `Action` messages as the game progresses.
5. Each valid action is broadcast as a `game_action` event, followed by a `game_response` event with the resolved outcome.
6. All players in the game receive both broadcasts.

Messages: Client → Server

Game Action

Sends a player action for the current game phase. The `type` field must correspond to a valid `Action` type for the active phase. All other fields are action-specific.

```
{
  "type": "SomeActionType",
  "...": "action-specific fields"
}
```

The `game` and `player` fields are injected server-side — do **not** include them manually.

Chat Message

```
{
  "type": "ChatMessage",
  "msg": "Hello everyone!"
}
```

Messages: Server → Client

Game State

Sent immediately on connection. Contains the full serialized game state.

```
{
  "event_type": "game_state",
  "game_state": { "...": "full GameStatusSerializer output" }
}
```

Game Action

Broadcast to all players when a valid action is received. Contains the raw action data as sent by the acting player.

```
{
  "event_type": "game_action",
  "data": { "type": "SomeActionType", "game": 42, "player": 7, "...": "..." }
}
```

Cheats Documentation (DEBUG mode only)

When `DEBUG=True` in Django settings, players can send `cheat` commands to forcefully alter the state of the active game.

Base Message Structure:

```
{
  "action": "cheat",
  "data": {
    "cheat": "<CheatName>",
    ...cheat_specific_args
  }
}
```

Game Response

Broadcast to all players immediately after `game_action`. Contains the resolved outcome of the action as serialized by `GeneralResponseSerializer`.

```
{
  "event_type": "game_response",
  "data": { "...": "GeneralResponseSerializer output" }
}
```

Chat Message

Broadcast to all players in the game.

```
{
  "event_type": "chat_message",
  "game": 42,
  "user": "alice",
  "msg": "Hello everyone!"
}
```

Error

Sent only to the client that triggered the error (invalid action, wrong phase, etc.).

```
{
  "event_type": "error",
  "message": "Acción no válida en la fase actual."
}
```

Close Codes

Code Meaning

4002 Unauthorized or missing route kwargs.

4003 User is not a participant in this game.

Important Notes

- Every valid action triggers **two** consecutive broadcasts: `game_action` (what was sent) and `game_response` (the outcome). Always handle both.
- Invalid actions (wrong phase, serialization errors, game logic errors) produce an `error` event and are **not** broadcast to other players.
- The game state snapshot sent on connect may be slightly stale by the time the first `game_action` arrives; prefer the response stream for live updates.

Example Flow (JavaScript)

```
const socket = new WebSocket(`ws://localhost:8000/ws/game/${gameId}/`);

socket.onmessage = (event) => {
  const data = JSON.parse(event.data);

  switch (data.event_type) {
```

```

        case "game_state":
            initializeBoard(data.game_state);
            break;
        case "game_action":
            highlightAction(data.data);
            break;
        case "game_response":
            applyOutcome(data.data);
            break;
        case "chat_message":
            appendChat(data.user, data.msg);
            break;
        case "error":
            showError(data.message);
            break;
    }
};

// Send an action:
socket.send(JSON.stringify({ type: "RollDice" }));

// Send a chat message:
socket.send(JSON.stringify({ type: "ChatMessage", msg: "Good luck!" }));

```

• Source code in `magnate/consumers.py`

```

class GameConsumer(AsyncWebsocketConsumer):
    """
    WebSocket consumer for an active game session.

    This is the main gameplay socket. Once connected, the client receives the
    current game state, can send player actions, and receives broadcasts of
    all actions and their resolved outcomes in real time.

    ---
    ## How to Connect

    **Endpoint:** ``ws://<host>/ws/game/<game_id>/``

    **Authentication required:** Yes. Unauthenticated connections are closed with code ``4002``.

    The ``game_id`` must match an existing ``Game`` that the authenticated user
    belongs to. If the user is not a participant, the connection is rejected with
    code ``4003``.

    ---
    ## Connection Lifecycle

    1. Client opens the WebSocket with the ``game_id`` received from ``PublicQueueConsumer``
       or ``PrivateRoomConsumer``.
    2. Server validates that the user is a participant.
    3. On success, the server immediately sends a ``game_state`` event to the
       connecting client with the full current game state.
    4. The client sends ``Action`` messages as the game progresses.
    5. Each valid action is broadcast as a ``game_action`` event, followed by a
       ``game_response`` event with the resolved outcome.
    6. All players in the game receive both broadcasts.

    ---
    """

```

```
## Messages: Client → Server
```

```
### Game Action
```

Sends a player action for the current game phase. The ``type`` field must correspond to a valid ``Action`` type for the active phase. All other fields are action-specific.

```
```json
{
 "type": "SomeActionType",
 "...": "action-specific fields"
}
```
```

The ``game`` and ``player`` fields are injected server-side – do **not** include them manually.

```
### Chat Message
```

```
```json
{
 "type": "ChatMessage",
 "msg": "Hello everyone!"
}
```
```

```
---
```

```
## Messages: Server → Client
```

```
### Game State
```

Sent immediately on connection. Contains the full serialized game state.

```
```json
{
 "event_type": "game_state",
 "game_state": { "...": "full GameStatusSerializer output" }
}
```
```

```
### Game Action
```

Broadcast to all players when a valid action is received. Contains the raw action data as sent by the acting player.

```
```json
{
 "event_type": "game_action",
 "data": { "type": "SomeActionType", "game": 42, "player": 7, "...": "..." }
}
```
```

```
---
```

```
## Cheats Documentation (DEBUG mode only)
```

When ``DEBUG=True`` in Django settings, players can send ``cheat`` commands to forcefully alter the state of the active game.

```
**Base Message Structure:**
```

```
```json
{
 "action": "cheat",
```

```

 "data": {
 "cheat": "<CheatName>",
 ...cheat_specific_args
 }
}
...

```

### Game Response

Broadcast to all players immediately after ``game\_action``. Contains the resolved outcome of the action as serialized by ``GeneralResponseSerializer``.

```

```json
{
    "event_type": "game_response",
    "data": { "...": "GeneralResponseSerializer output" }
}
...

```

Chat Message

Broadcast to all players in the game.

```

```json
{
 "event_type": "chat_message",
 "game": 42,
 "user": "alice",
 "msg": "Hello everyone!"
}
...

```

### Error

Sent only to the client that triggered the error (invalid action, wrong phase, etc.).

```

```json
{
    "event_type": "error",
    "message": "Acción no válida en la fase actual."
}
...

```

Close Codes

Code	Meaning
4002	Unauthorized or missing route kwargs.
4003	User is not a participant in this game.

Important Notes

- Every valid action triggers **two** consecutive broadcasts: ``game\_action`` (what was sent) and ``game\_response`` (the outcome). Always handle both.
- Invalid actions (wrong phase, serialization errors, game logic errors) produce an ``error`` event and are **not** broadcast to other players.
- The game state snapshot sent on connect may be slightly stale by the time the first ``game\_action`` arrives; prefer the response stream for live updates.


```

---
## Example Flow (JavaScript)

```js
const socket = new WebSocket(`ws://localhost:8000/ws/game/${gameId}/`);

socket.onmessage = (event) => {
 const data = JSON.parse(event.data);

 switch (data.event_type) {
 case "game_state":
 initializeBoard(data.game_state);
 break;
 case "game_action":
 highlightAction(data.data);
 break;
 case "game_response":
 applyOutcome(data.data);
 break;
 case "chat_message":
 appendChat(data.user, data.msg);
 break;
 case "error":
 showError(data.message);
 break;
 }
};

// Send an action:
socket.send(JSON.stringify({ type: "RollDice" }));

// Send a chat message:
socket.send(JSON.stringify({ type: "ChatMessage", msg: "Good luck!" }));
```

"""
async def connect(self):
    """
    Handles initial WebSocket connection to an active game.
    Validates participant status, accepts connection, and sends the initial
    full game state.

    Args:
        None

    Returns:
        None
    """
    # Triggered when user joins a specific match ID (game really begins) -> add to Redis room group.
    scope_user = self.scope.get('user')
    if scope_user is None or getattr(scope_user, 'is_anonymous', True):
        await self.close(code=4002)
        return

    # Extraemos la instancia real de CustomUser de la BD
    self.user = await database_sync_to_async(CustomUser.objects.get)(pk=scope_user.pk)

    self.url = self.scope.get('url_route')
    if self.url is None:
        await self.close(code=4002)

```

```

        return

    kwargs = self.url.get('kwargs')
    if not kwargs or 'room_id' not in kwargs:
        await self.close(code=4002)
        return

    self.game_id = int(kwargs['room_id'])

    self.game_group_name = f"game_{self.game_id}"

    player_is_in_game = await self.is_player_in_game(self.user, self.game_id)

    if not player_is_in_game:
        await self.close(code=4003)
        return

    await self.channel_layer.group_add(
        self.game_group_name,
        self.channel_name
    )

    await self.accept()

    game = await self.get_game()
    game_state = await database_sync_to_async(
        lambda: GameStatusSerializer(game).data)()

    await self.channel_layer.send(
        self.channel_name,
        {
            'type': 'game_state',
            'game_state': game_state
        }
    )

async def disconnect(self, close_code):
    """
    Handles WebSocket disconnection from the game session.

    Args:
        close_code (int): The WebSocket close code.

    Returns:
        None
    """
    # Triggered when user leaves game -> notify opponent.
    await self.channel_layer.group_discard(
        self.game_group_name,
        self.channel_name
    )

async def receive(self, text_data):
    """
    Processes WebSocket messages in the active game.
    Differentiates between chat messages and game actions. Deserializes actions,
    processes them via GameManager, and broadcasts the events (action sent and
    outcome) to all clients in the group. Deletes the action from the DB if a

```

```

logic error occurs.

Args:
    text_data (str): The raw JSON string received from the WebSocket client.

Returns:
    None

Raises:
    Sends an error via WebSocket (does not explicitly raise a Python exception)
    if the game is not found, data is invalid, or if GameManager throws a
    MaliciousUserInput, GameLogicError, or GameDesignError.
"""

game = await self.get_game()
if game is None:
    await self.send_error("Game not found")
    return

data = json.loads(text_data)

if data.get('type') == 'ChatMessage':
    message = data.get('msg')
    if message:
        await self.channel_layer.group_send(
            self.game_group_name,
            {
                'type': 'chat_message',
                'game': self.game_id,
                'user': self.user.username,
                'msg': message
            }
        )
    return
elif data.get('type') == 'Cheat':
    try:
        await handle_cheat(game, data)
        game_state = await database_sync_to_async(
            lambda: GameStatusSerializer(game).data)()
        await self.channel_layer.send(
            self.channel_name,
            {
                'type': 'game_state',
                'game_state': game_state
            }
        )
    except CheatException as e:
        await self.send_error(f"{e}")

    return

data['game'] = self.game_id
data['player'] = self.user.pk

action, errors = await validate_and_save_action(data)

if errors or action is None:
    await self.send_error(f"Invalid data: {errors}")
    return

```

```

action = cast(Action, action)

try:
    response = await GameManager.process_action(game, self.user, action)

    if response is None:
        await database_sync_to_async(action.delete)() # Limpiamos BD
        await self.send_error("Acción no válida en la fase actual.")
        return

    # Broadcast action if not an ActionBid
    # TODO: Expand to a list if necessary
    if not isinstance(action, ActionBid):
        await self.channel_layer.group_send(
            self.game_group_name,
            {
                'type': 'game_action_event',
                'data': data
            }
        )

    response_data = await database_sync_to_async(lambda: GeneralResponseSerializer(response).data)()

    await self.channel_layer.group_send(
        self.game_group_name,
        {
            'type': 'game_response_event',
            'data': response_data
        }
    )

except (MaliciousUserInput, GameLogicError, GameDesignError, Exception) as e:
    await database_sync_to_async(action.delete)()
    await self.send_error(f"{e}")

# ----- Handlers ----- #

async def game_state(self, event):
    """
    Handler for sending the game state to the client.

    Args:
        event (dict): The event dictionary containing 'game_state'.

    Returns:
        None
    """
    await self.send(text_data=json.dumps({
        'event_type': 'game_state',
        'game_state': event['game_state']
    }))

async def game_action_event(self, event):
    """
    Handler for broadcasting a game action to the client.

    Args:
        event (dict): The event dictionary containing 'data' (raw action).

```

```

Returns:
    None
    """
    await self.send(text_data=json.dumps({
        'event_type': 'game_action',
        'data': event['data']
    }))

async def game_response_event(self, event):
    """
    Handler for broadcasting a game response (outcome) to the client.

    Args:
        event (dict): The event dictionary containing 'data' (serialized outcome).

    Returns:
        None
        """
    await self.send(text_data=json.dumps({
        'event_type': 'game_response',
        'data': event['data']
    }))

async def chat_message(self, event):
    """
    Handler for game chat messages.

    Args:
        event (dict): The event dictionary containing 'game', 'user', and 'msg'.

    Returns:
        None
        """
    await self.send(text_data=json.dumps({
        'event_type': 'chat_message',
        'game': event['game'],
        'user': event['user'],
        'msg': event['msg']
    }))

async def send_error(self, message):
    """
    Sends an error message to the client.

    Args:
        message (str): The error description.

    Returns:
        None
        """
    await self.send(text_data=json.dumps({
        'event_type': 'error',
        'message': message
    }))

```

```

#----- DB access -----#
@database_sync_to_async

```

```
def is_player_in_game(self, user, game_id):
    """
    Verifies if a specific user is a participant in a given game.

    Args:
        user (CustomUser): The user to check.
        game_id (int): The primary key of the game.

    Returns:
        bool: True if the user is a participant, False otherwise.
    """
    try:
        game = Game.objects.get(pk=game_id)
        return game.players.filter(pk=user.pk).exists()
    except Game.DoesNotExist:
        return False

@database_sync_to_async
def get_game(self):
    """
    Retrieves the current game instance from the database.

    Args:
        None

    Returns:
        Game | None: The Game instance or None if not found.
    """
    try:
        return Game.objects.get(pk=self.game_id)
    except Game.DoesNotExist:
        return None
```

```
chat_message(event) async
```

Handler for game chat messages.

Parameters:

| Name | Type | Description | Default |
|-------|------|--|-----------------|
| event | dict | The event dictionary containing 'game', 'user', and 'msg'. | <i>required</i> |

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```
async def chat_message(self, event):
    """
    Handler for game chat messages.

    Args:
        event (dict): The event dictionary containing 'game', 'user', and 'msg'.

    Returns:
```

```

        None
    """
    await self.send(text_data=json.dumps({
        'event_type': 'chat_message',
        'game': event['game'],
        'user': event['user'],
        'msg': event['msg']
    }))

```

`connect()` `async`

Handles initial WebSocket connection to an active game. Validates participant status, accepts connection, and sends the initial full game state.

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```

async def connect(self):
    """
    Handles initial WebSocket connection to an active game.
    Validates participant status, accepts connection, and sends the initial
    full game state.

    Args:
        None

    Returns:
        None
    """
    # Triggered when user joins a specific match ID (game really begins) -> add to Redis room group.
    scope_user = self.scope.get('user')
    if scope_user is None or getattr(scope_user, 'is_anonymous', True):
        await self.close(code=4002)
        return

    # Extraemos la instancia real de CustomUser de la BD
    self.user = await database_sync_to_async(CustomUser.objects.get)(pk=scope_user.pk)

    self.url = self.scope.get('url_route')
    if self.url is None:
        await self.close(code=4002)
        return

    kwargs = self.url.get('kwargs')
    if not kwargs or 'room_id' not in kwargs:
        await self.close(code=4002)
        return

    self.game_id = int(kwargs['room_id'])

    self.game_group_name = f"game_{self.game_id}"

    player_is_in_game = await self.is_player_in_game(self.user, self.game_id)

```

```

if not player_is_in_game:
    await self.close(code=4003)
    return

await self.channel_layer.group_add(
    self.game_group_name,
    self.channel_name
)

await self.accept()

game = await self.get_game()
game_state = await database_sync_to_async(
    lambda: GameStatusSerializer(game).data)()

await self.channel_layer.send(
    self.channel_name,
    {
        'type': 'game_state',
        'game_state': game_state
    }
)

```

`disconnect(close_code)` `async`

Handles WebSocket disconnection from the game session.

Parameters:

| Name | Type | Description | Default |
|-------------------------|------------------|---|---------|
| <code>close_code</code> | <code>int</code> | The WebSocket close code. <i>required</i> | |

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```

async def disconnect(self, close_code):
    """
    Handles WebSocket disconnection from the game session.

    Args:
        close_code (int): The WebSocket close code.

    Returns:
        None
    """
    # Triggered when user leaves game -> notify opponent.
    await self.channel_layer.group_discard(
        self.game_group_name,
        self.channel_name
    )

```


`game_action_event(event)` `async`

Handler for broadcasting a game action to the client.

Parameters:

| Name | Type | Description | Default |
|--------------------|-------------------|--|---------|
| <code>event</code> | <code>dict</code> | The event dictionary containing 'data' (raw action). <i>required</i> | |

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```
async def game_action_event(self, event):
    """
    Handler for broadcasting a game action to the client.

    Args:
        event (dict): The event dictionary containing 'data' (raw action).

    Returns:
        None
    """
    await self.send(text_data=json.dumps({
        'event_type': 'game_action',
        'data': event['data']
    })))
```

`game_response_event(event)` `async`

Handler for broadcasting a game response (outcome) to the client.

Parameters:

| Name | Type | Description | Default |
|--------------------|-------------------|--|---------|
| <code>event</code> | <code>dict</code> | The event dictionary containing 'data' (serialized outcome). <i>required</i> | |

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```
async def game_response_event(self, event):
    """
    Handler for broadcasting a game response (outcome) to the client.

    Args:
        event (dict): The event dictionary containing 'data' (serialized outcome).

    Returns:
        None
    """
```

```
await self.send(text_data=json.dumps({
    'event_type': 'game_response',
    'data': event['data']
})))
```

`game_state(event)` `async`

Handler for sending the game state to the client.

Parameters:

| Name | Type | Description | Default |
|--------------------|-------------------|---|---------|
| <code>event</code> | <code>dict</code> | The event dictionary containing 'game_state'. <i>required</i> | |

Returns:

Type Description

None

• Source code in `magnate/consumers.py`

```
async def game_state(self, event):
    """
    Handler for sending the game state to the client.

    Args:
        event (dict): The event dictionary containing 'game_state'.

    Returns:
        None
    """
    await self.send(text_data=json.dumps({
        'event_type': 'game_state',
        'game_state': event['game_state']
    })))
```

`get_game()`

Retrieves the current game instance from the database.

Returns:

Type Description

Game | None: The Game instance or None if not found.

• Source code in `magnate/consumers.py`

```
@database_sync_to_async
def get_game(self):
    """
    Retrieves the current game instance from the database.

    Args:
        None
```

```

Returns:
    Game | None: The Game instance or None if not found.
"""
try:
    return Game.objects.get(pk=self.game_id)
except Game.DoesNotExist:
    return None

```

```
is_player_in_game(user, game_id)
```

Verifies if a specific user is a participant in a given game.

Parameters:

| Name | Type | Description | Default |
|----------------------|-------------------------|------------------------------|-----------------|
| <code>user</code> | <code>CustomUser</code> | The user to check. | <i>required</i> |
| <code>game_id</code> | <code>int</code> | The primary key of the game. | <i>required</i> |

Returns:

Name Type Description

| | |
|-------------------|---|
| <code>bool</code> | True if the user is a participant, False otherwise. |
|-------------------|---|

• Source code in `magnate/consumers.py`

```

@database_sync_to_async
def is_player_in_game(self, user, game_id):
    """
    Verifies if a specific user is a participant in a given game.

    Args:
        user (CustomUser): The user to check.
        game_id (int): The primary key of the game.

    Returns:
        bool: True if the user is a participant, False otherwise.
    """
    try:
        game = Game.objects.get(pk=game_id)
        return game.players.filter(pk=user.pk).exists()
    except Game.DoesNotExist:
        return False

```

```
receive(text_data) async
```

Processes WebSocket messages in the active game. Differentiates between chat messages and game actions. Deserializes actions, processes them via GameManager, and broadcasts the events (action sent and outcome) to all clients in the group. Deletes the action from the DB if a logic error occurs.

Parameters:

| Name | Type | Description | Default |
|------------------------|------------------|---|-----------------|
| <code>text_data</code> | <code>str</code> | The raw JSON string received from the WebSocket client. | <i>required</i> |

Returns:**Type Description**

None

• **Source code in** `magnate/consumers.py`

```

async def receive(self, text_data):
    """
    Processes WebSocket messages in the active game.
    Differentiates between chat messages and game actions. Deserializes actions,
    processes them via GameManager, and broadcasts the events (action sent and
    outcome) to all clients in the group. Deletes the action from the DB if a
    logic error occurs.

    Args:
        text_data (str): The raw JSON string received from the WebSocket client.

    Returns:
        None

    Raises:
        Sends an error via WebSocket (does not explicitly raise a Python exception)
        if the game is not found, data is invalid, or if GameManager throws a
        MaliciousUserInput, GameLogicError, or GameDesignError.
    """

    game = await self.get_game()
    if game is None:
        await self.send_error("Game not found")
        return

    data = json.loads(text_data)

    if data.get('type') == 'ChatMessage':
        message = data.get('msg')
        if message:
            await self.channel_layer.group_send(
                self.game_group_name,
                {
                    'type': 'chat_message',
                    'game': self.game_id,
                    'user': self.user.username,
                    'msg': message
                }
            )
        return
    elif data.get('type') == 'Cheat':
        try:
            await handle_cheat(game, data)
            game_state = await database_sync_to_async(
                lambda: GameStatusSerializer(game).data)()
            await self.channel_layer.send(
                self.channel_name,

```

```

        {
            'type': 'game_state',
            'game_state': game_state
        }
    )
except CheatException as e:
    await self.send_error(f"{e}")

return

data['game'] = self.game_id
data['player'] = self.user.pk

action, errors = await validate_and_save_action(data)

if errors or action is None:
    await self.send_error(f"Invalid data: {errors}")
    return

action = cast(Action, action)

try:
    response = await GameManager.process_action(game, self.user, action)

    if response is None:
        await database_sync_to_async(action.delete)() # Limpiamos BD
        await self.send_error("Acción no válida en la fase actual.")
        return

    # Broadcast action if not an ActionBid
    # TODO: Expand to a list if necessary
    if not isinstance(action, ActionBid):
        await self.channel_layer.group_send(
            self.game_group_name,
            {
                'type': 'game_action_event',
                'data': data
            }
        )

    response_data = await database_sync_to_async(lambda: GeneralResponseSerializer(response).data)()

    await self.channel_layer.group_send(
        self.game_group_name,
        {
            'type': 'game_response_event',
            'data': response_data
        }
    )

except (MaliciousUserInput, GameLogicError, GameDesignError, Exception) as e:
    await database_sync_to_async(action.delete)()
    await self.send_error(f"{e}")

```

`send_error(message)` `async`

Sends an error message to the client.

Parameters:

| Name | Type | Description | Default |
|---------|------|------------------------|-----------------|
| message | str | The error description. | <i>required</i> |

Returns:**Type Description**

None

- **Source code in** `magnate/consumers.py`

```

async def send_error(self, message):
    """
    Sends an error message to the client.

    Args:
        message (str): The error description.

    Returns:
        None
    """
    await self.send(text_data=json.dumps({
        'event_type': 'error',
        'message': message
    }))

```

2.3 Game

Game class

2.3.1 `Game`

Bases: `Model`

Represents the core state and data of a single "Magnate" game session.

Attributes:

| Name | Type | Description |
|------------------------------------|-----------------------------------|---|
| <code>datetime</code> | <code>DateTimeField</code> | The timestamp of when the game was created or started. |
| <code>positions</code> | <code>JSONField</code> | Maps a player's user ID (str/int) to their current square's custom ID (int). |
| <code>money</code> | <code>JSONField</code> | Maps a player's user ID (str/int) to their current money balance (int). |
| <code>active_phase_player</code> | <code>ForeignKey</code> | The user who must take action in the current micro-phase (e.g., the user who needs to respond to a trade, which might differ from the <code>active_turn_player</code>). |
| <code>active_turn_player</code> | <code>ForeignKey</code> | The user whose actual turn it is on the board. |
| <code>phase</code> | <code>CharField</code> | The current <code>GamePhase</code> of the game state machine. |
| <code>players</code> | <code>ManyToManyField</code> | The pool of users actively participating in this game. |
| <code>ordered_players</code> | <code>JSONField</code> | A list of player primary keys <code>[pk1, pk2, pk3, ...]</code> representing the strict turn order of the game. |
| <code>streak</code> | <code>IntegerField</code> | Tracks consecutive identical dice rolls (e.g., rolling doubles). Usually triggers jail time if it hits 3. |
| <code>possible_destinations</code> | <code>JSONField list</code> | Maps a target <code>square_id</code> (str) to the <code>dice_combination</code> (int) required to get there. Used when a player has multiple routing options (e.g., taking a tram). |
| <code>parking_money</code> | <code>PositiveIntegerField</code> | The accumulated jackpot for landing on the "Free Parking" equivalent. |
| <code>jail_remaining_turns</code> | <code>JSONField</code> | Maps a player's user ID (str/int) to the number of turns (int) they have left to serve in jail. |
| <code>proposal</code> | <code>ForeignKey</code> | A reference to an active trade proposal (<code>ActionTradeProposal</code>) currently blocking the game state awaiting a response. |
| <code>fantasy_event</code> | <code>ForeignKey</code> | A reference to an active chance/community chest style event (<code>FantasyEvent</code>) currently being resolved. |
| <code>current_auction</code> | <code>ForeignKey</code> | A reference to an active property auction (<code>Auction</code>) taking place. |
| <code>finished</code> | <code>BooleanField</code> | Flag indicating if the game has concluded. |
| <code>bonus_response</code> | <code>ForeignKey</code> | Reference to a specific bonus or penalty modifier (<code>ResponseBonus</code>) applied to the current state. |
| <code>kick_out_task_id</code> | <code>CharField</code> | The ID of the scheduled Celery task responsible for kicking a player if they fail to act within the time limit. |
| <code>next_phase_task_id</code> | <code>CharField</code> | The ID of the scheduled Celery task responsible for auto-advancing the game to the next phase if a timeout occurs. |
| <code>auction_task_id</code> | <code>CharField</code> | The ID of the scheduled Celery task managing auction timing and auto-advancing the auction if no bids are placed within the time limit. |
| <code>current_turn</code> | <code>PositiveIntegerField</code> | The global counter for the number of turns that have elapsed. |

• Source code in `magnate/models.py`

```
class Game(models.Model):
    """
    Represents the core state and data of a single "Magnate" game session.

    Attributes:
        datetime (DateTimeField): The timestamp of when the game was created or started.
        positions (JSONField): Maps a player's user ID (str/int) to their current square's custom ID (int).
```



```

money (JSONField): Maps a player's user ID (str/int) to their current money balance (int).
active_phase_player (ForeignKey): The user who must take action in the current micro-phase
    (e.g., the user who needs to respond to a trade, which might differ from the active_turn_player).
active_turn_player (ForeignKey): The user whose actual turn it is on the board.
phase (CharField): The current ``GamePhase`` of the game state machine.
players (ManyToManyField): The pool of users actively participating in this game.
ordered_players (JSONField): A list of player primary keys ``[pk1, pk2, pk3, ...]``
    representing the strict turn order of the game.
streak (IntegerField): Tracks consecutive identical dice rolls (e.g., rolling doubles).
    Usually triggers jail time if it hits 3.
possible_destinations (JSONField list): Maps a target ``square_id`` (str) to the ``dice_combination`` (int)
    required to get there. Used when a player has multiple routing options (e.g., taking a tram).
parking_money (PositiveIntegerField): The accumulated jackpot for landing on the "Free Parking" equivalent.
jail_remaining_turns (JSONField): Maps a player's user ID (str/int) to the number of turns (int)
    they have left to serve in jail.
proposal (ForeignKey): A reference to an active trade proposal (``ActionTradeProposal``) currently
    blocking the game state awaiting a response.
fantasy_event (ForeignKey): A reference to an active chance/community chest style event
    (``FantasyEvent``) currently being resolved.
current_auction (ForeignKey): A reference to an active property auction (``Auction``) taking place.
finished (BooleanField): Flag indicating if the game has concluded.
bonus_response (ForeignKey): Reference to a specific bonus or penalty modifier (``ResponseBonus``)
    applied to the current state.
kick_out_task_id (CharField): The ID of the scheduled Celery task responsible for kicking
    a player if they fail to act within the time limit.
next_phase_task_id (CharField): The ID of the scheduled Celery task responsible for auto-advancing
    the game to the next phase if a timeout occurs.
auction_task_id (CharField): The ID of the scheduled Celery task managing auction timing and
    auto-advancing the auction if no bids are placed within the time limit.
current_turn (PositiveIntegerField): The global counter for the number of turns that have elapsed.
"""

datetime = models.DateTimeField()
# Maps user_id -> square_custom_id
positions = models.JSONField(default=dict, blank=True)
# Maps user_id -> amount
money = models.JSONField(default=dict, blank=True)
active_phase_player = models.ForeignKey('CustomUser', on_delete=models.SET_NULL, null=True, related_name='phase_f')
active_turn_player = models.ForeignKey('CustomUser', on_delete=models.SET_NULL, null=True, related_name='turns_to')
has_bots = models.BooleanField(default=False)

class GamePhase(models.TextChoices):
    """
    Enumeration of the possible states (phases) within a game's turn cycle.

    Attributes:
        roll_the_dices: The initial phase of a turn. The ``active_turn_player``
            must throw the dice to determine movement.
        choose_square: Triggered when a player's movement path presents a fork
            or routing option (e.g., deciding whether to take a tram/subway line).
            The player must select their specific destination square.
        choose_fantasy: Triggered when a player lands on a dynamic event square
            The player must acknowledge and resolve the active ``fantasy_event``.
        management: Triggered when a player lands on an unowned property. The
            player must choose to either purchase the property at its list price
            or decline the purchase (which typically immediately triggers an ``auction``).
        business: A versatile phase usually occurring at the end of a turn
            or before a roll. The player can build/demolish houses, mortgage/unmortgage
            properties, or finalize their board state before passing the turn.
        liquidation: An emergency phase triggered when a player owes a debt

```

```

        (to the bank or another player) that exceeds their current liquid cash.
        They are forced to sell assets or mortgage properties to cover the debt,
        or face bankruptcy.
    auction: A competitive, multi-player phase triggered when a property
        is declined in the ``management`` phase. The standard turn loop pauses, and
        players take turns placing bids until a winner is determined.
    proposal_acceptance: An interruptive phase triggered when one player
        sends a trade request to another. The game loop pauses, the
        ``active_phase_player`` switches to the recipient, and they must either
        accept or decline the pending ``proposal``.
    end_game: A terminal state indicating the match has concluded, usually
        because all other players have gone bankrupt. No further actions can
        be taken.

    """
    roll_the_dices = 'roll_the_dices'
    choose_square = 'choose_square'
    choose_fantasy = 'choose_fantasy'
    management = 'management'
    liquidation = 'liquidation'
    business = 'business'
    auction = 'auction'
    proposal_acceptance = 'proposal_acceptance'
    end_game = 'end_game'

phase = models.CharField(choices=GamePhase, max_length=20, default='roll_the_dices')
players = models.ManyToManyField('CustomUser', related_name='active_playing')
# ordered_player = [pk1, pk2, pk3, ...]
ordered_players = models.JSONField(default=list)
streak = models.IntegerField(default=0)
#dict[string,int], key=square_id, value=dice_combination to get there
possible_destinations = models.JSONField(default=dict, null=True, blank=True)
parking_money = models.PositiveIntegerField(default=0)
# Maps user_id -> uint
jail_remaining_turns = models.JSONField(default=dict, blank=True)
proposal = models.ForeignKey('ActionTradeProposal', on_delete=models.SET_NULL, null=True, blank=True, related_name='proposals')

fantasy_event = models.ForeignKey('FantasyEvent', on_delete=models.SET_NULL, null=True, blank=True, related_name='events')

current_auction = models.ForeignKey('Auction', on_delete=models.SET_NULL, null=True, blank=True, related_name='current_auctions')

finished = models.BooleanField(default=False)
bonus_response = models.ForeignKey('ResponseBonus', on_delete=models.SET_NULL, null=True, blank=True, related_name='bonus_responses')

kick_out_task_id = models.CharField(max_length=255, null=True, blank=True)
next_phase_task_id = models.CharField(max_length=255, null=True, blank=True)
auction_task_id = models.CharField(max_length=255, null=True, blank=True)

current_turn = models.PositiveIntegerField(default=1)

current_round = models.PositiveIntegerField(default=1)
max_rounds = models.PositiveIntegerField(default=NUM_ROUNDS) # 0 for unlimited

```

GamePhase

Bases: TextChoices

Enumeration of the possible states (phases) within a game's turn cycle.

Attributes:

| Name | Type | Description |
|----------------------------------|-------------------------|--|
| <code>roll_the_dices</code> | | The initial phase of a turn. The <code>active_turn_player</code> must throw the dice to determine movement. |
| <code>choose_square</code> | | Triggered when a player's movement path presents a fork or routing option (e.g., deciding whether to take a tram/subway line). The player must select their specific destination square. |
| <code>choose_fantasy</code> | | Triggered when a player lands on a dynamic event square The player must acknowledge and resolve the active <code>fantasy_event</code> . |
| <code>management</code> | | Triggered when a player lands on an unowned property. The player must choose to either purchase the property at its list price or decline the purchase (which typically immediately triggers an <code>auction</code>). |
| <code>business</code> | | A versatile phase usually occurring at the end of a turn or before a roll. The player can build/demolish houses, mortgage/unmortgage properties, or finalize their board state before passing the turn. |
| <code>liquidation</code> | | An emergency phase triggered when a player owes a debt (to the bank or another player) that exceeds their current liquid cash. They are forced to sell assets or mortgage properties to cover the debt, or face bankruptcy. |
| <code>auction</code> | <code>management</code> | A competitive, multi-player phase triggered when a property is declined in the <code>management</code> phase. The standard turn loop pauses, and players take turns placing bids until a winner is determined. |
| <code>proposal_acceptance</code> | | An interruptive phase triggered when one player sends a trade request to another. The game loop pauses, the <code>active_phase_player</code> switches to the recipient, and they must either accept or decline the pending <code>proposal</code> . |
| <code>end_game</code> | | A terminal state indicating the match has concluded, usually because all other players have gone bankrupt. No further actions can be taken. |

• Source code in `magnate/models.py`

```
class GamePhase(models.TextChoices):
    """
    Enumeration of the possible states (phases) within a game's turn cycle.

    Attributes:
        roll_the_dices: The initial phase of a turn. The ``active_turn_player``
            must throw the dice to determine movement.
        choose_square: Triggered when a player's movement path presents a fork
            or routing option (e.g., deciding whether to take a tram/subway line).
            The player must select their specific destination square.
        choose_fantasy: Triggered when a player lands on a dynamic event square
            The player must acknowledge and resolve the active ``fantasy_event``.
        management: Triggered when a player lands on an unowned property. The
            player must choose to either purchase the property at its list price
            or decline the purchase (which typically immediately triggers an ``auction``).
        business: A versatile phase usually occurring at the end of a turn
            or before a roll. The player can build/demolish houses, mortgage/unmortgage
            properties, or finalize their board state before passing the turn.
        liquidation: An emergency phase triggered when a player owes a debt
            (to the bank or another player) that exceeds their current liquid cash.
```

```

        They are forced to sell assets or mortgage properties to cover the debt,
        or face bankruptcy.
    auction: A competitive, multi-player phase triggered when a property
        is declined in the ``management`` phase. The standard turn loop pauses, and
        players take turns placing bids until a winner is determined.
    proposal_acceptance: An interruptive phase triggered when one player
        sends a trade request to another. The game loop pauses, the
        ``active_phase_player`` switches to the recipient, and they must either
        accept or decline the pending ``proposal``.
    end_game: A terminal state indicating the match has concluded, usually
        because all other players have gone bankrupt. No further actions can
        be taken.

"""
roll_the_dices = 'roll_the_dices'
choose_square = 'choose_square'
choose_fantasy = 'choose_fantasy'
management = 'management'
liquidation = 'liquidation'
business = 'business'
auction = 'auction'
proposal_acceptance = 'proposal_acceptance'
end_game = 'end_game'

```

2.3.2 GameStateSerializer

Bases: `ModelSerializer`

Serializes the game status allowing reconnection. It excludes certain fields from Game model and also includes active `property_relationships`. Example: A standard serialized response during the 'roll_the_dices' phase: `json`

```

{
    "id": 1,
    "datetime": "2026-04-06T18:30:00Z",
    "positions": {"42": 0, "85": 12},
    "money": {"42": 1500, "85": 1350},
    "active_phase_player": 42,
    "active_turn_player": 42,
    "phase": "roll_the_dices",
    "players": [42, 85],
    "ordered_players": [42, 85],
    "streak": 0,
    "possible_destinations": [],
    "parking_money": 200,
    "jail_remaining_turns": {'2': 3},
    "finished": false,
    "bonus_response": null,
    "current_turn": 5,
    "proposal": null,
    "property_relationships": [
        {"owner": 1, "square": 3, "houses": 2, "mortgage": False},
        {"owner": 2, "square": 4, "houses": 3, "mortgage": False}
    ]
}

```

```
],
```

```
}
```

• **Source code in** `magnate/serializers.py`

```
class GameStateSerializer(serializers.ModelSerializer):
    """
    Serializes the game status allowing reconnection. It excludes certain
    fields from Game model and also includes active `property_relationships`.
    Example:
        A standard serialized response during the 'roll_the_dices' phase:
        ```json
 {
 "id": 1,
 "datetime": "2026-04-06T18:30:00Z",
 "positions": {"42": 0, "85": 12},
 "money": {"42": 1500, "85": 1350},
 "active_phase_player": 42,
 "active_turn_player": 42,
 "phase": "roll_the_dices",
 "players": [42, 85],
 "ordered_players": [42, 85],
 "streak": 0,
 "possible_destinations": [],
 "parking_money": 200,
 "jail_remaining_turns": {'2': 3},
 "finished": false,
 "bonus_response": null,
 "current_turn": 5,
 "proposal": null,
 "property_relationships": [
 {"owner": 1, "square": 3, "houses": 2, "mortgage": False},
 {"owner": 2, "square": 4, "houses": 3, "mortgage": False}
],
 }
 """
 fantasy_event = FantasyEventSerializer(read_only=True)
 property_relationships = PropertyRelationshipSerializer(many=True, read_only=True)
 possible_destinations = serializers.SerializerMethodField()
 proposal = ActionTradeProposalSerializer(read_only=True)

 class Meta:
 model = Game
 exclude = ['current_auction',
 'bonus_response',
 'kick_out_task_id', 'next_phase_task_id']

 def get_possible_destinations(self, obj):
 if obj.possible_destinations and '__mock_dice__' not in obj.possible_destinations:
 return list([int(dest) for dest in obj.possible_destinations.keys()])
 return []
```

## 2.4 Actions

### 2.4.1 Action

Bases: `Model`

Base model representing a player action within a game.

All concrete action types inherit from this model, sharing the game context and the player who performed the action. The action log is queryable for replays, auditing, and bot decision-making.

#### Attributes:

| Name   | Type                                     | Description                                         |
|--------|------------------------------------------|-----------------------------------------------------|
| game   | <code>ForeignKey -&gt; Game</code>       | The game session in which the action was performed. |
| player | <code>ForeignKey -&gt; CustomUser</code> | The player who performed the action.                |

#### • Source code in `magnate/models.py`

```
class Action(models.Model):
 """
 Base model representing a player action within a game.

 All concrete action types inherit from this model, sharing the game context
 and the player who performed the action. The action log is queryable for
 replays, auditing, and bot decision-making.

 Attributes:
 game (ForeignKey -> Game): The game session in which the action was performed.
 player (ForeignKey -> CustomUser): The player who performed the action.
 """
 game = models.ForeignKey('Game', on_delete=models.CASCADE, related_name='in_game')
 player = models.ForeignKey('CustomUser', on_delete=models.CASCADE, related_name='made_by')
```

#### Serializer

Bases: `ModelSerializer`

Frontend Request Payload Example: ``json { "type": "Action", "game": 1, "player": 2, }

#### • Source code in `magnate/serializers.py`

```
class ActionSerializer(serializers.ModelSerializer):
 """
 Frontend Request Payload Example:
 ``json
 {
 "type": "Action",
 "game": 1,
 "player": 2,
 }
 """
 type = serializers.SerializerMethodField()
 class Meta:
 model = Action
```

```

fields = ['type', 'game', 'player']
def get_type(self, obj) -> str:
 """
 Retrieves the class name of the action instance.

 Args:
 obj (Action): The action instance.

 Returns:
 str: The name of the class.
 """
 return obj.__class__.__name__

```

### 2.4.2 get\_type(obj)

Retrieves the class name of the action instance.

#### Parameters:

| Name | Type                   | Description          | Default         |
|------|------------------------|----------------------|-----------------|
| obj  | <a href="#">Action</a> | The action instance. | <i>required</i> |

#### Returns:

#### Name Type Description

|     |     |                        |
|-----|-----|------------------------|
| str | str | The name of the class. |
|-----|-----|------------------------|

#### • Source code in [magnate/serializers.py](#)

```

def get_type(self, obj) -> str:
 """
 Retrieves the class name of the action instance.

 Args:
 obj (Action): The action instance.

 Returns:
 str: The name of the class.
 """
 return obj.__class__.__name__

```

### 2.4.3 ActionThrowDices

Bases: [Action](#)

Action representing a player throwing the dice to begin their turn.

No additional data is stored beyond the inherited [Action](#) fields; the actual dice values are computed server-side and returned in a [ResponseThrowDices](#).

• **Source code in** `magnate/models.py`

```
class ActionThrowDices(Action):
 """
 Action representing a player throwing the dice to begin their turn.

 No additional data is stored beyond the inherited ``Action`` fields; the
 actual dice values are computed server-side and returned in a ``ResponseThrowDices``.
 """
 pass
```

## Serializer

Bases: `ActionSerializer`

### Frontend Request Payload Example:

```
{
 "type": "ActionThrowDices",
 "game": 1,
 "player": 2,
}
```

• **Source code in** `magnate/serializers.py`

```
class ActionThrowDicesSerializer(ActionSerializer):
 """
 Frontend Request Payload Example:
    ```json
    {
      "type": "ActionThrowDices",
      "game": 1,
      "player": 2,
    }
    ```
 """
 class Meta(ActionSerializer.Meta):
 model = ActionThrowDices
 fields = ActionSerializer.Meta.fields
```

## 2.4.4 ActionMoveTo

Bases: `Action`

Action representing a player moving to a specific square after a dice roll.

### Attributes:

| Name                | Type                                     | Description                                                                |
|---------------------|------------------------------------------|----------------------------------------------------------------------------|
| <code>square</code> | <code>ForeignKey -&gt; BaseSquare</code> | The destination square the player moves to following normal dice movement. |



• **Source code in** `magnate/models.py`

```
class ActionMoveTo(Action):
 """
 Action representing a player moving to a specific square after a dice roll.

 Attributes:
 square (ForeignKey -> BaseSquare): The destination square the player
 moves to following normal dice movement.
 """
 square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='move_to')
```

Serializer

Bases: `ActionSerializer`

Frontend Request Payload Example:

```
{
 "type": "ActionThrowDices",
 "game": 1,
 "player": 2,
 "square": 101,
}
```

• **Source code in** `magnate/serializers.py`

```
class ActionMoveToSerializer(ActionSerializer):
 """
 Frontend Request Payload Example:
    ```json
    {
      "type": "ActionThrowDices",
      "game": 1,
      "player": 2,
      "square": 101,
    }
    ```
 """
 square = SquareCustomIdField()
 class Meta(ActionSerializer.Meta):
 model = ActionMoveTo
 fields = ActionSerializer.Meta.fields + ['square']
```

## 2.4.5 ActionTakeTram

Bases: `Action`

Action representing a player choosing to board the tram network.

Triggered during the `GamePhase.choose_square` phase when a player's dice roll lands them on or near a `TramSquare` with an available shortcut route.

**Attributes:**

| Name   | Type                     | Description                                                     |
|--------|--------------------------|-----------------------------------------------------------------|
| square | ForeignKey -> BaseSquare | The tram destination square the player has chosen to travel to. |

- **Source code in** `magnate/models.py`

```
class ActionTakeTram(Action):
 """
 Action representing a player choosing to board the tram network.

 Triggered during the ``GamePhase.choose_square`` phase when a player's dice
 roll lands them on or near a ``TramSquare`` with an available shortcut route.

 Attributes:
 square (ForeignKey -> BaseSquare): The tram destination square the player
 has chosen to travel to.
 """
 square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='tram_move_to')
```

**Serializer**

Bases: `ActionSerializer`

**Frontend Request Payload Example:**

```
{
 "type": "ActionTakeTram",
 "game": 1,
 "player": 2,
 "square": 200,
}
```

- **Source code in** `magnate/serializers.py`

```
class ActionTakeTramSerializer(ActionSerializer):
 """
 Frontend Request Payload Example:
    ```json
    {
      "type": "ActionTakeTram",
      "game": 1,
      "player": 2,
      "square": 200,
    }
    ```
 """
 square = SquareCustomIdField()
 class Meta(ActionSerializer.Meta):
 model = ActionTakeTram
 fields = ActionSerializer.Meta.fields + ['square']
```

**2.4.6 ActionDropPurchase**

Bases: [Action](#)

Action representing a player declining to buy an unowned property.

Declining immediately triggers an [Auction](#) for the property, entering the [GamePhase.auction](#) phase.

### Attributes:

| Name   | Type                     | Description                     |
|--------|--------------------------|---------------------------------|
| square | ForeignKey -> BaseSquare | The property that was declined. |

#### • Source code in [magnate/models.py](#)

```
class ActionDropPurchase(Action):
 """
 Action representing a player declining to buy an unowned property.

 Declining immediately triggers an ``Auction`` for the property, entering the
 ``GamePhase.auction`` phase.

 Attributes:
 square (ForeignKey -> BaseSquare): The property that was declined.
 """
 square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='dropped')
```

### Serializer

Bases: [ActionSerializer](#)

### Frontend Request Payload Example:

```
{
 "type": "ActionDropPurchase",
 "game": 1,
 "player": 2,
 "square": 15
}
```

#### • Source code in [magnate/serializers.py](#)

```
class ActionDropPurchaseSerializer(ActionSerializer):
 """
 Frontend Request Payload Example:
 ``json
 {
 "type": "ActionDropPurchase",
 "game": 1,
 "player": 2,
 "square": 15
 }
 """
 square = SquareCustomIdField()
 class Meta(ActionSerializer.Meta):
```

```
model = ActionDropPurchase
fields = ActionSerializer.Meta.fields + ['square']
```

## 2.4.7 ActionBuySquare

Bases: [Action](#)

Action representing a player purchasing an unowned property from the bank.

The property's `buy_price` is deducted from the player's balance and a `PropertyRelationship` record is created.

### Attributes:

| Name   | Type                     | Description                   |
|--------|--------------------------|-------------------------------|
| square | ForeignKey -> BaseSquare | The property being purchased. |

#### • Source code in [magnate/models.py](#)

```
class ActionBuySquare(Action):
 """
 Action representing a player purchasing an unowned property from the bank.

 The property's ``buy_price`` is deducted from the player's balance and a
 ``PropertyRelationship`` record is created.

 Attributes:
 square (ForeignKey -> BaseSquare): The property being purchased.
 """
 square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='bought')
```

### Serializer

Bases: [ActionSerializer](#)

### Frontend Request Payload Example:

```
{
 "type": "ActionBuySquare",
 "game": 1,
 "player": 2,
 "square": 15
}
```

#### • Source code in [magnate/serializers.py](#)

```
class ActionBuySquareSerializer(ActionSerializer):
 """
 Frontend Request Payload Example:
    ```json
    {
      "type": "ActionBuySquare",
      "game": 1,
      "player": 2,
```

```

    "square": 15
}
...
"""
square = SquareCustomIdField()
class Meta(ActionSerializer.Meta):
    model = ActionBuySquare
    fields = ActionSerializer.Meta.fields + ['square']

```

2.4.8 ActionBuild

Bases: [Action](#)

Action representing a player building houses or a hotel on an owned property.

Only valid during `GamePhase.business` when the player owns the full colour group. Building is limited by the even-build rule (houses must be distributed evenly across the group).

Attributes:

| Name | Type | Description |
|--------|--------------------------|---|
| houses | IntegerField | The number of houses being added in this action. A value of <code>5</code> indicates a hotel is being placed. |
| square | ForeignKey -> BaseSquare | The property on which construction takes place. |

• Source code in `magnate/models.py`

```

class ActionBuild(Action):
    """
    Action representing a player building houses or a hotel on an owned property.

    Only valid during ``GamePhase.business`` when the player owns the full colour
    group. Building is limited by the even-build rule (houses must be distributed
    evenly across the group).

    Attributes:
        houses (IntegerField): The number of houses being added in this action.
            A value of ``5`` indicates a hotel is being placed.
        square (ForeignKey -> BaseSquare): The property on which construction takes place.
    """
    houses = models.IntegerField(default=1)
    square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='build_square')

```

Serializer

Bases: [ActionSerializer](#)

Frontend Request Payload Example:

```

{
  "type": "ActionBuild",
  "game": 1,

```

```
"player": 2,
"houses": 1,
"square": 12
}
```

• Source code in `magnate/serializers.py`

```
class ActionBuildSerializer(ActionSerializer):
    """
    Frontend Request Payload Example:
    ```json
 {
 "type": "ActionBuild",
 "game": 1,
 "player": 2,
 "houses": 1,
 "square": 12
 }
    ```
    """
    square = SquareCustomIdField()
    class Meta(ActionSerializer.Meta):
        model = ActionBuild
        fields = ActionSerializer.Meta.fields + ['houses', 'square']
```

2.4.9 ActionDemolish

Bases: `Action`

Action representing a player demolishing houses or a hotel on an owned property.

Demolishing returns a portion of the build cost. The even-build rule also applies to demolition (houses must be removed evenly across the group).

Attributes:

| Name | Type | Description |
|--------|--|--|
| houses | <code>IntegerField</code> | The number of houses being removed in this action. |
| square | <code>ForeignKey -> BaseSquare</code> | The property from which construction is removed. |

• Source code in `magnate/models.py`

```
class ActionDemolish(Action):
    """
    Action representing a player demolishing houses or a hotel on an owned property.

    Demolishing returns a portion of the build cost. The even-build rule also
    applies to demolition (houses must be removed evenly across the group).

    Attributes:
        houses (IntegerField): The number of houses being removed in this action.
        square (ForeignKey -> BaseSquare): The property from which construction is removed.
    """
```

```
houses = models.IntegerField(default=0)
square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='demolish_square')
```

Serializer

Bases: [ActionSerializer](#)

Frontend Request Payload Example:

```
{
  "type": "ActionDemolish",
  "game": 1,
  "player": 2,
  "houses": 1,
  "square": 12
}
```

• Source code in [magnate/serializers.py](#)

```
class ActionDemolishSerializer(ActionSerializer):
    """
    Frontend Request Payload Example:
    ```json
 {
 "type": "ActionDemolish",
 "game": 1,
 "player": 2,
 "houses": 1,
 "square": 12
 }
    ```
    """
    square = SquareCustomIdField()
    class Meta(ActionSerializer.Meta):
        model = ActionDemolish
        fields = ActionSerializer.Meta.fields + ['houses', 'square']
```

2.4.10 ActionChooseCard

Bases: [Action](#)

Action representing a player interacting with a fantasy card draw.

Players may have the option to reveal a card they purchased in advance (`chosen_revealed_card = True`) or draw a random card from the deck.

Attributes:

| Name | Type | Description |
|-----------------------------------|---------------------------|--|
| <code>chosen_revealed_card</code> | <code>BooleanField</code> | <code>True</code> if the player chose to play a pre-purchased revealed card; <code>False</code> if drawing randomly from the deck. |

• **Source code in** `magnate/models.py`

```
class ActionChooseCard(Action):
    """
    Action representing a player interacting with a fantasy card draw.

    Players may have the option to reveal a card they purchased in advance
    (``chosen_revealed_card = True``) or draw a random card from the deck.

    Attributes:
        chosen_revealed_card (BooleanField): ``True`` if the player chose to play
        a pre-purchased revealed card; ``False`` if drawing randomly from the deck.
    """
    chosen_revealed_card = models.BooleanField(default=False)
```

Serializer

Bases: `ActionSerializer`

Frontend Request Payload Example:

```
{
  "type": "ActionChooseCard",
  "game": 1,
  "player": 2,
  "chosen_revealed_card": true
}
```

• **Source code in** `magnate/serializers.py`

```
class ActionChooseCardSerializer(ActionSerializer):
    """
    Frontend Request Payload Example:
    ``json
    {
      "type": "ActionChooseCard",
      "game": 1,
      "player": 2,
      "chosen_revealed_card": true
    }
    ``
    """
    class Meta(ActionSerializer.Meta):
        model = ActionChooseCard
        fields = ActionSerializer.Meta.fields + ['chosen_revealed_card']
```

2.4.11 ActionSurrender

Bases: `Action`

Action representing a player voluntarily quitting the game.

The player's assets are returned to the bank and they are removed from `Game.players`. No additional data beyond the inherited `Action` fields is required.

• **Source code in** `magnate/models.py`

```
class ActionSurrender(Action):
    """
    Action representing a player voluntarily quitting the game.

    The player's assets are returned to the bank and they are removed from
    ``Game.players``. No additional data beyond the inherited ``Action`` fields
    is required.
    """
    pass
```

Serializer

Bases: `ActionSerializer`

Frontend Request Payload Example:

```
{
  "type": "ActionSurrender",
  "game": 1,
  "player": 2
}
```

• **Source code in** `magnate/serializers.py`

```
class ActionSurrenderSerializer(ActionSerializer):
    """
    Frontend Request Payload Example:
    ``json
    {
      "type": "ActionSurrender",
      "game": 1,
      "player": 2
    }
    ``
    """
    class Meta(ActionSerializer.Meta):
        model = ActionSurrender
        fields = ActionSerializer.Meta.fields
```

2.4.12 ActionTradeProposal

Bases: `Action`

Action representing a trade offer sent from one player to another.

Creates a pending `proposal` on the `Game`, pausing the game in `GamePhase.proposal_acceptance` until the recipient responds.

Attributes:

| Name | Type | Description |
|--------------------|---|--|
| destination_user | ForeignKey -> CustomUser | The player receiving the trade offer. |
| offered_money | PositiveIntegerField | Amount of money the proposing player is offering to give to the destination player. |
| asked_money | PositiveIntegerField | Amount of money the proposing player is requesting from the destination player. |
| offered_properties | ManyToManyField -> PropertyRelationship | Properties the proposing player is offering to transfer to the destination player. |
| asked_properties | ManyToManyField -> PropertyRelationship | Properties the proposing player is requesting from the destination player in return. |

• Source code in
[magnate/models.py](#)

```
class ActionTradeProposal(Action):
    """
    Action representing a trade offer sent from one player to another.

    Creates a pending ``proposal`` on the ``Game``, pausing the game in
    ``GamePhase.proposal_acceptance`` until the recipient responds.

    Attributes:
        destination_user (ForeignKey -> CustomUser): The player receiving the trade offer.
        offered_money (PositiveIntegerField): Amount of money the proposing player
            is offering to give to the destination player.
        asked_money (PositiveIntegerField): Amount of money the proposing player
            is requesting from the destination player.
        offered_properties (ManyToManyField -> PropertyRelationship): Properties the
            proposing player is offering to transfer to the destination player.
        asked_properties (ManyToManyField -> PropertyRelationship): Properties the
            proposing player is requesting from the destination player in return.
    """
    destination_user = models.ForeignKey('CustomUser', on_delete=models.CASCADE, related_name='destination_user')
    offered_money = models.PositiveIntegerField(default=0)
    asked_money = models.PositiveIntegerField(default=0)
    offered_properties = models.ManyToManyField('PropertyRelationship', related_name='offered_properties')
    asked_properties = models.ManyToManyField('PropertyRelationship', related_name='asked_properties')
```

Serializer
Bases: [ActionSerializer](#)
Frontend Request Payload Example:

```
{
  "type": "ActionTradeProposal",
  "game": 1,
  "player": 2,
  "destination_user": 3,
  "offered_money": 200,
  "asked_money": 0,
  "offered_properties": [5, 6],
```

```
"asked_properties": [12]
}
```

• Source code in `magnate/serializers.py`

```
class ActionTradeProposalSerializer(ActionSerializer):
    """
    Frontend Request Payload Example:
    ```json
 {
 "type": "ActionTradeProposal",
 "game": 1,
 "player": 2,
 "destination_user": 3,
 "offered_money": 200,
 "asked_money": 0,
 "offered_properties": [5, 6],
 "asked_properties": [12]
 }
    ```
    """
    offered_properties = serializers.ListField(
        child=serializers.IntegerField(),
        required=False,
        default=list,
        write_only=True
    )
    asked_properties = serializers.ListField(
        child=serializers.IntegerField(),
        required=False,
        default=list,
        write_only=True
    )

    class Meta(ActionSerializer.Meta):
        model = ActionTradeProposal
        fields = ActionSerializer.Meta.fields + ['destination_user', 'offered_money', 'asked_money', 'offered_properties', 'asked_properties']

    def to_representation(self, instance):
        ret = super().to_representation(instance)
        # convert prop rel to custom id for frontend compatibility
        ret['offered_properties'] = [rel.square.custom_id for rel in instance.offered_properties.all()]
        ret['asked_properties'] = [rel.square.custom_id for rel in instance.asked_properties.all()]
        return ret

    def create(self, validated_data) -> ActionTradeProposal:
        """
        Creates an ActionTradeProposal instance and sets Many-to-Many properties.

        Args:
            validated_data (dict): The validated data.

        Returns:
            ActionTradeProposal: The created instance.
        """
        offered_custom_ids = validated_data.pop('offered_properties', [])
        asked_custom_ids = validated_data.pop('asked_properties', [])

        game = validated_data.get('game')
        player = validated_data.get('player')
        destination_user = validated_data.get('destination_user')
```

```

# validate
if offered_custom_ids:
    offered_rels = PropertyRelationship.objects.filter(
        game=game,
        owner=player,
        square__custom_id__in=offered_custom_ids
    )

    if offered_rels.count() != len(offered_custom_ids):
        raise serializers.ValidationError({
            "offered_properties": "Has incluido propiedades que no posees o no existen."
        })
else:
    offered_rels = []

if asked_custom_ids:
    asked_rels = PropertyRelationship.objects.filter(
        game=game,
        owner=destination_user,
        square__custom_id__in=asked_custom_ids
    )
    if asked_rels.count() != len(asked_custom_ids):
        raise serializers.ValidationError({
            "asked_properties": "Has pedido propiedades que el destinatario no posee o no existen."
        })
else:
    asked_rels = []

# instance
instance = ActionTradeProposal.objects.create(**validated_data)

# convert custom id to prop rel
if offered_rels:
    instance.offered_properties.set(offered_rels)
if asked_rels:
    instance.asked_properties.set(asked_rels)

return instance

```

2.4.13 create(validated_data)

Creates an ActionTradeProposal instance and sets Many-to-Many properties.

Parameters:

| Name | Type | Description | Default |
|----------------|------|-------------------------------------|---------|
| validated_data | dict | The validated data. <i>required</i> | |

Returns:

| Name | Type | Description |
|---------------------|-------------------------------------|-----------------------|
| ActionTradeProposal | ActionTradeProposal | The created instance. |

• Source code in `magnate/serializers.py`

```

def create(self, validated_data) -> ActionTradeProposal:
    """

```

Creates an ActionTradeProposal instance and sets Many-to-Many properties.

Args:

validated_data (dict): The validated data.

Returns:

ActionTradeProposal: The created instance.

```
"""
offered_custom_ids = validated_data.pop('offered_properties', [])
asked_custom_ids = validated_data.pop('asked_properties', [])

game = validated_data.get('game')
player = validated_data.get('player')
destination_user = validated_data.get('destination_user')

# validate
if offered_custom_ids:
    offered_rels = PropertyRelationship.objects.filter(
        game=game,
        owner=player,
        square__custom_id__in=offered_custom_ids
    )

    if offered_rels.count() != len(offered_custom_ids):
        raise serializers.ValidationError({
            "offered_properties": "Has incluido propiedades que no posees o no existen."
        })
else:
    offered_rels = []

if asked_custom_ids:
    asked_rels = PropertyRelationship.objects.filter(
        game=game,
        owner=destination_user,
        square__custom_id__in=asked_custom_ids
    )

    if asked_rels.count() != len(asked_custom_ids):
        raise serializers.ValidationError({
            "asked_properties": "Has pedido propiedades que el destinatario no posee o no existen."
        })
else:
    asked_rels = []

# instance
instance = ActionTradeProposal.objects.create(**validated_data)

# convert custom id to prop rel
if offered_rels:
    instance.offered_properties.set(offered_rels)
if asked_rels:
    instance.asks_properties.set(asked_rels)

return instance
```

2.4.14 ActionTradeAnswer

Bases: [Action](#)

Action representing a player's response to a pending trade proposal.

Resolves the `GamePhase.proposal_acceptance` phase. If accepted, properties and money are transferred between the two parties.

Attributes:

| Name | Type | Description |
|--------|--------------|--|
| choose | BooleanField | True if the trade was accepted; False if declined. |

• Source code in `magnate/models.py`

```
class ActionTradeAnswer(Action):
    """
    Action representing a player's response to a pending trade proposal.

    Resolves the ``GamePhase.proposal_acceptance`` phase. If accepted, properties
    and money are transferred between the two parties.

    Attributes:
        choose (BooleanField): ``True`` if the trade was accepted; ``False`` if declined.
    """
    choose = models.BooleanField(default=False)
```

Serializer

Bases: [ActionSerializer](#)

Frontend Request Payload Example:

```
{
  "type": "ActionTradeAnswer",
  "game": 1,
  "player": 2,
  "choose": true,
}
```

• Source code in `magnate/serializers.py`

```
class ActionTradeAnswerSerializer(ActionSerializer):
    """
    Frontend Request Payload Example:
    ``json
    {
      "type": "ActionTradeAnswer",
      "game": 1,
      "player": 2,
      "choose": true,
    }
    ...
    """
    class Meta(ActionSerializer.Meta):
```

```
model = ActionTradeAnswer
fields = ActionSerializer.Meta.fields + ['choose']
```

2.4.15 ActionMortgageSet

Bases: [Action](#)

Action representing a player mortgaging one of their properties.

The player receives half the property's `buy_price` from the bank. While mortgaged, no rent is collected and no houses can be built on the property.

Attributes:

| Name | Type | Description |
|--------|--------------------------|-------------------------------|
| square | ForeignKey -> BaseSquare | The property being mortgaged. |

• Source code in [magnate/models.py](#)

```
class ActionMortgageSet(Action):
    """
    Action representing a player mortgaging one of their properties.

    The player receives half the property's ``buy_price`` from the bank. While
    mortgaged, no rent is collected and no houses can be built on the property.

    Attributes:
        square (ForeignKey -> BaseSquare): The property being mortgaged.
    """
    square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='mortgage_set_square')
```

Serializer

Bases: [ActionSerializer](#)

Frontend Request Payload Example:

```
{
  "type": "ActionMortgageSet",
  "game": 1,
  "player": 2,
  "square": 15
}
```

• Source code in [magnate/serializers.py](#)

```
class ActionMortgageSetSerializer(ActionSerializer):
    """
    Frontend Request Payload Example:
    ``json
    {
      "type": "ActionMortgageSet",
      "game": 1,
```

```

    "player": 2,
    "square": 15
}
...
"""
square = SquareCustomIdField()
class Meta(ActionSerializer.Meta):
    model = ActionMortgageSet
    fields = ActionSerializer.Meta.fields + ['square']

```

2.4.16 ActionMortgageUnset

Bases: [Action](#)

Action representing a player lifting the mortgage on one of their properties.

The player pays the bank to reinstate the property (typically the mortgage value plus an interest surcharge). The property can then collect rent again.

Attributes:

| Name | Type | Description |
|--------|--------------------------|---------------------------------|
| square | ForeignKey -> BaseSquare | The property being unmortgaged. |

• Source code in [magnate/models.py](#)

```

class ActionMortgageUnset(Action):
    """
    Action representing a player lifting the mortgage on one of their properties.

    The player pays the bank to reinstate the property (typically the mortgage
    value plus an interest surcharge). The property can then collect rent again.

    Attributes:
        square (ForeignKey -> BaseSquare): The property being unmortgaged.
    """
    square = models.ForeignKey('BaseSquare', on_delete=models.CASCADE, related_name='mortgage_unset_square')

```

Serializer

Bases: [ActionSerializer](#)

Frontend Request Payload Example:

```

{
  "type": "ActionMortgageUnset",
  "game": 1,
  "player": 2,
  "square": 15
}

```


• **Source code in** `magnate/serializers.py`

```
class ActionMortgageUnsetSerializer(ActionSerializer):
    """
    Frontend Request Payload Example:
    ```json
 {
 "type": "ActionMortgageUnset",
 "game": 1,
 "player": 2,
 "square": 15
 }
    ```
    """
    square = SquareCustomIdField()
    class Meta(ActionSerializer.Meta):
        model = ActionMortgageUnset
        fields = ActionSerializer.Meta.fields + ['square']
```

2.4.17 ActionPayBail

Bases: `Action`

Action representing a player paying the bail fee to leave jail immediately.

The player's balance is reduced by `JailSquare.bail_price` and they are released to resume normal turns. No additional data is needed beyond the inherited `Action` fields. Attributes: `to_pay` (BooleanField): Whether to pay the bill or not

• **Source code in** `magnate/models.py`

```
class ActionPayBail(Action):
    """
    Action representing a player paying the bail fee to leave jail immediately.

    The player's balance is reduced by ``JailSquare.bail_price`` and they are
    released to resume normal turns. No additional data is needed beyond the
    inherited ``Action`` fields.
    Attributes:
        to_pay (BooleanField): Whether to pay the bill or not
    """
    to_pay = models.BooleanField(default=False)
```

Serializer

Bases: `ActionSerializer`

Frontend Request Payload Example:

```
{
    "type": "ActionPayBail",
    "game": 1,
```

```
"player": 2
}
```

• Source code in `magnate/serializers.py`

```
class ActionPayBailSerializer(ActionSerializer):
    """
    Frontend Request Payload Example:
    ``json
    {
        "type": "ActionPayBail",
        "game": 1,
        "player": 2
    }
    """
    class Meta(ActionSerializer.Meta):
        model = ActionPayBail
        fields = ActionSerializer.Meta.fields + ['to_pay']
```

2.4.18 ActionNextPhase

Bases: `Action`

Action representing a player explicitly advancing to the next game phase or ending their turn.

Used as a confirmation signal (e.g. ending the `business` phase) when the player has no further actions to perform. No additional data is needed beyond the inherited `Action` fields.

• Source code in `magnate/models.py`

```
class ActionNextPhase(Action):
    """
    Action representing a player explicitly advancing to the next game phase or ending their turn.

    Used as a confirmation signal (e.g. ending the ``business`` phase) when the player has
    no further actions to perform. No additional data is needed beyond the inherited
    ``Action`` fields.
    """
    pass
```

Serializer

Bases: `ActionSerializer`

Frontend Request Payload Example:

```
{
    "type": "ActionNextPhase",
    "game": 1,
    "player": 2
}
```

• **Source code in** `magnate/serializers.py`

```
class ActionNextPhaseSerializer(ActionSerializer):
    """
    Frontend Request Payload Example:
    ```json
 {
 "type": "ActionNextPhase",
 "game": 1,
 "player": 2
 }
    ```
    """
    class Meta(ActionSerializer.Meta):
        model = ActionNextPhase
        fields = ActionSerializer.Meta.fields
```

2.4.19 ActionBid

Bases: `Action`

Action representing a player placing a bid during an active auction.

Submitted bids are recorded in `Auction.bids`. The auction ends once all players have bid or the auction timer expires.

Attributes:

| Name | Type | Description |
|---------------------|-----------------------------------|-------------------------------------|
| <code>amount</code> | <code>PositiveIntegerField</code> | The bid amount in in-game currency. |

• **Source code in** `magnate/models.py`

```
class ActionBid(Action):
    """
    Action representing a player placing a bid during an active auction.

    Submitted bids are recorded in ``Auction.bids``. The auction ends once all
    players have bid or the auction timer expires.

    Attributes:
        amount (PositiveIntegerField): The bid amount in in-game currency.
    """
    amount = models.PositiveIntegerField(default=0)
```

Serializer

Bases: `ActionSerializer`

Frontend Request Payload Example:

```
{
    "type": "ActionBid",
```

```
"game": 1,  
"player": 2,  
"auction": 4,  
"amount": 150  
}
```

• **Source code in** `magnate/serializers.py`

```
class ActionBidSerializer(ActionSerializer):  
    """  
    Frontend Request Payload Example:  
    ``json  
    {  
        "type": "ActionBid",  
        "game": 1,  
        "player": 2,  
        "auction": 4,  
        "amount": 150  
    }  
    """  
  
    class Meta(ActionSerializer.Meta):  
        model = ActionBid  
        fields = ActionSerializer.Meta.fields + ['amount']
```

2.5 Responses

2.5.1 Response

Bases: `Model`

Base model for game state snapshots broadcasted to all connected clients.

Each concrete `Response` subclass corresponds to a specific game event and carries enough state for the frontend to render the updated board without needing to re-fetch the full `Game` object.

Attributes:

| Name | Type | Description |
|----------------------------------|---|--|
| <code>money</code> | <code>JSONField</code> | Current balances for all players, mapping user ID (str) to amount (int). |
| <code>active_phase_player</code> | <code>ForeignKey -> CustomUser None</code> | The user expected to act next in the current micro-phase. |
| <code>active_turn_player</code> | <code>ForeignKey -> CustomUser None</code> | The user whose overall board turn it is. |
| <code>phase</code> | <code>CharField</code> | The <code>GamePhase</code> the game has transitioned to after this event. |
| <code>positions</code> | <code>JSONField</code> | Current board positions for all players, mapping user ID (str) to square <code>custom_id</code> (int). |

• Source code in `magnate/models.py`

```
class Response(models.Model):
    """
    Base model for game state snapshots broadcasted to all connected clients.

    Each concrete ``Response`` subclass corresponds to a specific game event and
    carries enough state for the frontend to render the updated board without
    needing to re-fetch the full ``Game`` object.

    Attributes:
        money (JSONField): Current balances for all players, mapping user ID (str)
            to amount (int).
        active_phase_player (ForeignKey -> CustomUser | None): The user expected to
            act next in the current micro-phase.
        active_turn_player (ForeignKey -> CustomUser | None): The user whose overall
            board turn it is.
        phase (CharField): The ``GamePhase`` the game has transitioned to after this event.
        positions (JSONField): Current board positions for all players, mapping user ID
            (str) to square ``custom_id`` (int).
    """
    money = models.JSONField(default=dict, blank=True)
    active_phase_player = models.ForeignKey('CustomUser', on_delete=models.SET_NULL, null=True, related_name='response')
    active_turn_player = models.ForeignKey('CustomUser', on_delete=models.SET_NULL, null=True, related_name='response')
    phase = models.CharField(choices=Game.GamePhase, max_length=20)
    positions = models.JSONField(default=dict, blank=True) # user -> custom_id (int)
    parking_money = models.PositiveIntegerField(default=0)
    current_round = models.PositiveIntegerField(default=1)
    max_rounds = models.PositiveIntegerField(default=2)
```

Serializer

Bases: `ModelSerializer`

Frontend Response Payload Example:

```
{
  "type": "Response",
  "money": {"1": 1500, "2": 1200},
  "active_phase_player": 2,
  "active_turn_player": 2,
  "phase": "management",
  "positions": {"1": 10, "2": 20}
}
```

• Source code in `magnate/serializers.py`

```
class ResponseSerializer(serializers.ModelSerializer):
    """
    Frontend Response Payload Example:
    ```json
 {
 "type": "Response",
 "money": {"1": 1500, "2": 1200},
 "active_phase_player": 2,
 "active_turn_player": 2,
 "phase": "management",
 "positions": {"1": 10, "2": 20}
 }
    ```
    """
    class Meta:
        model = Response
        exclude = ['id']
```

2.5.2 ResponseMovement

Bases: `Response`

Base response for any event that involves a player moving across the board.

Provides the ordered path of squares the player traversed so the frontend can animate the token movement step by step.

Attributes:

| Name | Type | Description |
|----------------------------|---|--|
| <code>path</code> | <code>JSONField</code> | Ordered list of square <code>custom_id</code> values the player passed through, from their starting position to their final destination. |
| <code>fantasy_event</code> | <code>ForeignKey -> FantasyEvent None</code> | Set if the player's movement ended on a <code>FantasySquare</code> , indicating which event was drawn. |

• **Source code in** `magnate/models.py`

```
class ResponseMovement(Response):
    """
    Base response for any event that involves a player moving across the board.

    Provides the ordered path of squares the player traversed so the frontend
    can animate the token movement step by step.

    Attributes:
        path (JSONField): Ordered list of square ``custom_id`` values the player
            passed through, from their starting position to their final destination.
        fantasy_event (ForeignKey -> FantasyEvent | None): Set if the player's
            movement ended on a ``FantasySquare``, indicating which event was drawn.
    """
    path = models.JSONField(default=list, blank=True)
    fantasy_event = models.ForeignKey('FantasyEvent', on_delete=models.CASCADE, null=True, blank=True)
```

Serializer

Bases: `ModelSerializer`

Frontend Response Payload Example:

```
{
  "type": "ResponseMovement",
  "money": {"1": 1500, "2": 1200},
  "active_phase_player": 2,
  "active_turn_player": 2,
  "phase": "moving",
  "path": [10, 11, 12, 13, 14, 15],
  "fantasy_event": null
}
```

• **Source code in** `magnate/serializers.py`

```
class ResponseMovementSerializer(serializers.ModelSerializer):
    """
    Frontend Response Payload Example:
    ```json
 {
 "type": "ResponseMovement",
 "money": {"1": 1500, "2": 1200},
 "active_phase_player": 2,
 "active_turn_player": 2,
 "phase": "moving",
 "path": [10, 11, 12, 13, 14, 15],
 "fantasy_event": null
 }
    ```
    """
    class Meta(ResponseSerializer.Meta):
        model = ResponseMovement
```

2.5.3 ResponseThrowDices

Bases: [ResponseMovement](#)

Response delivering the result of a dice roll and valid movement destinations.

Extends [ResponseMovement](#) with dice values and routing information so the frontend can display the roll result and highlight reachable squares.

Attributes:

| Name | Type | Description |
|------------------------------|--------------------------------------|---|
| dice1 | PositiveIntegerField | Value of the first standard die (1-6). |
| dice2 | PositiveIntegerField | Value of the second standard die (1-6). |
| dice_bus | PositiveIntegerField | Value of the special bus/wildcard die, used for tram or alternative routing mechanics. |
| destinations | JSONField | Dict of possible destination squares the player can move to given the dice result. Each entry maps a square custom_id to the dice combination needed to reach it. |
| triple | BooleanField | True if all three dice showed the same value, triggering a special triple-dice effect. |
| streak | IntegerField | Updated consecutive-doubles count after this roll. Reaching 3 sends the player to jail. |

• Source code in [magnate/models.py](#)

```
class ResponseThrowDices(ResponseMovement):
    """
    Response delivering the result of a dice roll and valid movement destinations.

    Extends ``ResponseMovement`` with dice values and routing information so the
    frontend can display the roll result and highlight reachable squares.

    Attributes:
        dice1 (PositiveIntegerField): Value of the first standard die (1-6).
        dice2 (PositiveIntegerField): Value of the second standard die (1-6).
        dice_bus (PositiveIntegerField): Value of the special bus/wildcard die, used
            for tram or alternative routing mechanics.
        destinations (JSONField): Dict of possible destination squares the player can
            move to given the dice result. Each entry maps a square ``custom_id`` to
            the dice combination needed to reach it.
        triple (BooleanField): ``True`` if all three dice showed the same value,
            triggering a special triple-dice effect.
        streak (IntegerField): Updated consecutive-doubles count after this roll.
            Reaching 3 sends the player to jail.
    """
    dice1 = models.PositiveIntegerField(default=0)
    dice2 = models.PositiveIntegerField(default=0)
    dice_bus = models.PositiveIntegerField(default=0)
    destinations = models.JSONField(default=list, blank=True)
    triple = models.BooleanField(default=False)
    streak = models.IntegerField(default=0)
```


Serializer

Bases: `ModelSerializer`

Frontend Response Payload Example:

```
{
  "type": "ResponseThrowDices",
  "money": {"1": 1500, "2": 1200},
  "active_phase_player": 2,
  "active_turn_player": 2,
  "phase": "moving",
  "path": [10, 11, 12],
  "fantasy_event": null,
  "dice1": 4,
  "dice2": 3,
  "dice_bus": 1,
  "destinations": [17],
  "triple": false,
  "streak": 0
}
```

• Source code in `magnate/serializers.py`

```
class ResponseThrowDicesSerializer(serializers.ModelSerializer):
    """
    Frontend Response Payload Example:
    ```json
 {
 "type": "ResponseThrowDices",
 "money": {"1": 1500, "2": 1200},
 "active_phase_player": 2,
 "active_turn_player": 2,
 "phase": "moving",
 "path": [10, 11, 12],
 "fantasy_event": null,
 "dice1": 4,
 "dice2": 3,
 "dice_bus": 1,
 "destinations": [17],
 "triple": false,
 "streak": 0
 }
    ```
    """
    fantasy_event = FantasyEventSerializer(read_only=True)
    class Meta(ResponseSerializer.Meta):
        model = ResponseThrowDices
```

2.5.4 ResponseChooseSquare

Bases: `ResponseMovement`

Response confirming a player's movement to a specifically chosen square.

Used after the player selects their destination during the `GamePhase.choose_square` phase (e.g. after deciding to take the tram). Inherits path animation data from `ResponseMovement`.

• **Source code in** `magnate/models.py`

```
class ResponseChooseSquare(ResponseMovement):
    """
    Response confirming a player's movement to a specifically chosen square.

    Used after the player selects their destination during the ``GamePhase.choose_square``
    phase (e.g. after deciding to take the tram). Inherits path animation data from
    ``ResponseMovement``.
    """
    pass
```

Serializer

Bases: `ModelSerializer`

Frontend Response Payload Example:

```
{
  "type": "ResponseChooseSquare",
  "money": {"1": 1500, "2": 1200},
  "active_phase_player": 2,
  "active_turn_player": 2,
  "phase": "moving",
  "path": [10, 25],
  "fantasy_event": null
}
```

• **Source code in** `magnate/serializers.py`

```
class ResponseChooseSquareSerializer(serializers.ModelSerializer):
    """
    Frontend Response Payload Example:
    ```json
 {
 "type": "ResponseChooseSquare",
 "money": {"1": 1500, "2": 1200},
 "active_phase_player": 2,
 "active_turn_player": 2,
 "phase": "moving",
 "path": [10, 25],
 "fantasy_event": null
 }
    ```
    """
    fantasy_event = FantasyEventSerializer(read_only=True)
    class Meta(ResponseSerializer.Meta):
        model = ResponseChooseSquare
```

2.5.5 ResponseChooseFantasy

Bases: [Response](#)

Response delivering the resolved outcome of a drawn fantasy card.

Sent after the server resolves the [FantasyEvent](#) the player triggered by landing on a [FantasySquare](#).

Attributes:

| Name | Type | Description |
|--------------------------------|--|---|
| fantasy_result | ForeignKey -> FantasyResult | The concrete result of the fantasy event, including the effect type and any numeric outcomes. |

• Source code in [magnate/models.py](#)

```
class ResponseChooseFantasy(Response):
    """
    Response delivering the resolved outcome of a drawn fantasy card.

    Sent after the server resolves the ``FantasyEvent`` the player triggered by
    landing on a ``FantasySquare``.

    Attributes:
        fantasy_result (ForeignKey -> FantasyResult): The concrete result of the
            fantasy event, including the effect type and any numeric outcomes.
    """
    fantasy_result = models.ForeignKey('FantasyResult', on_delete=models.CASCADE)
```

Serializer

Bases: [ModelSerializer](#)

Frontend Response Payload Example:

```
{'type': 'ResponseChooseFantasy',
 'fantasy_result': {'fantasy_event': {'fantasy_type': 'freeHouse', 'value': None, 'card_cost': 80}, 'result': None},
 'money': {'1': 1440, '2': 1500},
 'phase': 'business',
 'positions': {'1': '10', '2': '17'},
 'active_phase_player': 2,
 'active_turn_player': 2
}
```

• Source code in [magnate/serializers.py](#)

```
class ResponseChooseFantasySerializer(serializers.ModelSerializer):
    """
    Frontend Response Payload Example:
    ```json
 {'type': 'ResponseChooseFantasy',
 'fantasy_result': {'fantasy_event': {'fantasy_type': 'freeHouse', 'value': None, 'card_cost': 80}, 'result': Nor
 'money': {'1': 1440, '2': 1500},
 'phase': 'business',
 'positions': {'1': '10', '2': '17'},
```

```

 'active_phase_player': 2,
 'active_turn_player': 2
}
'''
"""
fantasy_result = FantasyResultSerializer(read_only=True)
class Meta(ResponseSerializer.Meta):
 model = ResponseChooseFantasy

```

## 2.5.6 ResponseAuction

Bases: [Response](#)

Response updating all clients with the current or final state of an auction.

Sent after every bid and upon auction conclusion. Exposes auction data as properties delegated to the linked [Auction](#) record.

### Attributes:

| Name                    | Type                                                     | Description                                 |
|-------------------------|----------------------------------------------------------|---------------------------------------------|
| <a href="#">auction</a> | <a href="#">OneToOneField</a> -> <a href="#">Auction</a> | The auction whose state is being broadcast. |

### • Properties

[winner](#): Delegates to [Auction.winner](#). [final\\_amount](#): Delegates to [Auction.final\\_amount](#). [is\\_tie](#): Delegates to [Auction.is\\_tie](#). [bids](#): Delegates to [Auction.bids](#).

### • Source code in [magnate/models.py](#)

```

class ResponseAuction(Response):
 """
 Response updating all clients with the current or final state of an auction.

 Sent after every bid and upon auction conclusion. Exposes auction data as
 properties delegated to the linked ``Auction`` record.

 Attributes:
 auction (OneToOneField -> Auction): The auction whose state is being broadcast.

 Properties:
 winner: Delegates to ``Auction.winner``.
 final_amount: Delegates to ``Auction.final_amount``.
 is_tie: Delegates to ``Auction.is_tie``.
 bids: Delegates to ``Auction.bids``.
 """
 auction = models.OneToOneField('Auction', on_delete=models.CASCADE, related_name='response')

 @property
 def winner(self):
 """
 Retrieves the winner of the auction.

 Returns:

```

```

 CustomUser | None: The winning user.
 """
 return self.auction.winner

@property
def final_amount(self):
 """
 Retrieves the final bid amount of the auction.

 Returns:
 int: The winning bid amount.
 """
 return self.auction.final_amount

@property
def is_tie(self):
 """
 Indicates if the auction ended in a tie.

 Returns:
 bool: True if tied, False otherwise.
 """
 return self.auction.is_tie

@property
def bids(self):
 """
 Retrieves all bids placed during the auction.

 Returns:
 dict: Mapping of user IDs to bid amounts.
 """
 return self.auction.bids

```

## 2.5.7 `bids` `property`

Retrieves all bids placed during the auction.

### Returns:

#### Name Type Description

| Name              | Type | Description                         |
|-------------------|------|-------------------------------------|
| <code>dict</code> |      | Mapping of user IDs to bid amounts. |

## 2.5.8 `final_amount` `property`

Retrieves the final bid amount of the auction.

### Returns:

#### Name Type Description

| Name             | Type | Description             |
|------------------|------|-------------------------|
| <code>int</code> |      | The winning bid amount. |

## 2.5.9 `is_tie` `property`

Indicates if the auction ended in a tie.

Returns:

Name Type Description

|        |                                |
|--------|--------------------------------|
| bool   | True if tied, False otherwise. |
| 2.5.10 | winner property                |

Retrieves the winner of the auction.

Returns:

Type Description

CustomUser | None: The winning user.  
Serializer

Bases: [ResponseSerializer](#)

Frontend Response Payload Example:

```
{
 "type": "ResponseAuction",
 "money": {"1": 1150, "2": 1200},
 "active_phase_player": 1,
 "active_turn_player": 2,
 "phase": "management",
 "winner": 1,
 "final_amount": 350,
 "is_tie": false,
 "bids": {1: 100, 2: 200}
}
```

• Source code in `magnate/serializers.py`

```
class ResponseAuctionSerializer(ResponseSerializer):
 """
 Frontend Response Payload Example:
    ```json
    {
      "type": "ResponseAuction",
      "money": {"1": 1150, "2": 1200},
      "active_phase_player": 1,
      "active_turn_player": 2,
      "phase": "management",
      "winner": 1,
      "final_amount": 350,
      "is_tie": false,
      "bids": {1: 100, 2: 200}
    }
    ```
 """
 auction = AuctionSerializer()
 class Meta(ResponseSerializer.Meta):
 model = ResponseAuction
```

2.5.11 ResponseSkipPhase

Bases: [Response](#)

Response broadcast when a game phase is skipped without any board movement.

Inherits all fields from [Response](#). Used for phase transitions where no positional or balance changes occur (e.g. skipping straight from one phase to another due to game logic).

• **Source code in** [magnate/models.py](#)

```
class ResponseSkipPhase(Response):
 """
 Response broadcast when a game phase is skipped without any board movement.

 Inherits all fields from ``Response``. Used for phase transitions where no
 positional or balance changes occur (e.g. skipping straight from one phase
 to another due to game logic).
 """
 pass
```

## Serializer

Bases: [ModelSerializer](#)

Frontend Response Payload Example:

```
{
 "type": "Response",
 "money": {"1": 1500, "2": 1200},
 "active_phase_player": 2,
 "active_turn_player": 2,
 "phase": "management"
}
```

• **Source code in** [magnate/serializers.py](#)

```
class ResponseSkipPhaseSerializer(serializers.ModelSerializer):
 """
 Frontend Response Payload Example:
 ``json
 {
 "type": "Response",
 "money": {"1": 1500, "2": 1200},
 "active_phase_player": 2,
 "active_turn_player": 2,
 "phase": "management"
 }
 """
 class Meta(ResponseSerializer.Meta):
 model = ResponseSkipPhase
```

## 2.5.12 ResponseBonus

Bases: `Response`

Response delivering bonus or penalty modifiers applied to a player at a specific game moment.

Used to broadcast end-of-game or milestone achievement bonuses (see `BonusCategory`) that affect player balances or scores.

#### Attributes:

| Name                 | Type                   | Description                                                                                                            |
|----------------------|------------------------|------------------------------------------------------------------------------------------------------------------------|
| <code>bonuses</code> | <code>JSONField</code> | Dict mapping user ID (str) to a bonus amount (int) or a nested structure describing multiple bonus categories awarded. |

#### • Source code in `magnate/models.py`

```
class ResponseBonus(Response):
 """
 Response delivering bonus or penalty modifiers applied to a player at a specific game moment.

 Used to broadcast end-of-game or milestone achievement bonuses (see ``BonusCategory``)
 that affect player balances or scores.

 Attributes:
 bonuses (JSONField): Dict mapping user ID (str) to a bonus amount (int) or a
 nested structure describing multiple bonus categories awarded.
 """
 bonuses = models.JSONField(default=dict, blank=True)
 last_money = models.JSONField(default=dict, blank=True)
```

#### Serializer

Bases: `ModelSerializer`

#### Frontend Response Payload Example:

```
{
 "type": "ResponseBonus",
 "money": {"1": 1600, "2": 1200},
 "active_phase_player": 2,
 "active_turn_player": 2,
 "phase": "management",
 "bonuses": {
 "walked_squares": {"display_name": "El más viajero", "bonus_amount": 200, "winners": [1, 3]},
 "built_houses": {"display_name": "El más constructor", "bonus_amount": 200, "winners": [2]},
 "num_trades": {"display_name": "El más trader", "bonus_amount": 200, "winners": []}
 }
}
```

#### • Source code in `magnate/serializers.py`

```
class ResponseBonusSerializer(serializers.ModelSerializer):
 """
 Frontend Response Payload Example:
 ``json
 {
```



```

"type": "ResponseBonus",
"money": {"1": 1600, "2": 1200},
"active_phase_player": 2,
"active_turn_player": 2,
"phase": "management",
"bonuses": {
 "walked_squares": {"display_name": "El más viajero", "bonus_amount": 200, "winners": [1, 3]},
 "built_houses": {"display_name": "El más constructor", "bonus_amount": 200, "winners": [2]},
 "num_trades": {"display_name": "El más trader", "bonus_amount": 200, "winners": []}}
}
...
"""
class Meta(ResponseSerializer.Meta):
 model = ResponseBonus

```

## 2.6 Fantasy events

### 2.6.1 FantasyEvent

Bases: `Model`

A Chance/Community-Chest style event card drawn when a player lands on a `FantasySquare`.

Each `FantasyEvent` encodes a specific effect type and an optional numeric value that parameterises the effect (e.g. the amount of money won or lost).

#### Attributes:

| Name                      | Type                                          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                           |                                               | The category of effect to apply. One of the <code>FantasyType</code> choices:                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|                           |                                               | <ul style="list-style-type: none"> <li><code>winPlainMoney</code> / <code>losePlainMoney</code>: Add or subtract a fixed <code>value</code> from the player's balance.</li> <li><code>winRatioMoney</code> / <code>loseRatioMoney</code>: Add or subtract a ratio of the player's current balance.</li> <li><code>breakOpponentHouse</code>: Destroy one house on a randomly chosen opponent's property.</li> <li><code>breakOwnHouse</code>: Destroy one house on the drawing player's own property.</li> <li><code>shufflePositions</code>: Randomly redistribute all players' board positions.</li> <li><code>moveAnywhereRandom</code>: Teleport the drawing player to a random square.</li> <li><code>moveOpponentAnywhereRandom</code>: Teleport a random opponent to a random square.</li> <li><code>shareMoneyAll</code>: The drawing player gives an amount of money to each player.</li> <li><code>freeHouse</code>: Grant the drawing player a free house on one of their properties.</li> <li><code>goToJail</code>: Send the drawing player directly to jail.</li> <li><code>sendToJail</code>: Send a randomly chosen opponent to jail.</li> <li><code>everybodyToJail</code>: Send all players to jail simultaneously.</li> <li><code>doubleOrNothing</code>: Double the player's money or reduce it to zero (50/50 chance).</li> <li><code>getParkingMoney</code>: Award the player the current <code>ParkingSquare</code> jackpot.</li> <li><code>reviveProperty</code>: Lift the mortgage on one of the player's mortgaged properties.</li> <li><code>earthquake</code>: Destroy one house from each square.</li> <li><code>everybodySendsYouMoney</code>: Every other player pays the drawing player <code>value</code>.</li> <li><code>magnetism</code>: Move all the players to the drawing player position.</li> <li><code>goToStart</code>: Move the drawing player back to the <code>ExitSquare</code> (GO).</li> </ul> |
| <code>fantasy_type</code> | <code>CharField</code>                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>value</code>        | <code>IntegerField</code>   <code>None</code> | Numeric parameter for the event, e.g. a fixed money amount. May be null for events that do not require a value.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>card_cost</code>    | <code>IntegerField</code>                     | The points cost to purchase this card in advance (for pre-bought fantasy card mechanics).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

#### • Source code in `magnate/models.py`

```
class FantasyEvent(models.Model):
 """
 A Chance/Community-Chest style event card drawn when a player lands on a ``FantasySquare``.
```

Each ``FantasyEvent`` encodes a specific effect type and an optional numeric value that parameterises the effect (e.g. the amount of money won or lost).

Attributes:

`fantasy_type` (CharField): The category of effect to apply. One of the ``FantasyType`` choices:

- ``winPlainMoney`` / ``losePlainMoney``: Add or subtract a fixed ``value`` from the player's balance.
- ``winRatioMoney`` / ``loseRatioMoney``: Add or subtract a ratio of the player's current balance.
- ``breakOpponentHouse``: Destroy one house on a randomly chosen opponent's property.
- ``breakOwnHouse``: Destroy one house on the drawing player's own property.
- ``shufflePositions``: Randomly redistribute all players' board positions.
- ``moveAnywhereRandom``: Teleport the drawing player to a random square.
- ``moveOpponentAnywhereRandom``: Teleport a random opponent to a random square.
- ``shareMoneyAll``: The drawing player gives an amount of money to each player.
- ``freeHouse``: Grant the drawing player a free house on one of their properties.
- ``goToJail``: Send the drawing player directly to jail.
- ``sendToJail``: Send a randomly chosen opponent to jail.
- ``everybodyToJail``: Send all players to jail simultaneously.
- ``doubleOrNothing``: Double the player's money or reduce it to zero (50/50 chance).
- ``getParkingMoney``: Award the player the current ``ParkingSquare`` jackpot.
- ``reviveProperty``: Lift the mortgage on one of the player's mortgaged properties.
- ``earthquake``: Destroy one house from each square.
- ``everybodySendsYouMoney``: Every other player pays the drawing player ``value``.
- ``magnetism``: Move all the players to the drawing player position.
- ``goToStart``: Move the drawing player back to the ``ExitSquare`` (G0).

`value` (IntegerField | None): Numeric parameter for the event, e.g. a fixed money amount. May be null for events that do not require a value.

`card_cost` (IntegerField): The points cost to purchase this card in advance (for pre-bought fantasy card mechanics).

"""

```
class FantasyType(models.TextChoices):
 winPlainMoney = 'winPlainMoney',
 winRatioMoney = 'winRatioMoney',
 losePlainMoney = 'losePlainMoney',
 loseRatioMoney = 'loseRatioMoney',
 breakOpponentHouse = 'breakOpponentHouse',
 breakOwnHouse = 'breakOwnHouse',
 shufflePositions = 'shufflePositions',
 moveAnywhereRandom = 'moveAnywhereRandom',
 moveOpponentAnywhereRandom = 'moveOpponentAnywhereRandom',
 shareMoneyAll = 'shareMoneyAll',
 freeHouse = 'freeHouse',
 goToJail = 'goToJail',
 sendToJail = 'sendToJail',
 everybodyToJail = 'everybodyToJail',
 doubleOrNothing = 'doubleOrNothing',
 getParkingMoney = 'getParkingMoney',
 reviveProperty = 'reviveProperty',
 earthquake = 'earthquake',
 everybodySendsYouMoney = 'everybodySendsYouMoney',
 magnetism = 'magnetism',
 goToStart = 'goToStart'

fantasy_type = models.CharField(choices=FantasyType, max_length=40)
value = models.IntegerField(default=0, null=True)
card_cost = models.IntegerField(default=0)
```

Generate a random FantasyEvent with a rolled type, card cost, and optional value.

The fantasy type is chosen uniformly at random from all values defined in `FantasyEvent.FantasyType`. Depending on the type, a `card_cost` (what the player pays to buy the card) and an optional `value` (monetary amount or multiplier used when the event is applied) are assigned.

Cost / value summary by type:

| Type                       | Cost | Value options         |
|----------------------------|------|-----------------------|
| winPlainMoney              | 130  | 20, 60, 120, 150, 200 |
| winRatioMoney              | 200  | 5, 10, 25, 50 (%)     |
| losePlainMoney             | 80   | 40, 80, 120, 150, 200 |
| loseRatioMoney             | 30   | 5, 10, 25, 50 (%)     |
| shareMoneyAll              | 5    | 20, 30, 50            |
| everybodySendsYouMoney     | 120  | 20, 30, 50            |
| doubleOrNothing            | 50   | doubled (bool)        |
| getParkingMoney            | 500  | money                 |
| goToJail                   | 25   | —                     |
| sendToJail                 | 80   | —                     |
| everybodyToJail            | 50   | —                     |
| shufflePositions           | 50   | —                     |
| moveAnywhereRandom         | 50   | —                     |
| moveOpponentAnywhereRandom | 60   | —                     |
| magnetism                  | 100  | —                     |
| goToStart                  | 90   | —                     |
| breakOpponentHouse         | 150  | —                     |
| breakOwnHouse              | 30   | —                     |
| freeHouse                  | 80   | —                     |
| reviveProperty             | 100  | revived property      |
| earthquake                 | 200  | {affected squares}    |

#### Returns:

**Name**      **Type**      **Description**

`FantasyEvent`   [FantasyEvent](#)   A new, unsaved `FantasyEvent` instance with `fantasy_type`, `card_cost`, and `value` (`None` when not applicable) populated.

#### • Source code in `magnate/fantasy.py`

```
@staticmethod
def generate() -> FantasyEvent:
 """
 Generate a random FantasyEvent with a rolled type, card cost, and optional value.

 The fantasy type is chosen uniformly at random from all values defined in
 ``FantasyEvent.FantasyType``. Depending on the type, a ``card_cost`` (what the
 player pays to buy the card) and an optional ``value`` (monetary amount or
 multiplier used when the event is applied) are assigned.

 Cost / value summary by type:
```

| Type                       | Cost | Value options         |
|----------------------------|------|-----------------------|
| -----                      | ---- | -----                 |
| winPlainMoney              | 130  | 20, 60, 120, 150, 200 |
| winRatioMoney              | 200  | 5, 10, 25, 50 (%)     |
| losePlainMoney             | 80   | 40, 80, 120, 150, 200 |
| loseRatioMoney             | 30   | 5, 10, 25, 50 (%)     |
| shareMoneyAll              | 5    | 20, 30, 50            |
| everybodySendsYouMoney     | 120  | 20, 30, 50            |
| doubleOrNothing            | 50   | doubled (bool)        |
| getParkingMoney            | 500  | money                 |
| goToJail                   | 25   | –                     |
| sendToJail                 | 80   | –                     |
| everybodyToJail            | 50   | –                     |
| shufflePositions           | 50   | –                     |
| moveAnywhereRandom         | 50   | –                     |
| moveOpponentAnywhereRandom | 60   | –                     |
| magnetism                  | 100  | –                     |
| goToStart                  | 90   | –                     |
| breakOpponentHouse         | 150  | –                     |
| breakOwnHouse              | 30   | –                     |
| freeHouse                  | 80   | –                     |
| reviveProperty             | 100  | revived property      |
| earthquake                 | 200  | {affected squares}    |

Args:

None

Returns:

FantasyEvent: A new, unsaved ``FantasyEvent`` instance with ``fantasy\_type``, ``card\_cost``, and ``value`` (``None`` when not applicable) populated.

"""

```

fantasy_type = random.choice(FantasyEvent.FantasyType.values)
if fantasy_type == 'sendToJail':
 fantasy_type = random.choice(['winPlainMoney', 'losePlainMoney'])
elif fantasy_type == 'everybodyToJail':
 fantasy_type = random.choice(['winPlainMoney', 'losePlainMoney'])
elif fantasy_type == 'goToJail':
 fantasy_type = random.choice(['winPlainMoney', 'losePlainMoney'])
elif fantasy_type == 'freeHouse':
 fantasy_type = random.choice(['doubleOrNothing', 'getParkingMoney'])
value = None
card_cost = None

if fantasy_type == 'winPlainMoney':
 card_cost = 130
 rand = random.randrange(5)
 if(rand == 0):
 value = 20
 elif(rand == 1):
 value = 60
 elif(rand == 2):
 value = 120
 elif(rand == 3):
 value = 150
 elif(rand == 4):
 value = 200

```

```

elif fantasy_type == 'winRatioMoney':
 card_cost = 200
 rand = random.randrange(4)
 if(rand == 0):
 value = 5
 elif(rand == 1):
 value = 10
 elif(rand == 2):
 value = 25
 elif(rand == 3):
 value = 50

elif fantasy_type == 'losePlainMoney':
 card_cost = 80
 rand = random.randrange(5)
 if(rand == 0):
 value = 40
 elif(rand == 1):
 value = 80
 elif(rand == 2):
 value = 120
 elif(rand == 3):
 value = 150
 elif(rand == 4):
 value = 200

elif fantasy_type == 'loseRatioMoney':
 card_cost = 30
 rand = random.randrange(4)
 if(rand == 0):
 value = 5
 elif(rand == 1):
 value = 10
 elif(rand == 2):
 value = 25
 elif(rand == 3):
 value = 50

elif fantasy_type == 'breakOpponentHouse':
 card_cost = 150

elif fantasy_type == 'breakOwnHouse':
 card_cost = 30

elif fantasy_type == 'shufflePositions':
 card_cost = 50

elif fantasy_type == 'moveAnywhereRandom':
 card_cost = 50

elif fantasy_type == 'moveOpponentAnywhereRandom':
 card_cost = 60

elif fantasy_type == 'shareMoneyAll':
 card_cost = 5
 rand = random.randrange(3)
 if(rand == 0):

```

```

 value = 20
 elif(rand == 1):
 value = 30
 elif(rand == 2):
 value = 50

elif fantasy_type == 'freeHouse':
 card_cost = 80

elif fantasy_type == 'goToJail':
 card_cost = 25

elif fantasy_type == 'sendToJail':
 card_cost = 80

elif fantasy_type == 'everybodyToJail':
 card_cost = 50

elif fantasy_type == 'doubleOrNothing':
 card_cost = 50

elif fantasy_type == 'getParkingMoney':
 card_cost = COST_PARKING_MONEY

elif fantasy_type == 'reviveProperty':
 card_cost = 100

elif fantasy_type == 'earthquake':
 card_cost = 200

elif fantasy_type == 'everybodySendsYouMoney':
 card_cost = 120
 rand = random.randrange(3)
 if(rand == 0):
 value = 20
 elif(rand == 1):
 value = 30
 elif(rand == 2):
 value = 50

elif fantasy_type == 'magnetism':
 card_cost = 100

elif fantasy_type == 'goToStart':
 card_cost = 90

return FantasyEvent(
 fantasy_type = fantasy_type,
 value = value,
 card_cost = card_cost
)
```

## 2.6.2 Fantasy results

Apply a FantasyEvent to the current game state and return the outcome.

Executes the side-effects of `fantasy_event` on `game` — mutating money, positions, jail state, and/or property house counts — then saves all affected model instances and increments the triggering player's `num_fantasy_events` statistic.

The `result` field of the returned `FantasyResult` varies by event type:

**Always** `None` : `winPlainMoney` , `winRatioMoney` , `losePlainMoney` , `loseRatioMoney` , `shareMoneyAll` , `everybodySendsYouMoney` , `getParkingMoney` , `goToJail` , `everybodyToJail` , `shufflePositions` , `moveAnywhereRandom` , `magnetism` , `goToStart`

Also ``None`` for property events when no valid target exists at the time of the call (no houses, no mortgaged properties, etc.).

**Single targeted square** — `{"square": custom_id} : breakOpponentHouse , breakOwnHouse , freeHouse , reviveProperty`

**Multiple targeted squares** — `{"squares": [custom_id, ...]} : earthquake` — one entry per property that had a house removed.

**Targeted player** — `{"target_player": pk} : moveOpponentAnywhereRandom , sendToJail`

**Outcome flag** — `{"doubled": bool} : doubleOrNothing` — `True` if the player's money was doubled, `False` if it was zeroed.

House-targeting logic for break/build events follows Monopoly-standard even-build rules: only properties in the group with the current maximum (for demolition) or minimum (for construction) house count are eligible, ensuring the house count within a colour group remains as even as possible.

#### Parameters:

| Name                       | Type                      | Description                                                                                                                                                                              | Default         |
|----------------------------|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| <code>game</code>          | <code>Game</code>         | The active game instance. <code>game.money</code> , <code>game.positions</code> , <code>game.jail_remaining_turns</code> , and <code>game.parking_money</code> may be mutated and saved. | <i>required</i> |
| <code>user</code>          | <code>CustomUser</code>   | The player who triggered the event. Used as the default actor for all money and property operations; some events target a randomly selected opponent instead.                            | <i>required</i> |
| <code>fantasy_event</code> | <code>FantasyEvent</code> | The event to apply. Must have a valid <code>fantasy_type</code> ; <code>value</code> must not be <code>None</code> for event types that require it (plain/ratio money, share events).    | <i>required</i> |

#### Returns:

| Name                       | Type                       | Description                                                                                                                                        |
|----------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>FantasyResult</code> | <code>FantasyResult</code> | An unsaved result object linking the original <code>fantasy_event</code> to a <code>result</code> dict (or <code>None</code> ) as described above. |



**Raises:**

| Type                        | Description                                                                                                                              |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>GameLogicError</code> | If <code>fantasy_event.fantasy_type</code> is not a recognised value.                                                                    |
| Exception                   | If a money-based event is called with <code>fantasy_event.value</code> is <code>None</code> .                                            |
| Exception                   | If internal targeting invariants are violated (e.g. no valid candidate found after filtering — indicates a logic bug, not a user error). |

• Source code in `magnate/fantasy.py`

```
def apply_fantasy_event(game: Game, user: CustomUser, fantasy_event: FantasyEvent) -> FantasyResult:
 """
```

Apply a FantasyEvent to the current game state and return the outcome.

Executes the side-effects of `fantasy_event` on `game` — mutating money, positions, jail state, and/or property house counts — then saves all affected model instances and increments the triggering player's `num_fantasy_events` statistic.

The `result` field of the returned `FantasyResult` varies by event type:

**\*\*Always\*\*** `None`:

```
 winPlainMoney, winRatioMoney, losePlainMoney, loseRatioMoney,
 shareMoneyAll, everybodySendsYouMoney, getParkingMoney,
 goToJail, everybodyToJail, shufflePositions,
 moveAnywhereRandom, magnetism, goToStart
```

Also `None` for property events when no valid target exists at the time of the call (no houses, no mortgaged properties, etc.).

**\*\*Single targeted square\*\*** — `{"square": custom_id}`:

```
 breakOpponentHouse, breakOwnHouse, freeHouse, reviveProperty
```

**\*\*Multiple targeted squares\*\*** — `{"squares": [custom_id, ...]}`:

```
 earthquake — one entry per property that had a house removed.
```

**\*\*Targeted player\*\*** — `{"target_player": pk}`:

```
 moveOpponentAnywhereRandom, sendToJail
```

**\*\*Outcome flag\*\*** — `{"doubled": bool}`:

```
 doubleOrNothing — True if the player's money was doubled,
 False if it was zeroed.
```

House-targeting logic for break/build events follows Monopoly-standard even-build rules: only properties in the group with the current maximum (for demolition) or minimum (for construction) house count are eligible, ensuring the house count within a colour group remains as even as possible.

**Args:**

`game` (`Game`): The active game instance. `game.money`, `game.positions`, `game.jail_remaining_turns`, and `game.parking_money` may be mutated and saved.

`user` (`CustomUser`): The player who triggered the event. Used as the default actor for all money and property operations; some events target a randomly selected opponent instead.

`fantasy_event` (`FantasyEvent`): The event to apply. Must have a valid `fantasy_type`; `value` must not be `None` for event types that require it (plain/ratio money, share events).

**Returns:**

FantasyResult: An unsaved result object linking the original ``fantasy\_event`` to a ``result`` dict (or ``None``) as described above.

**Raises:**

GameLogicError: If ``fantasy\_event.fantasy\_type`` is not a recognised value.  
 Exception: If a money-based event is called with ``fantasy\_event.value is None``.  
 Exception: If internal targeting invariants are violated (e.g. no valid candidate found after filtering – indicates a logic bug, not a user error).

```
"""
```

```
stats = PlayerGameStatistic.objects.get(user=user, game=game)
stats.num_fantasy_events += 1
stats.save()
```

```
if fantasy_event.fantasy_type == 'winPlainMoney':
 if fantasy_event.value is None:
 raise Exception('FantasyEvent value is None')
```

```
 money_to_add = fantasy_event.value
 game.money[str(user.pk)] += money_to_add
 game.save()
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.won_money += money_to_add
 stats.save()
```

```
 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)
```

```
elif fantasy_event.fantasy_type == 'winRatioMoney':
 if fantasy_event.value is None:
 raise Exception('FantasyEvent value is None')
```

```
 ratio_to_add = fantasy_event.value
 previous_money = game.money[str(user.pk)]
 game.money[str(user.pk)] = int(game.money[str(user.pk)] * (1 + ratio_to_add/100))
 game.save()
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.won_money += game.money[str(user.pk)] - previous_money
 stats.save()
```

```
 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)
```

```
elif fantasy_event.fantasy_type == 'losePlainMoney':
 if fantasy_event.value is None:
 raise Exception('FantasyEvent value is None')
```

```
 money_to_sub = fantasy_event.value
 game.money[str(user.pk)] -= money_to_sub
 game.parking_money += money_to_sub
 game.save()
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.lost_money += money_to_sub
 stats.save()
```

```

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)

elif fantasy_event.fantasy_type == 'loseRatioMoney':
 if fantasy_event.value is None:
 raise Exception('FantasyEvent value is None')

 ratio_to_sub = fantasy_event.value
 previous_money = game.money[str(user.pk)]
 end_money = int(game.money[str(user.pk)] * (1 - ratio_to_sub/100))
 game.parking_money += game.money[str(user.pk)] - end_money
 game.money[str(user.pk)] = end_money
 game.save()
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.lost_money += previous_money - game.money[str(user.pk)]
 stats.save()

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)

elif fantasy_event.fantasy_type == 'breakOpponentHouse':
 #random opponent, random house
 properties = PropertyRelationship.objects.filter(
 game=game,
 houses__gte=1, # at least 1 house
 square__propertysquare__isnull=False # look for properties
 # that support houses
).exclude(
 owner=user
).select_related('square__propertysquare', 'owner')

 if not properties.exists():
 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)

 groups = defaultdict(list)
 for prop in properties:
 key = (prop.owner, prop.square.get_real_instance().group)
 groups[key].append(prop)

 valid_candidates = []
 for (owner, color_group), props_in_group in groups.items():
 max_houses = max(prop.houses for prop in props_in_group)
 if max_houses > 0:
 for prop in props_in_group:
 if prop.houses == max_houses:
 valid_candidates.append(prop)

 if valid_candidates:
 target_prop : PropertyRelationship = random.choice(valid_candidates)
 else:
 raise Exception("esto no deberia pasar")

```

```

result_property = _demolish_square(game=game, user=target_prop.owner, demolition_square=target_prop.square,
 number_demolished=1, free_demolish=True)

return FantasyResult(
 fantasy_event = fantasy_event,
 result = {'square': result_property.square.custom_id}
)

elif fantasy_event.fantasy_type == 'breakOwnHouse':
 #fantasy maker user, random house
 properties = PropertyRelationship.objects.filter(
 game=game,
 houses__gte=1, # at least 1 house
 owner=user,
 square__property__square__isnull=False
).select_related('square')

 if not properties.exists():
 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None # empty, no house broken
)

 groups = defaultdict(list)
 for prop in properties:
 key = prop.square.get_real_instance().group
 groups[key].append(prop)

 valid_candidates = []
 for group, props_in_group in groups.items():
 max_houses = max(prop.houses for prop in props_in_group)
 if max_houses > 0:
 for prop in props_in_group:
 if prop.houses == max_houses:
 valid_candidates.append(prop)

 if valid_candidates:
 target_prop : PropertyRelationship = random.choice(valid_candidates)
 else:
 # FIXME
 raise Exception("esto no deberia pasar")

 result_property = _demolish_square(game=game, user=target_prop.owner, demolition_square=target_prop.square,
 number_demolished=1, free_demolish=True)

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = {'square': result_property.square.custom_id}
)

elif fantasy_event.fantasy_type == 'shufflePositions':
 id_jail = _get_jail_square().custom_id
 #move everybody to random square
 ids = list(BaseSquare.objects.values_list('custom_id', flat=True))
 ids.remove(id_jail)
 for player in game.players.all():
 if game.positions[str(player.pk)] != id_jail:
 rand_square_id = random.choice([n for n in ids if n != game.positions[str(player.pk)]])

```

```

 game.positions[str(player.pk)] = int(rand_square_id)

 game.save()

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)

elif fantasy_event.fantasy_type == 'moveAnywhereRandom':
 id_jail = _get_jail_square().custom_id
 if(game.positions[str(user.pk)] != id_jail):
 ids = list(BaseSquare.objects.values_list('custom_id', flat=True))
 ids.remove(id_jail)
 rand_square_id = random.choice([n for n in ids if n != game.positions[str(user.pk)]])
 game.positions[str(user.pk)] = int(rand_square_id)
 game.save()

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)

elif fantasy_event.fantasy_type == 'moveOpponentAnywhereRandom':
 opponents = game.players.exclude(pk=user.pk)
 if not opponents.exists():
 raise Exception('Catastrofe no deberias estar aqui')
 target_player = random.choice(list(opponents)) #con un order by ? tambien rularia

 id_jail = _get_jail_square().custom_id
 if(game.positions[str(target_player.pk)] != id_jail):
 ids = list(BaseSquare.objects.values_list('custom_id', flat=True))
 ids.remove(id_jail)
 rand_square_id = random.choice([n for n in ids if n != game.positions[str(target_player.pk)]])
 game.positions[str(target_player.pk)] = int(rand_square_id)
 game.save()

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = {'target_player':target_player.pk}
)

elif fantasy_event.fantasy_type == 'shareMoneyAll':
 if fantasy_event.value is None:
 raise Exception('FantasyEvent value is None')

 money_to_share = fantasy_event.value

 opponents = game.players.exclude(pk=user.pk)
 opponents_list = list(opponents)
 opponents_count = len(opponents_list)

 for player in opponents_list:
 game.money[str(player.pk)] += money_to_share
 game.save()
 stats = PlayerGameStatistic.objects.get(user=player, game=game)
 stats.won_money += money_to_share
 stats.save()

```

```

game.money[str(user.pk)] -= money_to_share*opponents_count
stats = PlayerGameStatistic.objects.get(user=user, game=game)
stats.lost_money += money_to_share*opponents_count
stats.save()

game.save()

return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)

elif fantasy_event.fantasy_type == 'freeHouse':
 properties = PropertyRelationship.objects.filter(
 game=game,
 houses__lt=5,
 houses__gte=0,
 owner=user,
).select_related('square')

 if not properties.exists():
 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)

 groups = defaultdict(list)
 for prop in properties:
 key = prop.square.get_real_instance().group
 groups[key].append(prop)

 valid_candidates = []
 for color_group, props_in_group in groups.items():
 min_houses = min(prop.houses for prop in props_in_group)
 for prop in props_in_group:
 if prop.houses == min_houses:
 valid_candidates.append(prop)

 if valid_candidates:
 target_prop = random.choice(valid_candidates)
 else:
 raise Exception('creo que esto no puede ocurrir')

 result_property = _build_square(game=game, user=target_prop.owner, building_square=target_prop.square,
 number_built=1, free_build=True)

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = {'square': result_property.square.custom_id}
)

elif fantasy_event.fantasy_type == 'goToJail':
 _send_to_jail(game, user)
 game.save()

 return FantasyResult(
 fantasy_event=fantasy_event,
 result=None
)

```

```

)

elif fantasy_event.fantasy_type == 'sendToJail':
 target_player = random.choice(list(game.players.exclude(pk=user.pk)))
 _send_to_jail(game, target_player)
 game.save()

 return FantasyResult(
 fantasy_event=fantasy_event,
 result={'target_player': target_player.pk}
)

elif fantasy_event.fantasy_type == 'everybodyToJail':
 for player in game.players.all():
 _send_to_jail(game, player)
 game.save()

 return FantasyResult(
 fantasy_event=fantasy_event,
 result=None
)

elif fantasy_event.fantasy_type == 'doubleOrNothing':
 r = random.choice([True, False])
 if r:
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.won_money += game.money[str(user.pk)]
 stats.save()
 game.money[str(user.pk)] *= 2
 game.save()
 else:
 previous_money = game.money[str(user.pk)]
 game.parking_money += previous_money
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.lost_money += previous_money
 stats.save()
 game.money[str(user.pk)] = 0
 game.save()

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = {'doubled': r}
)

elif fantasy_event.fantasy_type == 'getParkingMoney':
 parking_money = game.parking_money - COST_PARKING_MONEY
 parking_money = max(parking_money, 0)
 game.money[str(user.pk)] += parking_money
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.won_money += parking_money
 stats.save()
 game.parking_money = COST_PARKING_MONEY
 game.save()

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = parking_money
)

```

```

)

elif fantasy_event.fantasy_type == 'reviveProperty':
 properties = PropertyRelationship.objects.filter(
 game=game, owner=user, mortgage=True
)

 if not properties.exists():
 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)

 target = random.choice(properties)

 result = _unset_mortgage(user=user, game=game, target_square=target.square, free_unset_mortgage=True)

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = {'square': result.square.custom_id}
)

elif fantasy_event.fantasy_type == 'earthquake':
 properties = PropertyRelationship.objects.filter(
 game=game,
 houses__gte=1,
).select_related('square')

 if not properties.exists():
 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)

 demolished_houses = []

 for prop in properties:
 result_property = _demolish_square(game=game, user=prop.owner, demolition_square=prop.square,
 number_demolished=1, free_demolish=True)
 demolished_houses.append(result_property.square.custom_id)

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = {'squares': demolished_houses}
)

elif fantasy_event.fantasy_type == 'everybodySendsYouMoney':
 if fantasy_event.value is None:
 raise Exception('FantasyEvent value is None')

 money_to_share = fantasy_event.value

 opponents = game.players.exclude(pk=user.pk)
 opponents_list = list(opponents)
 opponents_count = len(opponents_list)

 for player in opponents_list:
 game.money[str(player.pk)] -= money_to_share

```



```

 game.save()
 stats = PlayerGameStatistic.objects.get(user=player, game=game)
 stats.lost_money += money_to_share
 stats.save()

 game.money[str(user.pk)] += money_to_share*opponents_count
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.won_money += money_to_share*opponents_count
 stats.save()

 game.save()

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)

elif fantasy_event.fantasy_type == 'magnetism':
 id_jail = _get_jail_square().custom_id
 target_id = game.positions[str(user.pk)]
 for player in game.players.all(): #no caso especial para el que lanza, se moverá al mismo sitio
 if(game.positions[str(player.pk)] != id_jail):
 game.positions[str(player.pk)] = int(target_id)

 game.save()

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)

elif fantasy_event.fantasy_type == 'goToStart':
 id_jail = _get_jail_square().custom_id
 start_square = ExitSquare.objects.first()
 if start_square is None:
 raise Exception('No encuentra casilla de salida')

 if(game.positions[str(user.pk)] != id_jail):
 game.positions[str(user.pk)] = int(start_square.custom_id)
 game.money[str(user.pk)] += start_square.init_money
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.won_money += start_square.init_money
 stats.save()
 game.save()

 return FantasyResult(
 fantasy_event = fantasy_event,
 result = None
)

else:
 raise GameLogicError(f"undefined fantasy event: {fantasy_event.fantasy_type}")

```

## 2.7 Cheats

Dispatches debug cheat commands. Only active when Django DEBUG=True.

- **Expected message shape**

```
{ "type": "Cheat", "cheat": "", ...cheat-specific fields }
```

Supported cheats

MockDice Force the next dice roll to specific values. { "cheat": "MockDice", "dice1": 3, "dice2": 4, "dice\_bus": 5 }

Teleport Move any player to any square immediately. { "cheat": "Teleport", "player\_id": 7, "square\_id": 12 }

SetMoney Set a player's money to an exact amount (can be negative for testing liquidation). { "cheat": "SetMoney", "player\_id": 7, "amount": 5000 }

CreateProperty Create a PropertyRelationship — give a player ownership of any square. { "cheat": "CreateProperty", "player\_id": 7, "square\_id": 15, "houses": 0, "mortgage": false }

DeleteProperty Delete a PropertyRelationship — strip ownership of a square from whoever holds it. { "cheat": "DeleteProperty", "square\_id": 15 }

- **Source code in** `magnate/cheats.py`

```
async def handle_cheat(game: Game, data: dict) -> None:
 """
 Dispatches debug cheat commands. Only active when Django DEBUG=True.

 Expected message shape:
 { "type": "Cheat", "cheat": "<cheat_name>", ...cheat-specific fields }

 Supported cheats

 MockDice
 Force the next dice roll to specific values.
 { "cheat": "MockDice", "dice1": 3, "dice2": 4, "dice_bus": 5 }

 Teleport
 Move any player to any square immediately.
 { "cheat": "Teleport", "player_id": 7, "square_id": 12 }

 SetMoney
 Set a player's money to an exact amount (can be negative for testing liquidation).
 { "cheat": "SetMoney", "player_id": 7, "amount": 5000 }

 CreateProperty
 Create a PropertyRelationship — give a player ownership of any square.
 { "cheat": "CreateProperty", "player_id": 7, "square_id": 15, "houses": 0, "mortgage": false }

 DeleteProperty
 Delete a PropertyRelationship — strip ownership of a square from whoever holds it.
 { "cheat": "DeleteProperty", "square_id": 15 }
```

```
""
response_data = await _apply_cheat(game, data)
```

# 3. Technical reference

## 3.1 Core Game Logic (games.py)

This module encapsulates the rules, state transitions, and actions of the game. It functions as the authoritative backend engine, ensuring that all moves are valid according to the game state.

### 3.1.1 🎮 Game State Machine Flow

The game operates as a finite state machine where players transition through specific phases. Any action sent by the frontend must be authorized by the `GameManager` based on the current active phase.

### 3.1.2 🛠️ Utility & Data Functions

These functions handle the database-level interactions required to resolve game logic.

#### 3.1.3 `_get_square_by_custom_id(custom_id)`

Retrieves a `BaseSquare` instance by its designated `custom_id`.

#### Parameters:

| Name                   | Type             | Description                                          | Default |
|------------------------|------------------|------------------------------------------------------|---------|
| <code>custom_id</code> | <code>int</code> | The custom identifier of the square. <i>required</i> |         |

#### Returns:

| Name                    | Type                    | Description                        |
|-------------------------|-------------------------|------------------------------------|
| <code>BaseSquare</code> | <code>BaseSquare</code> | The corresponding square instance. |

#### Raises:

| Type                        | Description                                                                |
|-----------------------------|----------------------------------------------------------------------------|
| <code>GameLogicError</code> | If no square with the given <code>custom_id</code> exists in the database. |

#### • Source code in `magnate/game_utils.py`

```
def _get_square_by_custom_id(custom_id: int) -> BaseSquare:
 """
 Retrieves a BaseSquare instance by its designated custom_id.

 Args:
 custom_id (int): The custom identifier of the square.

 Returns:
 BaseSquare: The corresponding square instance.

 Raises:
 GameLogicError: If no square with the given custom_id exists in the database.
 """
 square = BaseSquare.objects.filter(custom_id=custom_id).first()
 if square is None:
 raise GameLogicError(f"no square with id {custom_id}")
```

```
return square
```

3.1.4 `_get_user_square(game, user)`

Retrieves the square where a specific user is currently located.

Parameters:

| Name | Type                    | Description                               | Default         |
|------|-------------------------|-------------------------------------------|-----------------|
| game | <code>Game</code>       | The current game instance.                | <i>required</i> |
| user | <code>CustomUser</code> | The user whose position is being queried. | <i>required</i> |

Returns:

| Name       | Type                    | Description                                      |
|------------|-------------------------|--------------------------------------------------|
| BaseSquare | <code>BaseSquare</code> | The square where the user is currently standing. |

Raises:

| Type                        | Description                          |
|-----------------------------|--------------------------------------|
| <code>GameLogicError</code> | If the user is not part of the game. |

• Source code in `magnate/game_utils.py`

```
def _get_user_square(game: Game, user: CustomUser) -> BaseSquare:
 """
 Retrieves the square where a specific user is currently located.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user whose position is being queried.

 Returns:
 BaseSquare: The square where the user is currently standing.

 Raises:
 GameLogicError: If the user is not part of the game.
 """
 user_key = str(user.pk)

 if user_key not in game.positions:
 raise GameLogicError(f"user {user} not in the game")

 return _get_square_by_custom_id(game.positions[user_key])
```

3.1.5 `_get_relationship(game, square)`

Retrieves the ownership relationship for a specific square in a game.

Parameters:

| Name   | Type       | Description                | Default  |
|--------|------------|----------------------------|----------|
| game   | Game       | The current game instance. | required |
| square | BaseSquare | The square instance.       | required |

Returns:

| Type                           | Description                                                                      |
|--------------------------------|----------------------------------------------------------------------------------|
| Optional[PropertyRelationship] | PropertyRelationship   None: The relationship instance if owned, otherwise None. |

Raises:

| Type           | Description                                          |
|----------------|------------------------------------------------------|
| GameLogicError | If more than one owner is found for the same square. |

• Source code in `magnate/game_utils.py`

```
def _get_relationship(game: Game, square: BaseSquare) -> Optional[PropertyRelationship]:
 """
 Retrieves the ownership relationship for a specific square in a game.

 Args:
 game (Game): The current game instance.
 square (BaseSquare): The square instance.

 Returns:
 PropertyRelationship | None: The relationship instance if owned, otherwise None.

 Raises:
 GameLogicError: If more than one owner is found for the same square.
 """
 try:
 return PropertyRelationship.objects.get(game=game, square=square)
 except PropertyRelationship.DoesNotExist:
 return None
 except MultipleObjectsReturned:
 raise GameLogicError("more than one owners for the same square")
```

3.1.6 `_get_jail_square()`

Retrieves the designated Jail square for the board.

Returns:

| Name       | Type       | Description              |
|------------|------------|--------------------------|
| BaseSquare | BaseSquare | The JailSquare instance. |

Raises:

| Type            | Description                                                                       |
|-----------------|-----------------------------------------------------------------------------------|
| GameDesignError | If there are no jail squares or multiple jail squares configured in the database. |

• **Source code in** `magnate/game_utils.py`

```
def _get_jail_square() -> BaseSquare:
 """
 Retrieves the designated Jail square for the board.

 Args:
 None

 Returns:
 BaseSquare: The JailSquare instance.

 Raises:
 GameDesignError: If there are no jail squares or multiple jail squares configured in the database.
 """
 try:
 return JailSquare.objects.get()
 except JailSquare.DoesNotExist:
 raise GameDesignError("there are no jail squares in the game")
 except MultipleObjectsReturned:
 raise GameDesignError("there are too many jail squares")
```

### 3.1.7 `_calculate_net_worth(game, user)`

Calculates the total net worth of a user. This includes liquid cash, the full buy price of all unmortgaged properties, and the full build price of all constructed houses/hotels.

**Parameters:**

| Name              | Type                    | Description                              | Default         |
|-------------------|-------------------------|------------------------------------------|-----------------|
| <code>game</code> | <code>Game</code>       | The current game instance.               | <i>required</i> |
| <code>user</code> | <code>CustomUser</code> | The user to calculate the net worth for. | <i>required</i> |

**Returns:**

**Name Type Description**

|                  |                  |                                 |
|------------------|------------------|---------------------------------|
| <code>int</code> | <code>int</code> | The total calculated net worth. |
|------------------|------------------|---------------------------------|

• **Source code in** `magnate/game_utils.py`

```
def _calculate_net_worth(game: Game, user: CustomUser) -> int:
 """
 Calculates the total net worth of a user.
 This includes liquid cash, the full buy price of all unmortgaged properties,
 and the full build price of all constructed houses/hotels.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user to calculate the net worth for.

 Returns:
 int: The total calculated net worth.
 """
 total_value = game.money.get(str(user.pk), 0)

 properties = PropertyRelationship.objects.filter(game=game, owner=user).select_related('square')
```

```

for rel in properties:
 square = rel.square.get_real_instance()

 # value of properties
 if not rel.mortgage and hasattr(square, 'buy_price'):
 total_value += square.buy_price

 # value of constructions
 if hasattr(square, 'build_price') and rel.houses > 0:
 total_value += rel.houses * square.build_price

return total_value

```

### 3.1.8 \_get\_max\_liquidation\_value(game, user)

Calculates the absolute maximum money a user can raise by liquidating all assets. This simulates selling all built houses and mortgaging all unmortgaged properties.

#### Parameters:

| Name | Type                       | Description                | Default         |
|------|----------------------------|----------------------------|-----------------|
| game | <a href="#">Game</a>       | The current game instance. | <i>required</i> |
| user | <a href="#">CustomUser</a> | The target user.           | <i>required</i> |

#### Returns:

#### Name Type Description

|     |     |                                                  |
|-----|-----|--------------------------------------------------|
| int | int | The maximum potential liquidation value in cash. |
|-----|-----|--------------------------------------------------|

#### • Source code in [magnate/game\\_utils.py](#)

```

def _get_max_liquidation_value(game: Game, user: CustomUser) -> int:
 """
 Calculates the absolute maximum money a user can raise by liquidating all assets.
 This simulates selling all built houses and mortgaging all unmortgaged properties.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The target user.

 Returns:
 int: The maximum potential liquidation value in cash.
 """
 total_value = game.money[str(user.pk)]

 properties = PropertyRelationship.objects.filter(
 game=game,
 owner=user
).select_related('square')

 for rel in properties:
 square = rel.square.get_real_instance()

 if hasattr(square, 'build_price') and rel.houses > 0:
 total_value += rel.houses * (square.build_price // 2)

```



```

 if not rel.mortgage and hasattr(square, 'buy_price'):
 total_value += square.buy_price // 2

 return total_value

```

### 3.1.9 🏠 Property & Economy Rules

Calculates financial impacts, building constraints, and mortgage status.

#### 3.1.10 `_calculate_rent_price(game, user, square)`

Calculates the exact rent price a user must pay when landing on a square. Handles different pricing logic for PropertySquares (houses and monopolies), TramSquares, BridgeSquares, and ServerSquares based on ownership multipliers.

#### Parameters:

| Name                | Type                    | Description                     | Default         |
|---------------------|-------------------------|---------------------------------|-----------------|
| <code>game</code>   | <code>Game</code>       | The current game instance.      | <i>required</i> |
| <code>user</code>   | <code>CustomUser</code> | The user landing on the square. | <i>required</i> |
| <code>square</code> | <code>BaseSquare</code> | The square being landed on.     | <i>required</i> |

#### Returns:

#### Name Type Description

|                  |                  |                                                                                                                       |
|------------------|------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>int</code> | <code>int</code> | The calculated rent amount to pay. Returns 0 if the square is unowned, mortgaged, or owned by the user landing on it. |
|------------------|------------------|-----------------------------------------------------------------------------------------------------------------------|

#### Raises:

#### Type Description

|                              |                                                                         |
|------------------------------|-------------------------------------------------------------------------|
| <code>GameDesignError</code> | If the rent arrays on the square are incorrectly configured or missing. |
| <code>GameLogicError</code>  | If an unexpected condition occurs during server square calculation.     |

#### • Source code in `magnate/game_utils.py`

```

def _calculate_rent_price(game: Game, user: CustomUser, square: BaseSquare) -> int:
 """
 Calculates the exact rent price a user must pay when landing on a square.
 Handles different pricing logic for PropertySquares (houses and monopolies),
 TramSquares, BridgeSquares, and ServerSquares based on ownership multipliers.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user landing on the square.
 square (BaseSquare): The square being landed on.

 Returns:
 int: The calculated rent amount to pay. Returns 0 if the square is unowned,
 mortgaged, or owned by the user landing on it.

 Raises:
 GameDesignError: If the rent arrays on the square are incorrectly configured or missing.

```

```

 GameLogicError: If an unexpected condition occurs during server square calculation.
"""
If it is not owned or is owned by the same user, no rent is paid
prop_rel = _get_relationship(game, square)
if not prop_rel or prop_rel.owner.pk == user.pk or prop_rel.mortgage:
 return 0

houses = prop_rel.houses

if isinstance(square, PropertySquare):
 if not square.rent_prices or len(square.rent_prices) < 6:
 raise GameDesignError(f"Incorrect rent prices for square {square.custom_id}")
 if houses == -1:
 return square.rent_prices[0]
 elif houses == 0:
 return square.rent_prices[0] * 2 # grupo completo = doble del alquiler base
 elif houses == 1:
 return square.rent_prices[1]
 elif houses == 2:
 return square.rent_prices[2]
 elif houses == 3:
 return square.rent_prices[3]
 elif houses == 4:
 return square.rent_prices[4]
 elif houses == 5: # hotel
 return square.rent_prices[5]

 return 0

elif isinstance(square, TramSquare):
 # TODO
 return 0

elif isinstance(square, BridgeSquare):
 property_owner = prop_rel.owner
 bridges_owned = PropertyRelationship.objects.filter(game=game, square__bridgesquare__isnull=False, owner=prop
 if not square.rent_prices or len(square.rent_prices) < bridges_owned:
 raise GameDesignError(f"Incorrect rent prices for bridge {square.custom_id}")
 return square.rent_prices[bridges_owned - 1]

elif isinstance(square, ServerSquare):
 property_owner = prop_rel.owner

 if not square.rent_prices or len(square.rent_prices) < 2:
 raise GameDesignError(f"Incorrect rent prices for square {square.custom_id}")

 squares = PropertyRelationship.objects.filter(game=game, square__serversquare__isnull=False, owner=property_o

 if squares.count() == 2:
 return square.rent_prices[1]
 elif squares.count() == 1:
 return square.rent_prices[0]
 else:
 # TODO: Write something
 raise GameLogicError()
else:
 return 0

```

3.1.11 `_build_square(game, user, building_square, number_built, free_build)`

Builds houses or hotels on a property. Validates that the user owns the complete color group, ensures no properties in the group are mortgaged, and respects the uniform building rule. Updates player statistics and deducts funds accordingly.

Parameters:

| Name                         | Type                    | Description                                            | Default         |
|------------------------------|-------------------------|--------------------------------------------------------|-----------------|
| <code>game</code>            | <code>Game</code>       | The current game instance.                             | <i>required</i> |
| <code>user</code>            | <code>CustomUser</code> | The user attempting to build.                          | <i>required</i> |
| <code>building_square</code> | <code>BaseSquare</code> | The target property square.                            | <i>required</i> |
| <code>number_built</code>    | <code>int</code>        | Number of buildings to construct.                      | <i>required</i> |
| <code>free_build</code>      | <code>bool</code>       | If True, the user is not charged for the construction. | <i>required</i> |

Returns:

| Name                              | Type                              | Description                                                   |
|-----------------------------------|-----------------------------------|---------------------------------------------------------------|
| <code>PropertyRelationship</code> | <code>PropertyRelationship</code> | The updated ownership relationship reflecting the new houses. |

Raises:

| Type                            | Description                                                                                                                                                                        |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>MaliciousUserInput</code> | If the target is not a property square, the user does not own the full group, a square in the group is mortgaged, it violates uniform building rules, or it exceeds the max limit. |
| <code>GameLogicError</code>     | If negative house values are encountered in the database.                                                                                                                          |

• Source code in `magnate/game_utils.py`

```
def _build_square(game: Game,
 user: CustomUser,
 building_square: BaseSquare,
 number_built: int,
 free_build: bool) -> PropertyRelationship:
 """
 Builds houses or hotels on a property.
 Validates that the user owns the complete color group, ensures no properties
 in the group are mortgaged, and respects the uniform building rule. Updates
 player statistics and deducts funds accordingly.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user attempting to build.
 building_square (BaseSquare): The target property square.
 number_built (int): Number of buildings to construct.
 free_build (bool): If True, the user is not charged for the construction.

 Returns:
 PropertyRelationship: The updated ownership relationship reflecting the new houses.

 Raises:
 MaliciousUserInput: If the target is not a property square, the user does not
 own the full group, a square in the group is mortgaged,
 it violates uniform building rules, or it exceeds the max limit.
 GameLogicError: If negative house values are encountered in the database.
 """
 # Check if it's a property and take its group
```

```

building_square = building_square.get_real_instance()

if not isinstance(building_square, PropertySquare):
 raise MaliciousUserInput(user, "tried to build in a non property square")

relationship = _get_relationship(game, building_square)

if relationship is None:
 raise MaliciousUserInput(user, "no user owns this square")

if relationship.owner.pk != user.pk:
 raise MaliciousUserInput(user, "tried to build in an unowned property")

square_group = building_square.group
actual_houses = relationship.houses

Check if user has every square in the group and if its a property
total_squares_in_group = PropertySquare.objects.filter(
 board=building_square.board,
 group=square_group
).count()

group_relationships = PropertyRelationship.objects.filter(
 game=game,
 owner=user,
 square__propertysquare__group=square_group
).select_related('square')

if group_relationships.count() != total_squares_in_group:
 raise MaliciousUserInput(user, "does not own the group")

for rel in group_relationships:
 if rel.mortgage: # none of the group should be mortgaged
 raise MaliciousUserInput(user, "cannot build while any property in the group is mortgaged")
 if rel.houses < 0:
 raise GameLogicError(f"negative house value")
 elif actual_houses + number_built - 1 > rel.houses and rel.square.get_real_instance() != building_square:
 print(number_built, actual_houses, actual_houses + number_built - 1, rel.houses, rel.square.custom_id, rel.square)
 raise MaliciousUserInput(user, "already owns more than other")

if actual_houses == 5:
 raise MaliciousUserInput(user, "nothing more to build")

relationship.houses += number_built
relationship.save()

stats = PlayerGameStatistic.objects.get(user=user, game=game)
stats.built_houses += number_built

if not free_build:
 coste = building_square.build_price * number_built

 game.money[str(user.pk)] -= coste
 game.save()
 stats.lost_money += coste

stats.save()

return relationship #ack

```

3.1.12 `_demolish_square(game, user, demolition_square, number_demolished, free_demolish)`

Demolishes houses/hotels on a property and returns half the build price to the user. Ensures that uniform building rules are respected during the demolition process and updates player statistics.

Parameters:

| Name                           | Type                    | Description                                    | Default         |
|--------------------------------|-------------------------|------------------------------------------------|-----------------|
| <code>game</code>              | <code>Game</code>       | The current game instance.                     | <i>required</i> |
| <code>user</code>              | <code>CustomUser</code> | The user attempting to demolish.               | <i>required</i> |
| <code>demolition_square</code> | <code>BaseSquare</code> | The square where buildings will be demolished. | <i>required</i> |
| <code>number_demolished</code> | <code>int</code>        | The amount of houses to demolish.              | <i>required</i> |
| <code>free_demolish</code>     | <code>bool</code>       | If True, no money is refunded to the user.     | <i>required</i> |

Returns:

| Name                              | Type                              | Description                         |
|-----------------------------------|-----------------------------------|-------------------------------------|
| <code>PropertyRelationship</code> | <code>PropertyRelationship</code> | The updated ownership relationship. |

Raises:

| Type                            | Description                                                                                                                                                   |
|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>MaliciousUserInput</code> | If the property is unowned, owned by someone else, is not a property square, attempts to demolish more houses than exist, or violates uniform building rules. |

• Source code in `magnate/game_utils.py`

```
def _demolish_square(game: Game,
 user: CustomUser,
 demolition_square: BaseSquare,
 number_demolished: int,
 free_demolish: bool) -> PropertyRelationship:
 """
 Demolishes houses/hotels on a property and returns half the build price to the user.
 Ensures that uniform building rules are respected during the demolition process
 and updates player statistics.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user attempting to demolish.
 demolition_square (BaseSquare): The square where buildings will be demolished.
 number_demolished (int): The amount of houses to demolish.
 free_demolish (bool): If True, no money is refunded to the user.

 Returns:
 PropertyRelationship: The updated ownership relationship.

 Raises:
 MaliciousUserInput: If the property is unowned, owned by someone else,
 is not a property square, attempts to demolish more houses
 than exist, or violates uniform building rules.

 """
 demolition_square = demolition_square.get_real_instance()
 if not isinstance(demolition_square, PropertySquare):
 raise MaliciousUserInput(user, "tried to demolish a non property square")
```

```

relationship = _get_relationship(game, demolition_square)

if relationship is None:
 raise MaliciousUserInput(user, "no user owns this square")

if relationship.owner.pk != user.pk:
 raise MaliciousUserInput(user, "tried to demolish an unowned property")

actual_houses = relationship.houses

if actual_houses < number_demolished:
 raise MaliciousUserInput(user, "tried to demolish more houses than are built")

square_group = demolition_square.group

group_relationships = PropertyRelationship.objects.filter(
 game=game,
 owner=user,
 square__propertysquare__group=square_group
).select_related('square')

Check if we can demolish -> respect rule
for rel in group_relationships:
 if (actual_houses - number_demolished) < (rel.houses - 1) and rel.square.get_real_instance() != demolition_square:
 raise MaliciousUserInput(user, "unable to demolish so many houses: violates the uniform building rule")

demolish
relationship.houses -= number_demolished
relationship.save()

stats = PlayerGameStatistic.objects.get(user=user, game=game)
stats.demolished_houses += number_demolished

if not free_demolish:
 coste = demolition_square.build_price

 game.money[str(user.pk)] += coste // 2 * number_demolished
 game.save()
 stats.won_money += coste // 2 * number_demolished

stats.save()

return relationship

```

### 3.1.13 `_set_mortgage(game, user, target_square, free_mortgage)`

Mortgages a property to receive immediate funds (half the buy price). Ensures that no properties within the same color group currently have houses built.

**Parameters:**

| Name          | Type                       | Description                                       | Default         |
|---------------|----------------------------|---------------------------------------------------|-----------------|
| game          | <a href="#">Game</a>       | The current game instance.                        | <i>required</i> |
| user          | <a href="#">CustomUser</a> | The user attempting the mortgage.                 | <i>required</i> |
| target_square | <a href="#">BaseSquare</a> | The property to be mortgaged.                     | <i>required</i> |
| free_mortgage | <a href="#">bool</a>       | If True, no money is added to the user's balance. | <i>required</i> |

**Returns:**

| Name                 | Type                                 | Description                                              |
|----------------------|--------------------------------------|----------------------------------------------------------|
| PropertyRelationship | <a href="#">PropertyRelationship</a> | The updated relationship reflecting the active mortgage. |

**Raises:**

| Type                               | Description                                                                                                         |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <a href="#">MaliciousUserInput</a> | If the target is not a buyable square, is unowned, is owned by another player, or is already mortgaged.             |
| <a href="#">GameLogicError</a>     | If the user tries to mortgage a property while there are still houses built on any property within its color group. |

**• Source code in** [magnate/game\\_utils.py](#)

```
def _set_mortgage(game: Game, user: CustomUser, target_square: BaseSquare, free_mortgage: bool) -> PropertyRelationship:
 """
 Mortgages a property to receive immediate funds (half the buy price).
 Ensures that no properties within the same color group currently have houses built.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user attempting the mortgage.
 target_square (BaseSquare): The property to be mortgaged.
 free_mortgage (bool): If True, no money is added to the user's balance.

 Returns:
 PropertyRelationship: The updated relationship reflecting the active mortgage.

 Raises:
 MaliciousUserInput: If the target is not a buyable square, is unowned,
 is owned by another player, or is already mortgaged.
 GameLogicError: If the user tries to mortgage a property while there are still houses
 built on any property within its color group.
 """
 target_square = target_square.get_real_instance()
 if not (isinstance(target_square, PropertySquare) or
 isinstance(target_square, BridgeSquare) or
 isinstance(target_square, ServerSquare)):
 raise MaliciousUserInput(user, "tried to mortgage a non property/bridge/server square")

 relationship = _get_relationship(game=game, square=target_square)

 if relationship is None:
 raise MaliciousUserInput(user, "no user owns this square")

 if relationship.owner.pk != user.pk:
 raise MaliciousUserInput(user, "tried to mortgage an unowned property")
```

```

if relationship.mortgage:
 raise MaliciousUserInput(user, "tried to mortgage an already mortgaged property")

if isinstance(target_square, PropertySquare):
 group_has_houses = PropertyRelationship.objects.filter(
 game=game,
 owner=user,
 square__property__square__group=target_square.group,
 houses__gt=0
).exists()

 if group_has_houses:
 raise GameLogicError("cannot mortgage a property while there are houses in the group")

relationship.mortgage = True
relationship.save()
stats = PlayerGameStatistic.objects.get(user=user, game=game)
stats.num_mortgages += 1

if not free_mortgage:
 mortgage_value = target_square.buy_price // 2
 game.money[str(user.pk)] += mortgage_value

 stats.won_money += mortgage_value
 game.save()

stats.save()

return relationship

```

### 3.1.14 \_unset\_mortgage(game, user, target\_square, free\_unset\_mortgage)

Lifts the mortgage from a property by paying the required fee (half the buy price). Updates the player's lost money statistics.

#### Parameters:

| Name                             | Type                    | Description                                          | Default         |
|----------------------------------|-------------------------|------------------------------------------------------|-----------------|
| <code>game</code>                | <code>Game</code>       | The current game instance.                           | <i>required</i> |
| <code>user</code>                | <code>CustomUser</code> | The user lifting the mortgage.                       | <i>required</i> |
| <code>target_square</code>       | <code>BaseSquare</code> | The property to unmortgage.                          | <i>required</i> |
| <code>free_unset_mortgage</code> | <code>bool</code>       | If True, the user is not charged the unmortgage fee. | <i>required</i> |

#### Returns:

| Name                              | Type                              | Description                                              |
|-----------------------------------|-----------------------------------|----------------------------------------------------------|
| <code>PropertyRelationship</code> | <code>PropertyRelationship</code> | The updated relationship reflecting the lifted mortgage. |

#### Raises:

| Type                            | Description                                                                                                   |
|---------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>MaliciousUserInput</code> | If the target is not a buyable square, is unowned, is owned by another player, or is not currently mortgaged. |



• Source code in `magnate/game_utils.py`

```
def _unset_mortgage(game: Game, user: CustomUser, target_square: BaseSquare, free_unset_mortgage: bool) -> PropertyRelationship:
 """
 Lifts the mortgage from a property by paying the required fee (half the buy price).
 Updates the player's lost money statistics.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user lifting the mortgage.
 target_square (BaseSquare): The property to unmortgage.
 free_unset_mortgage (bool): If True, the user is not charged the unmortgage fee.

 Returns:
 PropertyRelationship: The updated relationship reflecting the lifted mortgage.

 Raises:
 MaliciousUserInput: If the target is not a buyable square, is unowned,
 is owned by another player, or is not currently mortgaged.
 """
 target_square = target_square.get_real_instance()
 if not (isinstance(target_square, PropertySquare) or
 isinstance(target_square, BridgeSquare) or
 isinstance(target_square, ServerSquare)):
 raise MaliciousUserInput(user, "tried to unset mortgage a non property/bridge/server square")

 relationship = _get_relationship(game=game, square=target_square)

 if relationship is None:
 raise MaliciousUserInput(user, "no user owns this square")

 if relationship.owner.pk != user.pk:
 raise MaliciousUserInput(user, "tried to unset mortgage an unowned property")

 if not relationship.mortgage:
 raise MaliciousUserInput(user, "tried to unset mortgage a not mortgaged property")

 relationship.mortgage = False
 relationship.save()
 target_square = target_square.get_real_instance()

 if not free_unset_mortgage:
 mortgage_value = target_square.buy_price // 2
 game.money[str(user.pk)] -= mortgage_value
 game.save()
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.lost_money += mortgage_value

 return relationship
```

### 3.1.15 GameManager

The primary gateway for all frontend actions. Every action sent via WebSocket or REST must be processed through the `process_action` method.

### 3.1.16 `magnate.games.GameManager`

State machine and core processor for Game Actions.

The GameManager encapsulates the logic for each specific phase of a player's turn, processing polymorphic Action objects and mutating the database state accordingly.

#### • Source code in `magnate/games.py`

```
class GameManager:
 """
 State machine and core processor for Game Actions.

 The GameManager encapsulates the logic for each specific phase of a
 player's turn, processing polymorphic Action objects and mutating the
 database state accordingly.
 """
 #####
 # Phase logic
 #####

 ROLL_THE_DICES = Game.GamePhase.roll_the_dices
 CHOOSE_SQUARE = Game.GamePhase.choose_square
 MANAGEMENT = Game.GamePhase.management
 BUSINESS = Game.GamePhase.business
 ANSWER_TRADE_PROPOSAL = Game.GamePhase.proposal_acceptance
 LIQUIDATION = Game.GamePhase.liquidation
 AUCTION = Game.GamePhase.auction
 PROPOSAL_ACCEPTANCE = Game.GamePhase.proposal_acceptance
 CHOOSE_FANTASY = Game.GamePhase.choose_fantasy
 END_GAME = Game.GamePhase.end_game

 @classmethod
 @database_sync_to_async
 def process_action(cls, game: Game, user: CustomUser, action: Action) -> Response:
 """
 The only public method exposed in the API. It processes each action
 in dedicated functions depending on the current phase and returns
 a Response.

 Args:
 game (Game): The current active game instance.
 user (CustomUser): The user performing the action.
 action (Action): The deserialized action payload.

 Returns:
 Response: A response object representing the outcome.

 Raises:
 MaliciousUserInput: If the user acts out of turn or phase.
 GameLogicError: If an unrecognized phase is encountered.
 """

 if isinstance(action, ActionSurrender):
 cls._bankrupt_player(game, user)
 if game.phase == cls.END_GAME:
 return cls._end_game_logic(game, user, action)
```

```

 response = Response()
 return _add_basic_response_data(game, response)

 if user.pk != game.active_phase_player.pk and not isinstance(action, ActionBid): # if auction there are no to
 raise MaliciousUserInput(user, "is not the active player")

 if game.phase == cls.ROLL_THE_DICES:
 if isinstance(action, ActionPayBail):
 response = cls._pay_bail_logic(game, user, action)
 elif isinstance(action, ActionThrowDices):
 response = cls._roll_dices_logic(game, user, action)
 else:
 raise MaliciousUserInputAction(game, user, action)
 elif game.phase == cls.CHOOSE_SQUARE:
 if not isinstance(action, ActionMoveTo):
 raise MaliciousUserInputAction(game, user, action)
 response = cls._square_chosen_logic(game, user, action)
 elif game.phase == cls.CHOOSE_FANTASY:
 if not isinstance(action, ActionChooseCard):
 raise MaliciousUserInputAction(game, user, action)
 response = cls._choose_fantasy_logic(game, user, action)
 elif game.phase == cls.MANAGEMENT:
 if not isinstance(action, (ActionBuySquare, ActionDropPurchase, ActionTakeTram, ActionNextPhase)):
 raise MaliciousUserInputAction(game, user, action)
 response = cls._management_logic(game, user, action)
 elif game.phase == cls.BUSINESS or game.phase == cls.LIQUIDATION:
 if not isinstance(action, (ActionBuild, ActionDemolish, ActionTradeProposal, ActionMortgageSet, ActionMortgageSet)):
 raise MaliciousUserInputAction(game, user, action)
 response = cls._business_logic(game, user, action)
 elif game.phase == cls.ANSWER_TRADE_PROPOSAL:
 if not isinstance(action, ActionTradeAnswer):
 raise MaliciousUserInputAction(game, user, action)
 response = cls._answer_trade_proposal_logic(game, user, action)
 elif game.phase == cls.AUCTION:
 if not isinstance(action, ActionBid):
 raise MaliciousUserInputAction(game, user, action)
 response = cls._bid_property_auction_logic(game, user, action)
 elif game.phase == cls.END_GAME:
 pass
 # en principio no check instance porque la partida ya ha acabado
 # response = cls._end_game_logic(game,user,action)
 # return cls._end_game_logic(game,user,action)
 else:
 raise GameLogicError(f"Unrecognized or unhandled phase: {game.phase}")

 if game.phase == cls.END_GAME:
 response = cls._end_game_logic(game, user, action)

 return _add_basic_response_data(game, response)

@staticmethod
def _pay_bail_logic(game: Game, user: CustomUser, action: ActionPayBail) -> Response:
 """
 Processes the action to pay the jail bail. Cancels previous timers, deducts money, resets jail turns, updates
 statistics, and sets the timer for the next phase.

 Args:

```

```

 game (Game): The current game instance.
 user (CustomUser): The jailed user.
 action (ActionPayBail): The bail action payload.

Returns:
 Response: The standard response.

Raises:
 MaliciousUserInput: If the user is not actually in jail.
 GameLogicError: If the user does not have enough money.
"""

GameManager._cancel_all_timers(game)

response: ResponseChooseSquare = ResponseChooseSquare()

square = _get_user_square(game, user).get_real_instance()

if not isinstance(square, JailSquare):
 raise MaliciousUserInput(user, "is not in jail")

remaining = game.jail_remaining_turns.get(str(user.pk), 0)
if remaining == 0:
 raise MaliciousUserInput(user, "is not in jail (no turns remaining)")

if len(game.possible_destinations) < 1:
 raise MaliciousUserInput(user, "cannot pay bail now")

if not action.to_pay:
 # Remain in jail
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.turns_in_jail += 1
 stats.save()
 game.phase = GameManager.BUSINESS
 game.possible_destinations = dict()
 game.save()
 response.path = [int(square.custom_id)]
 GameManager._set_next_phase_timer(game, user) #TODO: revisar este timer
 return response

The player paid the bail

bail_price = square.bail_price

if game.money[str(user.pk)] < bail_price:
 raise GameLogicError("not enough money to pay bail")

dest_square_id = next(iter(game.possible_destinations))
steps = game.possible_destinations[dest_square_id] # pasos

move_result = _move_player_logic(square, steps)
response.path = move_result["path"]

game.money[str(user.pk)] -= bail_price
game.parking_money += bail_price
game.jail_remaining_turns[str(user.pk)] = 0
game.possible_destinations = dict()

game.positions[str(user.pk)] = int(dest_square_id)

```

```

square = _get_square_by_custom_id(dest_square_id)
_apply_square_arrival(game, user, response, square, move_result["passed_go"])

stats = PlayerGameStatistic.objects.get(user=user, game=game)
stats.lost_money += bail_price
GameManager._set_next_phase_timer(game, user)

stats.save()
game.save()
return response

@staticmethod
def _roll_dices_logic(game: Game, user: CustomUser, action: ActionThrowDices) -> Response:
 """
 Handles the dice rolling process, including double/triple streaks and jail mechanics.

 This method calculates the result of rolling three dice (including the bus die),
 updates the player's movement streak, and determines possible destination squares.
 It also handles logic for players attempting to leave jail via doubles or
 forced payment.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user throwing the dice.
 action (ActionThrowDices): The dice throw action payload.

 Returns:
 ResponseThrowDices: Contains dice values, streak status, possible destinations,
 and the path to be traversed if only one destination exists.

 Raises:
 GameLogicError: If the player has a jailed status but is not on a jail square.
 """

 GameManager._cancel_all_timers(game)

 response: ResponseThrowDices = ResponseThrowDices()

 # jail logic
 remaining_jail_turns = game.jail_remaining_turns.get(str(user.pk), 0)
 is_jailed = remaining_jail_turns > 0
 was_jailed = is_jailed

 mock = None
 if (isinstance(game.possible_destinations, dict)
 and '__mock_dice__' in game.possible_destinations):
 mock = game.possible_destinations['__mock_dice__']
 game.possible_destinations = dict()

 if mock:
 d1, d2, d3 = mock[0], mock[1], mock[2]
 else:
 d1 = random.randint(1, 6)
 d2 = random.randint(1, 6)
 d3 = random.randint(1, 6) if not is_jailed else 0

 response.dice1, response.dice2, response.dice_bus = d1, d2, d3

 triples = d3 <= 3 and (d1 == d2 == d3)

```

```

doubles = (d1 == d2) and not triples

response.triple = triples

current_pos_square = _get_user_square(game, user).get_real_instance()
current_pos_id = current_pos_square.custom_id

jail logic
remaining_jail_turns = game.jail_remaining_turns.get(str(user.pk), 0)
is_jailed = remaining_jail_turns > 0
was_jailed = is_jailed

if is_jailed:
 jail_sq = current_pos_square
 if isinstance(jail_sq, JailSquare):
 if remaining_jail_turns == 1: # Forced to leave jail (last turn)
 game.money[str(user.pk)] -= jail_sq.bail_price
 game.jail_remaining_turns[str(user.pk)] = 0
 is_jailed = False
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.turns_in_jail += 1
 stats.lost_money += jail_sq.bail_price
 stats.save()
 elif doubles: # Leaves jail for free due to doubles
 game.jail_remaining_turns[str(user.pk)] = 0
 game.streak = 0
 is_jailed = False
 else:
 # stays in jail
 game.jail_remaining_turns[str(user.pk)] -= 1
 game.phase = GameManager.ROLL_THE_DICES
 game.save()
 else:
 raise GameLogicError(f"Cannot be in jail status and not in jail square")

if triples:
 # path current -> decided in chosen
 response.triple = True
 square = current_pos_square
 all_squares = BaseSquare.objects.filter(board=square.board)
 game.streak = 0
 response.streak = game.streak
 # All squares are suitable destinations
 possible_destinations = [s.custom_id for s in all_squares]
 possible_destinations.remove(_get_jail_square().custom_id)
 game.possible_destinations = {c_id: 0 for c_id in possible_destinations}
 response.destinations = [int(k) for k in game.possible_destinations.keys()]
 game.phase = GameManager.CHOOSE_SQUARE
 response.path = [current_pos_id]
 game.save()
 GameManager._set_next_phase_timer(game, user)
 return response
elif doubles: # doubles streak only if not getting out of jail via doubles
 if game.streak >= 2:
 # path -> current and jail
 jail_square = _get_jail_square()
 response.destinations = [jail_square.custom_id]
 game.streak = 0

```

```

 response.streak = game.streak

 game.positions[str(user.pk)] = int(jail_square.custom_id)
 game.jail_remaining_turns[str(user.pk)] = 3
 response.path = [current_pos_id, jail_square.custom_id]

 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.times_in_jail += 1
 stats.save()

 game.phase = GameManager.LIQUIDATION

 game.save()
 GameManager._set_next_phase_timer(game, user)
 return response
 elif not was_jailed:
 game.streak = game.streak + 1
else:
 game.streak = 0

response.streak = game.streak

dice_combinations = _compute_dice_combinations(d1, d2, d3) #ya con el bus cancelado si se está en la cárcel
game.possible_destinations, passed_go_map = _get_possible_destinations_ids(game, user, dice_combinations)

response.destinations = [int(k) for k in game.possible_destinations.keys()]

if is_jailed:
 # Now it should decide whether to pay bail or not
 if int(current_pos_id) not in response.destinations:
 response.destinations.append(int(current_pos_id))
 response.path = []
elif len(game.possible_destinations) > 1:
 # path in square chosen logic
 game.phase = GameManager.CHOOSE_SQUARE
 response.path = [current_pos_id]
else:
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.walked_squares += dice_combinations[0]
 stats.save()
 dest_square_id = next(iter(game.possible_destinations))
 steps = game.possible_destinations[dest_square_id]

 # Use _move_player_logic to get the traversed path and check for "Go to Jail" or "Passing Go".
 move_result = _move_player_logic(current_pos_square, steps)
 response.path = move_result["path"]

 if move_result["jailed"]:
 # landing in go to jail; update state to jail the player.
 jail = JailSquare.objects.first()
 if jail is None:
 raise GameDesignError('no jail in game')

 game.positions[str(user.pk)] = int(jail.custom_id)
 game.jail_remaining_turns[str(user.pk)] = 3
 game.phase = GameManager.LIQUIDATION
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.times_in_jail += 1
 stats.save()

```

```

 else:
 game.positions[str(user.pk)] = int(dest_square_id)
 square = _get_square_by_custom_id(dest_square_id)
 _apply_square_arrival(game, user, response, square, move_result["passed_go"])
 game.possible_destinations = dict()

 game.save()
 GameManager._set_next_phase_timer(game, user)
 return response

@staticmethod
def _square_chosen_logic(game: Game, user: CustomUser, action: Action) -> Response:
 """
 Handles the user's final destination selection.
 Moves the player, evaluates passing Go, updates walked squares statistics,
 and resets timers.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The current user.
 action (ActionMoveTo): Action indicating the chosen square.

 Returns:
 Response: A ResponseChooseSquare object including the path taken.

 Raises:
 MaliciousUserInputAction: If the action type is incorrect.
 MaliciousUserInput: If the chosen square is not in the allowed `possible_destinations`.
 GameDesignError: If the jail square is missing from the board.
 """

 GameManager._cancel_all_timers(game)
 response: ResponseChooseSquare = ResponseChooseSquare()

 if not isinstance(action, ActionMoveTo):
 raise MaliciousUserInputAction(game, user, action)

 square = action.square

 if str(square.custom_id) not in game.possible_destinations:
 raise MaliciousUserInput(user, "tried to move to an illegal square")

 current_pos_id = game.positions[str(user.pk)]
 current_pos_square = _get_square_by_custom_id(current_pos_id).get_real_instance()

 steps = game.possible_destinations.get(str(square.custom_id))

 if steps == 0: #triples
 move_result = {
 "jailed": isinstance(square.get_real_instance(), GoToJailSquare),
 "passed_go": False,
 "path": [current_pos_id, square.custom_id]
 }
 else:
 move_result = _move_player_logic(current_pos_square, steps)

 response.path = move_result["path"]

```



```

if move_result["jailed"]:
 # landing in go to jail; update state to jail the player.
 jail = JailSquare.objects.first()
 if jail is None:
 raise GameDesignError('no jail in game')

 game.positions[str(user.pk)] = int(jail.custom_id)
 game.jail_remaining_turns[str(user.pk)] = 3
 game.phase = GameManager.LIQUIDATION
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.times_in_jail += 1
 stats.save()
else:
 game.positions[str(user.pk)] = int(square.custom_id)
 square = _get_square_by_custom_id(square.custom_id)
 _apply_square_arrival(game, user, response, square, move_result["passed_go"])

stats = PlayerGameStatistic.objects.get(user=user, game=game)
stats.walked_squares += steps
stats.save()

game.possible_destinations = dict()
game.save()

GameManager._set_next_phase_timer(game, user)
return response

@staticmethod
def _choose_fantasy_logic(game: Game, user: CustomUser, action: ActionChooseCard) -> Response:
 """
 Applies a pre-existing fantasy event or randomly generates a new one.
 Determines the next game phase and updates timers.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The active user making the choice.
 action (ActionChooseCard): The payload of the card choice action.

 Returns:
 Response: A ResponseChooseFantasy object containing the fantasy event result.
 """

 GameManager._cancel_all_timers(game)

 response = ResponseChooseFantasy()
 fantasy_event = game.fantasy_event
 generate = not action.chosen_revealed_card
 new_fantasy = None

 if not generate:
 card_cost = fantasy_event.card_cost
 if game.money[str(user.pk)] < card_cost:
 raise MaliciousUserInput(user, "not enough money to pay for the revealed card")

 game.money[str(user.pk)] -= card_cost
 game.parking_money += card_cost
 game.money = dict(game.money) # to update
 game.save(update_fields=['money', 'parking_money'])
 stats = PlayerGameStatistic.objects.get(user=user, game=game)

```

```

 stats.lost_money += card_cost
 stats.save()

 fantasy_result = apply_fantasy_event(game, user, fantasy_event)
 fantasy_result.save()
 response.fantasy_result = fantasy_result
 game.fantasy_event = None
 game.save()

 else:
 new_fantasy = FantasyEventFactory.generate()
 new_fantasy.save()
 fantasy_result = apply_fantasy_event(game, user, new_fantasy)
 fantasy_result.save()
 response.fantasy_result = fantasy_result
 game.fantasy_event = None
 game.save()

 if game.streak == 0:
 if game.money[str(user.pk)] < 0:
 game.phase = GameManager.LIQUIDATION
 else:
 game.phase = GameManager.BUSINESS
 else:
 game.phase = GameManager.ROLL_THE_DICES

 if game.phase == GameManager.BUSINESS or game.phase == GameManager.LIQUIDATION:
 GameManager._set_next_phase_timer(game, user)
 elif game.phase == GameManager.ROLL_THE_DICES:
 GameManager._set_kick_out_timer(game, user)
 pass

 game.save()
 return response

@staticmethod
def _management_logic(game: Game, user: CustomUser, action: Action) -> Response:
 """
 Management phase logic (purchases, dropping a purchase, taking a tram).
 Includes handling purchases for PropertySquare, ServerSquare, and BridgeSquare,
 updating lost money statistics and timers.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The acting user.
 action (Action): The payload of the action taken (e.g., ActionBuySquare, ActionTakeTram).

 Returns:
 Response: Standard response.

 Raises:
 MaliciousUserInputAction: If the action does not fit the phase, context, or square type.
 MaliciousUserInput: If the user tries to take a tram to an invalid square or lacks funds.
 """
 GameManager._cancel_all_timers(game)

 current_square = _get_user_square(game, user).get_real_instance()
 prop_rel = _get_relationship(game, current_square)

```

```

if isinstance(action, ActionBuySquare):
 if not isinstance(current_square, (PropertySquare, ServerSquare, BridgeSquare)):
 raise MaliciousUserInputAction(game, user, action)

 _handle_property_acquisition(game, user, current_square)
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.lost_money += current_square.buy_price
 stats.save()
 game.money[str(user.pk)] -= current_square.buy_price

elif isinstance(action, ActionDropPurchase):
 if isinstance(action.square, (PropertySquare, ServerSquare, BridgeSquare)):
 return GameManager._initiate_auction(game, action.square, user)
 else:
 raise MaliciousUserInputAction(game, user, action)
elif isinstance(action, ActionTakeTram):
 if isinstance(current_square, TramSquare):
 square = action.square
 tram_squares = TramSquare.objects.filter()
 tram_square_actual_id = game.positions[str(user.pk)]
 tram_squares_extern_ids = [s.custom_id for s in tram_squares]

 if square.custom_id == tram_square_actual_id: # case stay in the same square, free
 pass
 elif square.custom_id in tram_squares_extern_ids: # Move to another square
 if game.money[str(user.pk)] < square.buy_price:
 raise MaliciousUserInput(user, "does not have enough money to take tram")
 game.money[str(user.pk)] -= square.buy_price
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.lost_money += square.buy_price
 stats.save()
 game.positions[str(user.pk)] = int(square.custom_id)
 else:
 raise MaliciousUserInput(user, "tried to take a tram to a non tram square")
 else:
 raise MaliciousUserInputAction(game, user, action)
elif isinstance(action, ActionNextPhase):
 # TODO: Remove?
 pass
else:
 raise MaliciousUserInputAction(game, user, action)

if game.phase == GameManager.MANAGEMENT:
 if game.streak == 0:
 game.phase = GameManager.BUSINESS
 else:
 game.phase = GameManager.ROLL_THE_DICES

if game.phase == GameManager.BUSINESS:
 GameManager._set_next_phase_timer(game, user)
elif game.phase == GameManager.ROLL_THE_DICES:
 GameManager._set_kick_out_timer(game, user)
 pass

game.save()
return Response()

```

@staticmethod

def \_business\_logic(game: Game, user: CustomUser, action: Action) -> Response:

```

"""
Unifies the business and liquidation phases. Handles building, demolishing,
trading, mortgaging, and proceeding to the next turn.

Args:
 game (Game): The current game instance.
 user (CustomUser): The active user.
 action (Action): The payload of the action taken.

Returns:
 Response: Standard response.

Raises:
 MaliciousUserInput: If restrictions (e.g., negative balance on next turn) are unmet.
"""

```

```

GameManager._cancel_all_timers(game)
Logic for the business phase where players can build houses, trade, etc.
if isinstance(action, ActionBuild):
 # Check owner
 building_square = action.square
 relationship = _build_square(game, user, building_square, action.houses, False)

elif isinstance(action, ActionDemolish):
 # Similiar to build
 demolition_square = action.square
 relationship = _demolish_square(game, user, demolition_square, action.houses, False)

elif isinstance(action, ActionTradeProposal):
 relationship = GameManager._propose_trade(game, user, action)
 return Response()

elif isinstance(action, ActionMortgageSet):
 relationship = _set_mortgage(game, user, action.square, False)

elif isinstance(action, ActionMortgageUnset):
 relationship = _unset_mortgage(game, user, action.square, False)
elif isinstance(action, ActionNextPhase):
 current_money = game.money[str(user.pk)]

 if game.phase == GameManager.BUSINESS:
 if current_money >= 0:
 GameManager._next_turn(game, user)
 else:
 game.phase = GameManager.LIQUIDATION
 game.save()

 elif game.phase == GameManager.LIQUIDATION:
 if current_money >= 0:
 GameManager._next_turn(game, user)
 else:
 raise MaliciousUserInput(user, "Cannot end in NEGATIVE")
 return Response() # timers set in next turn

GameManager._set_next_phase_timer(game, user)
return Response()

```

```
@staticmethod
```

```

def _answer_trade_proposal_logic(game: Game, user: CustomUser, action: Action) -> Response:
 """
 Processes the response (accept/reject) to a trade proposal.
 Transfers properties and money ensuring no player goes into negative balance,
 updates trade and money statistics for both players, and returns the turn
 to the proposer.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The targeted user responding to the offer.
 action (ActionTradeAnswer): The user's response payload (contains `choose` boolean).

 Returns:
 Response: Standard response.

 Raises:
 GameLogicError: If there is no active proposal in the game state.
 MaliciousUserInput: If an unauthorized user attempts to answer,
 or if the trade results in a negative money balance for either party.
 MaliciousUserInputAction: If an invalid action type is passed.
 """
 if isinstance(action, ActionTradeAnswer):
 offer = game.proposal

 if offer is None:
 raise GameLogicError("there should be an active trade proposal")

 if user.pk != offer.destination_user.pk:
 raise MaliciousUserInput(user, f"cannot accept proposal {offer}")

 offering = offer.player

 offered_money = offer.offered_money
 asked_money = offer.asked_money
 offered_properties = offer.offered_properties
 asked_properties = offer.asked_properties

 if action.choose:
 # Money logic (ensure no player goes to negative)
 money_diff = offered_money - asked_money
 acceptor_money = game.money[str(user.pk)] + money_diff
 offering_money = game.money[str(offering.pk)] - money_diff

 if acceptor_money < 0:
 raise MaliciousUserInput(user, f"cannot go to negative in trade")
 if offering_money < 0:
 raise MaliciousUserInput(offering, f"cannot go to negative in trade")

 for relationship in offered_properties.all():
 _handle_property_trade(game, user, relationship.square)

 for relationship in asked_properties.all():
 _handle_property_trade(game, offering, relationship.square)

 game.money[str(offering.pk)] = offering_money
 stats = PlayerGameStatistic.objects.get(user=offering, game=game)
 if money_diff > 0:
 stats.lost_money += abs(money_diff)
 else:

```

```

 stats.won_money += abs(money_diff)
 stats.num_trades += 1
 stats.save()

 game.money[str(user.pk)] = acceptor_money
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 if money_diff > 0:
 stats.won_money += abs(money_diff)
 else:
 stats.lost_money += abs(money_diff)
 stats.num_trades += 1
 stats.save()

 game.save()

 game.phase = GameManager.BUSINESS
 game.active_phase_player = offering
 game.proposal = None
 game.save()
 GameManager._set_next_phase_timer(game, offering)

 return Response()
else:
 raise MaliciousUserInputAction(game, user, action)

@staticmethod
def _initiate_auction(game: Game, square: BaseSquare, player: CustomUser) -> Response:
 """
 Transitions the game into an AUCTION phase for a dropped property.

 Args:
 game (Game): The current game instance.
 square (BaseSquare): The property square going up for auction.

 Returns:
 Response: Standard response.
 """
 GameManager._cancel_all_timers(game)

 ActionDropPurchase.objects.get_or_create(game=game, player=player, square=square)

 game.next_phase_task_id = None
 game.phase = GameManager.AUCTION

 auction = Auction.objects.create(game=game, square=square, is_active=True, bids = {})
 game.current_auction = auction
 game.save()

 # TODO: Remove magic number
 GameManager._set_auction_timer(game)
 # TODO: Remove in production
 game.refresh_from_db()

 return Response()

@staticmethod
def _bid_property_auction_logic(game: Game, user: CustomUser, action: Action) -> Response:
 """
 Registers a player's bid in an active auction.

```

Verifies that the player has not previously dropped the purchase of that property.  
Automatically ends the auction if all eligible players have bid.

Args:

game (Game): The current game instance.  
user (CustomUser): The user placing the bid.  
action (ActionBid): The action carrying the bid amount.

Returns:

Response: Standard response, or the resolved auction response if all players have bid.

Raises:

GameLogicError: If there is no active auction found.  
MaliciousUserInputAction: If an invalid action type is passed.  
MaliciousUserInput: If the user bids twice, triggered the auction drop themselves,  
or if the bid exceeds their current balance.

"""

```
if not isinstance(action, ActionBid):
 raise MaliciousUserInputAction(game, user, action)

auction = game.current_auction
if not auction:
 raise GameLogicError("No active auction")

bids = auction.bids
user has not bid yet
if bids.get(str(user.pk)):
 raise MaliciousUserInput(user, "User already placed a bid in this auction")

user who started the bid cant bid
dropped = ActionDropPurchase.objects.filter(game=game, player=user, square=auction.square).exists()
if dropped:
 raise MaliciousUserInput(user, "cannot bid in an auction they triggered")

user has enough money -> compulsory for auctions
amount = action.amount
if amount > game.money[str(user.pk)]:
 raise MaliciousUserInput(user, "Bid amount exceeds current balance")

bids[str(user.pk)] = amount
auction.bids = bids
auction.save()

game.save()

eligible_players_count = game.players.exclude(
 pk__in=ActionDropPurchase.objects.filter(game=game, square=auction.square).values('player')
).count()

if len(bids) >= eligible_players_count:
 # all bidded
 GameManager._cancel_auction_timer(game)
 return GameManager._end_auction(game) #type:ignore

return Response()
```

@staticmethod

```

def _end_auction(game: Game) -> Response | None:
 """
 Ends an active auction, resolves the winner based on the highest bid,
 handles ties, and transitions the game state back to BUSINESS.

 Args:
 game (Game): The current game instance.

 Returns:
 Auction: The finalized auction object.

 Raises:
 GameLogicError: If not in AUCTION phase or state is missing square ID.
 """
 ## call this with a timer -> end of the auction

 if game.phase == GameManager.END_GAME:
 return None # Inevitable if the auction callback is triggered after the game has ended, just ignore it

 if game.phase != GameManager.AUCTION:
 raise GameLogicError("Tried to end auction but game is not in auction phase")

 auction = game.current_auction
 if not auction:
 raise GameLogicError("Game is in AUCTION phase but no auction object found")

 square = auction.square.get_real_instance()
 bids = auction.bids

 # no one bid
 if not bids:
 if game.streak == 0:
 game.phase = GameManager.BUSINESS
 else:
 game.phase = GameManager.ROLL_THE_DICES

 auction.is_active = False
 auction.winner = None
 auction.final_amount = 0
 auction.is_tie = False
 auction.save()

 game.current_auction = None
 game.save()

 if game.phase == GameManager.BUSINESS:
 GameManager._set_next_phase_timer(game, game.active_turn_player)
 elif game.phase == GameManager.ROLL_THE_DICES:
 GameManager._set_kick_out_timer(game, game.active_turn_player)
 pass

 game.save()
 return ResponseAuction(auction=auction)

max_bid_amount = max(bids.values())
winners = [int(uid) for uid, amt in bids.items() if amt == max_bid_amount]

if len(winners)> 1 or max_bid_amount == 0:

```



```

 if game.streak == 0:
 game.phase = GameManager.BUSINESS
 else:
 game.phase = GameManager.ROLL_THE_DICES

 auction.is_active = False
 auction.winner = None
 auction.final_amount = max_bid_amount

 auction.is_tie = len(winners) > 1
 auction.save()

 game.current_auction = None
 game.save()

 if game.phase == GameManager.BUSINESS:
 GameManager._set_next_phase_timer(game, game.active_turn_player)
 elif game.phase == GameManager.ROLL_THE_DICES:
 GameManager._set_kick_out_timer(game, game.active_turn_player)
 pass

 game.save()
 return ResponseAuction(auction=auction)

winner_id = winners[0]
if str(winner_id) not in game.money: # surrenders mid auction -> no winner
 game.phase = GameManager.BUSINESS if game.streak == 0 else GameManager.ROLL_THE_DICES
 auction.is_active = False
 auction.save()

 game.current_auction = None
 game.save()

 if game.phase == GameManager.BUSINESS:
 GameManager._set_next_phase_timer(game, game.active_turn_player)
 elif game.phase == GameManager.ROLL_THE_DICES:
 GameManager._set_kick_out_timer(game, game.active_turn_player)
 pass

 game.save()
 return ResponseAuction(auction=auction)

winner = CustomUser.objects.get(pk=winner_id)
highest_bid = max_bid_amount

game.money[str(winner.pk)] -= highest_bid
stats = PlayerGameStatistic.objects.get(user=winner, game=game)
stats.lost_money += highest_bid
stats.save()

_handle_property_acquisition(game, winner, square)

auction.winner = winner
auction.final_amount = highest_bid
auction.is_active = False
auction.is_tie = False
auction.save()

if game.streak == 0:

```

```

 game.phase = GameManager.BUSINESS
 else:
 game.phase = GameManager.ROLL_THE_DICES

 game.current_auction = None
 game.save()

 if game.phase == GameManager.BUSINESS:
 GameManager._set_next_phase_timer(game, game.active_turn_player)
 elif game.phase == GameManager.ROLL_THE_DICES:
 GameManager._set_kick_out_timer(game, game.active_turn_player)
 pass

 game.save()

 return ResponseAuction(auction=auction)

@classmethod
def _next_turn(cls, game: Game, user: CustomUser) -> None:
 """
 Advances the game to the next player in the established order.
 Resets bot flags and manages timers for the new turn.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The current active user whose turn is ending.

 Returns:
 None

 Raises:
 GameLogicError: If the current or next player cannot be identified in the player list.
 """

 players_list = list(game.players.all())
 num_players = len(players_list)
 current_index = -1
 current_player_id = -1
 for p in players_list:
 if p.pk == game.active_turn_player.pk:
 current_player_id = p.pk
 current_index = game.ordered_players.index(current_player_id)
 break

 if current_index == -1:
 raise GameLogicError('current player not found')

 next_index = (current_index + 1) % num_players
 if next_index <= current_index:
 game.current_round += 1

 # The next active user is for both: phase and turn
 next_player = game.players.filter(pk=game.ordered_players[next_index]).first()
 if next_player is None:
 raise GameLogicError('next player is None')

 if game.current_round > game.max_rounds:

```

```

game.phase = GameManager.END_GAME
GameManager._cancel_all_timers(game)
if game.current_auction:
 game.current_auction.is_active = False
 game.current_auction.save()
 game.current_auction = None
game.save()

#cls._end_game_logic(game, next_player) #####
return

game.active_phase_player = next_player
game.active_turn_player = next_player
game.phase = GameManager.ROLL_THE_DICES
game.current_turn += 1
game.save()

bot_next = Bot.objects.filter(pk=next_player.pk).first()
if bot_next:
 bot_next.has_proposed_trade = False
 bot_next.save()

GameManager._cancel_all_timers(game)

GameManager._set_kick_out_timer(game, next_player)
pass

game.save()

@classmethod
def _propose_trade(cls, game: Game, user: CustomUser, action: ActionTradeProposal) -> None:
 """
 Validates and initiates a trade proposal between players.
 Checks that there are no built houses in the involved groups.
 Updates bot states if applicable and changes the phase to PROPOSAL_ACCEPTANCE.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user proposing the trade.
 action (ActionTradeProposal): The payload detailing offered/asked money and properties.

 Returns:
 None

 Raises:
 MaliciousUserInput: If the action involves negative money, references an invalid
 destination player, attempts to trade unowned properties, or tries
 to trade properties belonging to a group with built houses.
 """
 if action.player.pk != user.pk or action.offered_money < 0 or action.asked_money < 0:
 raise MaliciousUserInput(user, "cannot do operation")
 if action.destination_user.pk not in game.ordered_players:
 # FIXME: Change to internal order so that it handles player change to AI
 raise MaliciousUserInput(user, "referenced a player that is not in game")

 asked_properties_list = action.asked_properties.all()
 asked_count = PropertyRelationship.objects.filter(
 game=game,

```

```

 owner=action.destination_user,
 id__in=action.asks_properties.all()
).count()

 if asked_count != asked_properties_list.count():
 raise MaliciousUserInput(user, "destination does not have enough properties")

 offered_properties_list = action.offered_properties.all()
 offered_count = PropertyRelationship.objects.filter(
 game=game,
 owner=action.player,
 id__in=offered_properties_list
).count()

 if offered_count != offered_properties_list.count():
 raise MaliciousUserInput(user, f"offer does not have enough properties: player owns {offered_count} but t

 all_trade_properties = list(asks_properties_list) + list(offered_properties_list)
 for rel in all_trade_properties:
 real_sq = rel.square.get_real_instance()
 if isinstance(real_sq, PropertySquare):
 group_has_houses = PropertyRelationship.objects.filter(
 game=game,
 square__propertysquare__group=real_sq.group,
 houses__gt=0
).exists()
 if group_has_houses:
 raise MaliciousUserInput(user, "cannot trade properties from a group with constructions")

 bot = Bot.objects.filter(pk=user.pk).first()
 if bot:
 bot.has_proposed_trade = True
 bot.save()

 game.phase = GameManager.PROPOSAL_ACCEPTANCE
 game.active_phase_player = action.destination_user
 game.proposal = action # type: ignore
 game.save()
 GameManager._set_next_phase_timer(game, action.destination_user)

 @classmethod
 def _bankrupt_player(cls, game: Game, user: CustomUser):
 """
 Eliminates a player from the game due to bankruptcy, clearing their properties
 and state. If only one player remains, ends the game. Otherwise, passes the turn
 to the next player.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user going bankrupt.

 Returns:
 None
 """
 try:
 current_idx = game.ordered_players.index(user.pk)
 next_pk = game.ordered_players[(current_idx + 1) % len(game.ordered_players)]
 next_player = CustomUser.objects.get(pk=next_pk)

```

```

except ValueError:
 next_player = None

if user.pk in game.ordered_players:
 game.ordered_players.remove(user.pk)

game.players.remove(user)
game.money.pop(str(user.pk), None)
game.positions.pop(str(user.pk), None)
game.jail_remaining_turns.pop(str(user.pk), None)

PropertyRelationship.objects.filter(game=game, owner=user).delete()
user.active_game = None
user.save()

if game.players.count() == 1:
 game.phase = GameManager.END_GAME
 GameManager._cancel_all_timers(game)

 if game.current_auction:
 auction = game.current_auction
 auction.is_active = False
 auction.save()
 game.current_auction = None

 game.save()
 return #endgame logic called afterwards

from .models import Bot
bots_in_game = Bot.objects.filter(id__in=game.players.values_list('id', flat=True))
if game.players.count() == bots_in_game.count(): #solo quedan bots
 game.phase = GameManager.END_GAME
 GameManager._cancel_all_timers(game)

 if game.current_auction:
 auction = game.current_auction
 auction.is_active = False
 auction.save()
 game.current_auction = None

 game.save()
 return

if game.active_turn_player.pk == user.pk and next_player:
 game.active_turn_player = next_player
 game.active_phase_player = next_player
 game.phase = GameManager.ROLL_THE_DICES
 game.streak = 0

 GameManager._cancel_all_timers(game)

 # new task
 GameManager._set_kick_out_timer(game, next_player)

game.save()

```

@classmethod

```

def _apply_end_bonuses(cls, game: Game, num_bonuses: int = 3) -> ResponseBonus:
 """
 Selects random bonus categories and awards extra money to players based on
 their game statistics.

 Args:
 game (Game): The current game instance.
 num_bonuses (int): The number of categories to pick for bonuses (default: 3).

 Returns:
 ResponseBonus: An object mapping the chosen bonus categories and their winners.
 """

 active_players = game.players.values_list('pk', flat=True)
 stats = PlayerGameStatistic.objects.filter(game=game, user__pk__in=active_players)

 valid_categories_data = []
 for category in BonusCategory.objects.all():
 field = category.stat_field
 max_value = stats.aggregate(Max(field)).get(f'{field}__max')
 if max_value and max_value > 0:
 valid_categories_data.append((category, max_value))

 chosen_data = random.sample(valid_categories_data, min(num_bonuses, len(valid_categories_data)))

 response = ResponseBonus()
 bonuses = {}

 for category, max_value in chosen_data:
 field = category.stat_field
 winners = list(stats.filter(**{field: max_value}).values_list('user__pk', flat=True))

 for pk in winners:
 game.money[str(pk)] = game.money.get(str(pk), 0) + category.bonus_amount

 bonuses[field] = {
 'bonus_amount': category.bonus_amount,
 'winners': winners
 }

 response.bonuses = bonuses
 game.save()
 return response

@classmethod
def _end_game_logic(cls, game: Game, user: CustomUser, action: Optional[Action] = None) -> Response:
 """
 Ends the game, applies final bonuses, generates the summary (GameSummary)
 with the net worth of all participants (including eliminated ones), and
 removes the bots.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user associated with triggering the end game phase.
 action (Action): The final action payload.

 Returns:
 Response: The final response containing the applied bonuses.
 """

```

```

Raises:
 GameLogicError: If the game has already been marked as finished.
"""
GameManager._cancel_all_timers(game)

if not game.finished:
 game.finished = True
 response = cls._apply_end_bonuses(game, num_bonuses=3)

 from django.utils import timezone

 # include eliminated users
 all_participants = PlayerGameStatistic.objects.filter(game=game).select_related('user')
 active_players = game.players.all()

 final_money_dict = {}

 for stat in all_participants:
 participant = stat.user
 if participant.username in [participant.username for participant in active_players]:
 # not eliminated
 final_money_dict[str(participant.username)] = _calculate_net_worth(game, participant)
 if not game.has_bots:
 participant.points += final_money_dict[str(participant.username)]
 participant.elo += final_money_dict[str(participant.username)]
 participant.save()
 else:
 final_money_dict[str(participant.username)] = 0

 bonus_per_player = {}
 for field, info in response.bonuses.items():
 bonus_amount = info['bonus_amount']
 for winner_pk in info['winners']:
 key = str(winner_pk)
 bonus_per_player[key] = bonus_per_player.get(key, 0) + bonus_amount

 pk_to_username = {str(stat.user.pk): stat.user.username for stat in all_participants}
 final_money_dict_pk = {pk: final_money_dict[username] for pk, username in pk_to_username.items()}

 # last_money = dinero final sin contar los bonuses
 last_money_dict = {
 pk: amount - bonus_per_player.get(pk, 0)
 for pk, amount in final_money_dict_pk.items()
 }
 response.last_money = last_money_dict

 winner = None
 max_money = -1

 for stat in all_participants:
 participant = stat.user

 if not game.has_bots:
 participant.num_played_games += 1

 p_money = final_money_dict.get(str(participant.username), 0)
 if p_money > max_money:
 max_money = p_money
 winner = participant

```

```

 participant.save()

 if winner:
 if not game.has_bots:
 winner.num_won_games += 1
 winner.save()

 if not game.has_bots:
 GameSummary.objects.create(
 game=game,
 start_date=game.datetime,
 end_date=timezone.now(),
 final_money=final_money_dict
)

 response.save()
 game.bonus_response = response
 game.save()

 from .models import Bot
 bots_in_game = Bot.objects.filter(id__in=game.players.values_list('id', flat=True))
 bots_in_game.delete()

 active_players.update(active_game=None)

 return response
else:
 raise GameLogicError('game was already ended')

@staticmethod
def _set_next_phase_timer(game: Game, user: CustomUser):
 """
 Cancels all active timers and sets a new timer for the next game phase.
 If the active user is a bot, it also schedules an immediate bot play task.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user whose turn or phase it currently is.

 Returns:
 None
 """
 from .tasks import next_phase_callback, bot_play_callback
 from .celery import app

 GameManager._cancel_all_timers(game)

 task = next_phase_callback.apply_async(args=[game.pk, user.pk], countdown=50000)
 game.next_phase_task_id = task.id
 game.save()

 if Bot.objects.filter(pk=user.pk).exists():
 bot_play_callback.apply_async(args=[game.pk, user.pk], countdown=random.randint(8, 12))

```



```

@staticmethod
def _set_kick_out_timer(game: Game, user: CustomUser):
 """
 Revokes any existing kick-out timer and sets a new one to remove an inactive player.
 If the user is a bot, it schedules a bot play task.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user subject to the kick-out timer.

 Returns:
 None
 """
 from .tasks import kick_out_callback, bot_play_callback
 from .celery import app

 if game.kick_out_task_id:
 app.control.revoke(game.kick_out_task_id, terminate=True)
 game.kick_out_task_id = None
 game.save()

 if Bot.objects.filter(pk=user.pk).exists():
 bot_play_callback.apply_async(args=[game.pk, user.pk], countdown=random.randint(8, 12))

@staticmethod
def _cancel_all_timers(game: Game):
 """
 Revokes both the next phase and kick-out Celery tasks if they are active,
 ensuring the current task is not self-cancelled. Clears the task IDs from the game.

 Args:
 game (Game): The current game instance.

 Returns:
 None
 """
 from .celery import app
 from celery import current_task

 current_task_id = None
 if current_task and hasattr(current_task, 'request'):
 current_task_id = getattr(current_task.request, 'id', None)

 if game.next_phase_task_id:
 if game.next_phase_task_id != current_task_id:
 app.control.revoke(game.next_phase_task_id, terminate=True)
 game.next_phase_task_id = None

 if game.kick_out_task_id:
 if game.kick_out_task_id != current_task_id:
 app.control.revoke(game.kick_out_task_id, terminate=True)
 game.kick_out_task_id = None

 game.save()

@staticmethod

```

```

def _set_auction_timer(game: Game):
 """
 Cancels any existing auction timer and schedules a new one to end the auction.
 Also schedules bot play tasks with random delays for all bots currently in the game.

 Args:
 game (Game): The current game instance.

 Returns:
 None
 """
 import random
 from .tasks import auction_callback, bot_play_callback

 GameManager._cancel_auction_timer(game)
 task = auction_callback.apply_async(args=[game.pk], countdown=20)
 game.auction_task_id = task.id
 game.save()

 for player in game.players.all():
 if Bot.objects.filter(pk=player.pk).exists():
 bot_play_callback.apply_async(args=[game.pk, player.pk], countdown=random.randint(8, 12))

 @staticmethod
 def _cancel_auction_timer(game: Game):
 """
 Revokes the active auction Celery task if it exists and is not the current task.
 Clears the auction task ID from the game.

 Args:
 game (Game): The current game instance.

 Returns:
 None
 """
 from .celery import app
 from celery import current_task

 current_task_id = None
 if current_task and hasattr(current_task, 'request'):
 current_task_id = getattr(current_task.request, 'id', None)

 if game.auction_task_id:
 if game.auction_task_id != current_task_id:
 app.control.revoke(game.auction_task_id, terminate=True)
 game.auction_task_id = None
 game.save()

```

`process_action(game, user, action)` `classmethod`

The only public method exposed in the API. It processes each action in dedicated functions depending on the current phase and returns a Response.

**Parameters:**

| Name   | Type                       | Description                       | Default         |
|--------|----------------------------|-----------------------------------|-----------------|
| game   | <a href="#">Game</a>       | The current active game instance. | <i>required</i> |
| user   | <a href="#">CustomUser</a> | The user performing the action.   | <i>required</i> |
| action | <a href="#">Action</a>     | The deserialized action payload.  | <i>required</i> |

**Returns:**

| Name     | Type                     | Description                                 |
|----------|--------------------------|---------------------------------------------|
| Response | <a href="#">Response</a> | A response object representing the outcome. |

**Raises:**

| Type                               | Description                              |
|------------------------------------|------------------------------------------|
| <a href="#">MaliciousUserInput</a> | If the user acts out of turn or phase.   |
| <a href="#">GameLogicError</a>     | If an unrecognized phase is encountered. |

**• Source code in** [magnate/games.py](#)

```
@classmethod
@database_sync_to_async
def process_action(cls, game: Game, user: CustomUser, action: Action) -> Response:
 """
 The only public method exposed in the API. It processes each action
 in dedicated functions depending on the current phase and returns
 a Response.

 Args:
 game (Game): The current active game instance.
 user (CustomUser): The user performing the action.
 action (Action): The deserialized action payload.

 Returns:
 Response: A response object representing the outcome.

 Raises:
 MaliciousUserInput: If the user acts out of turn or phase.
 GameLogicError: If an unrecognized phase is encountered.
 """

 if isinstance(action, ActionSurrender):
 cls._bankrupt_player(game, user)
 if game.phase == cls.END_GAME:
 return cls._end_game_logic(game, user, action)
 response = Response()
 return _add_basic_response_data(game, response)

 if user.pk != game.active_phase_player.pk and not isinstance(action, ActionBid): # if auction there are no turns
 raise MaliciousUserInput(user, "is not the active player")

 if game.phase == cls.ROLL_THE_DICES:
 if isinstance(action, ActionPayBail):
 response = cls._pay_bail_logic(game, user, action)
 elif isinstance(action, ActionThrowDices):
```

```
 response = cls._roll_dices_logic(game, user, action)
 else:
 raise MaliciousUserInputAction(game, user, action)
elif game.phase == cls.CHOOSE_SQUARE:
 if not isinstance(action, ActionMoveTo):
 raise MaliciousUserInputAction(game, user, action)
 response = cls._square_chosen_logic(game, user, action)
elif game.phase == cls.CHOOSE_FANTASY:
 if not isinstance(action, ActionChooseCard):
 raise MaliciousUserInputAction(game, user, action)
 response = cls._choose_fantasy_logic(game, user, action)
elif game.phase == cls.MANAGEMENT:
 if not isinstance(action, (ActionBuySquare, ActionDropPurchase, ActionTakeTram, ActionNextPhase)):
 raise MaliciousUserInputAction(game, user, action)
 response = cls._management_logic(game, user, action)
elif game.phase == cls.BUSINESS or game.phase == cls.LIQUIDATION:
 if not isinstance(action, (ActionBuild, ActionDemolish, ActionTradeProposal, ActionMortgageSet, ActionMortgage)):
 raise MaliciousUserInputAction(game, user, action)
 response = cls._business_logic(game, user, action)
elif game.phase == cls.ANSWER_TRADE_PROPOSAL:
 if not isinstance(action, ActionTradeAnswer):
 raise MaliciousUserInputAction(game, user, action)
 response = cls._answer_trade_proposal_logic(game, user, action)
elif game.phase == cls.AUCTION:
 if not isinstance(action, ActionBid):
 raise MaliciousUserInputAction(game, user, action)
 response = cls._bid_property_auction_logic(game, user, action)
elif game.phase == cls.END_GAME:
 pass
 # en principio no check instance porque la partida ya ha acabado
 # response = cls._end_game_logic(game,user,action)
 # return cls._end_game_logic(game,user,action)
else:
 raise GameLogicError(f"Unrecognized or unhandled phase: {game.phase}")

if game.phase == cls.END_GAME:
 response = cls._end_game_logic(game, user, action)

return _add_basic_response_data(game, response)
```

`_pay_bail_logic(game, user, action)` `staticmethod`

Processes the action to pay the jail bail. Cancels previous timers, deducts money, resets jail turns, updates player statistics, and sets the timer for the next phase.

Parameters:

| Name   | Type                          | Description                | Default         |
|--------|-------------------------------|----------------------------|-----------------|
| game   | <a href="#">Game</a>          | The current game instance. | <i>required</i> |
| user   | <a href="#">CustomUser</a>    | The jailed user.           | <i>required</i> |
| action | <a href="#">ActionPayBail</a> | The bail action payload.   | <i>required</i> |

**Returns:**

| Name | Type                     | Description            |
|------|--------------------------|------------------------|
|      | <a href="#">Response</a> | The standard response. |

**Raises:**

| Type                               | Description                             |
|------------------------------------|-----------------------------------------|
| <a href="#">MaliciousUserInput</a> | If the user is not actually in jail.    |
| <a href="#">GameLogicError</a>     | If the user does not have enough money. |

**• Source code in** [magnate/games.py](#)

```
@staticmethod
def _pay_bail_logic(game: Game, user: CustomUser, action: ActionPayBail) -> Response:
 """
 Processes the action to pay the jail bail. Cancels previous timers, deducts money, resets jail turns, updates player
 statistics, and sets the timer for the next phase.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The jailed user.
 action (ActionPayBail): The bail action payload.

 Returns:
 Response: The standard response.

 Raises:
 MaliciousUserInput: If the user is not actually in jail.
 GameLogicError: If the user does not have enough money.
 """

 GameManager._cancel_all_timers(game)

 response: ResponseChooseSquare = ResponseChooseSquare()

 square = _get_user_square(game, user).get_real_instance()

 if not isinstance(square, JailSquare):
 raise MaliciousUserInput(user, "is not in jail")

 remaining = game.jail_remaining_turns.get(str(user.pk), 0)
 if remaining == 0:
 raise MaliciousUserInput(user, "is not in jail (no turns remaining)")

 if len(game.possible_destinations) < 1:
 raise MaliciousUserInput(user, "cannot pay bail now")

 if not action.to_pay:
 # Remain in jail
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.turns_in_jail += 1
 stats.save()
 game.phase = GameManager.BUSINESS
 game.possible_destinations = dict()
 game.save()
 response.path = [int(square.custom_id)]
 GameManager._set_next_phase_timer(game, user) #TODO: revisar este timer
 return response
```

```
The player paid the bail

bail_price = square.bail_price

if game.money[str(user.pk)] < bail_price:
 raise GameLogicError("not enough money to pay bail")

dest_square_id = next(iter(game.possible_destinations))
steps = game.possible_destinations[dest_square_id] # pasos

move_result = _move_player_logic(square, steps)
response.path = move_result["path"]

game.money[str(user.pk)] -= bail_price
game.parking_money += bail_price
game.jail_remaining_turns[str(user.pk)] = 0
game.possible_destinations = dict()

game.positions[str(user.pk)] = int(dest_square_id)
square = _get_square_by_custom_id(dest_square_id)
_apply_square_arrival(game, user, response, square, move_result["passed_go"])

stats = PlayerGameStatistic.objects.get(user=user, game=game)
stats.lost_money += bail_price
GameManager._set_next_phase_timer(game, user)

stats.save()
game.save()
return response
```

`_roll_dices_logic(game, user, action)` `staticmethod`

Handles the dice rolling process, including double/triple streaks and jail mechanics.

This method calculates the result of rolling three dice (including the bus die), updates the player's movement streak, and determines possible destination squares. It also handles logic for players attempting to leave jail via doubles or forced payment.

Parameters:

| Name   | Type                          | Description                    | Default         |
|--------|-------------------------------|--------------------------------|-----------------|
| game   | <code>Game</code>             | The current game instance.     | <i>required</i> |
| user   | <code>CustomUser</code>       | The user throwing the dice.    | <i>required</i> |
| action | <code>ActionThrowDices</code> | The dice throw action payload. | <i>required</i> |

Returns:

| Name               | Type                  | Description                                                                                                              |
|--------------------|-----------------------|--------------------------------------------------------------------------------------------------------------------------|
| ResponseThrowDices | <code>Response</code> | Contains dice values, streak status, possible destinations, and the path to be traversed if only one destination exists. |

**Raises:**

| Type                           | Description                                                    |
|--------------------------------|----------------------------------------------------------------|
| <a href="#">GameLogicError</a> | If the player has a jailed status but is not on a jail square. |

- **Source code in** `magnate/games.py`

```
@staticmethod
def _roll_dices_logic(game: Game, user: CustomUser, action: ActionThrowDices) -> Response:
 """
 Handles the dice rolling process, including double/triple streaks and jail mechanics.

 This method calculates the result of rolling three dice (including the bus die),
 updates the player's movement streak, and determines possible destination squares.
 It also handles logic for players attempting to leave jail via doubles or
 forced payment.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user throwing the dice.
 action (ActionThrowDices): The dice throw action payload.

 Returns:
 ResponseThrowDices: Contains dice values, streak status, possible destinations,
 and the path to be traversed if only one destination exists.

 Raises:
 GameLogicError: If the player has a jailed status but is not on a jail square.
 """

 GameManager._cancel_all_timers(game)

 response: ResponseThrowDices = ResponseThrowDices()

 # jail logic
 remaining_jail_turns = game.jail_remaining_turns.get(str(user.pk), 0)
 is_jailed = remaining_jail_turns > 0
 was_jailed = is_jailed

 mock = None
 if (isinstance(game.possible_destinations, dict)
 and '__mock_dice__' in game.possible_destinations):
 mock = game.possible_destinations['__mock_dice__']
 game.possible_destinations = dict()

 if mock:
 d1, d2, d3 = mock[0], mock[1], mock[2]
 else:
 d1 = random.randint(1, 6)
 d2 = random.randint(1, 6)
 d3 = random.randint(1, 6) if not is_jailed else 0

 response.dice1, response.dice2, response.dice_bus = d1, d2, d3

 triples = d3 <= 3 and (d1 == d2 == d3)
 doubles = (d1 == d2) and not triples

 response.triple = triples
```

```

current_pos_square = _get_user_square(game, user).get_real_instance()
current_pos_id = current_pos_square.custom_id

jail logic
remaining_jail_turns = game.jail_remaining_turns.get(str(user.pk), 0)
is_jailed = remaining_jail_turns > 0
was_jailed = is_jailed

if is_jailed:
 jail_sq = current_pos_square
 if isinstance(jail_sq, JailSquare):
 if remaining_jail_turns == 1: # Forced to leave jail (last turn)
 game.money[str(user.pk)] -= jail_sq.bail_price
 game.jail_remaining_turns[str(user.pk)] = 0
 is_jailed = False
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.turns_in_jail += 1
 stats.lost_money += jail_sq.bail_price
 stats.save()
 elif doubles: # Leaves jail for free due to doubles
 game.jail_remaining_turns[str(user.pk)] = 0
 game.streak = 0
 is_jailed = False
 else:
 # stays in jail
 game.jail_remaining_turns[str(user.pk)] -= 1
 game.phase = GameManager.ROLL_THE_DICES
 game.save()
 else:
 raise GameLogicError(f"Cannot be in jail status and not in jail square")

if triples:
 # path current -> decided in chosen
 response.triple = True
 square = current_pos_square
 all_squares = BaseSquare.objects.filter(board=square.board)
 game.streak = 0
 response.streak = game.streak
 # All squares are suitable destinations
 possible_destinations = [s.custom_id for s in all_squares]
 possible_destinations.remove(_get_jail_square().custom_id)
 game.possible_destinations = {c_id: 0 for c_id in possible_destinations}
 response.destinations = [int(k) for k in game.possible_destinations.keys()]
 game.phase = GameManager.CHOOSE_SQUARE
 response.path = [current_pos_id]
 game.save()
 GameManager._set_next_phase_timer(game, user)
 return response

elif doubles: # doubles streak only if not getting out of jail via doubles
 if game.streak >= 2:
 # path -> current and jail
 jail_square = _get_jail_square()
 response.destinations = [jail_square.custom_id]
 game.streak = 0
 response.streak = game.streak

 game.positions[str(user.pk)] = int(jail_square.custom_id)
 game.jail_remaining_turns[str(user.pk)] = 3

```



```

 response.path = [current_pos_id, jail_square.custom_id]

 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.times_in_jail += 1
 stats.save()

 game.phase = GameManager.LIQUIDATION

 game.save()
 GameManager._set_next_phase_timer(game, user)
 return response
 elif not was_jailed:
 game.streak = game.streak + 1
else:
 game.streak = 0

response.streak = game.streak

dice_combinations = _compute_dice_combinations(d1, d2, d3) #ya con el bus cancelado si se está en la cárcel
game.possible_destinations, passed_go_map = _get_possible_destinations_ids(game, user, dice_combinations)

response.destinations = [int(k) for k in game.possible_destinations.keys()]

if is_jailed:
 # Now it should decide whether to pay bail or not
 if int(current_pos_id) not in response.destinations:
 response.destinations.append(int(current_pos_id))
 response.path = []
elif len(game.possible_destinations) > 1:
 # path in square chosen logic
 game.phase = GameManager.CHOOSE_SQUARE
 response.path = [current_pos_id]
else:
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.walked_squares += dice_combinations[0]
 stats.save()
 dest_square_id = next(iter(game.possible_destinations))
 steps = game.possible_destinations[dest_square_id]

 # Use _move_player_logic to get the traversed path and check for "Go to Jail" or "Passing Go".
 move_result = _move_player_logic(current_pos_square, steps)
 response.path = move_result["path"]

 if move_result["jailed"]:
 # landing in go to jail; update state to jail the player.
 jail = JailSquare.objects.first()
 if jail is None:
 raise GameDesignError('no jail in game')

 game.positions[str(user.pk)] = int(jail.custom_id)
 game.jail_remaining_turns[str(user.pk)] = 3
 game.phase = GameManager.LIQUIDATION
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.times_in_jail += 1
 stats.save()
 else:
 game.positions[str(user.pk)] = int(dest_square_id)
 square = _get_square_by_custom_id(dest_square_id)
 _apply_square_arrival(game, user, response, square, move_result["passed_go"])

```

```
game.possible_destinations = dict()

game.save()
GameManager._set_next_phase_timer(game, user)
return response
```

```
_square_chosen_logic(game, user, action) staticmethod
```

Handles the user's final destination selection. Moves the player, evaluates passing Go, updates walked squares statistics, and resets timers.

Parameters:

| Name   | Type         | Description                          | Default  |
|--------|--------------|--------------------------------------|----------|
| game   | Game         | The current game instance.           | required |
| user   | CustomUser   | The current user.                    | required |
| action | ActionMoveTo | Action indicating the chosen square. | required |

Returns:

| Name     | Type     | Description                                             |
|----------|----------|---------------------------------------------------------|
| Response | Response | A ResponseChooseSquare object including the path taken. |

Raises:

| Type                     | Description                                                       |
|--------------------------|-------------------------------------------------------------------|
| MaliciousUserInputAction | If the action type is incorrect.                                  |
| MaliciousUserInput       | If the chosen square is not in the allowed possible_destinations. |
| GameDesignError          | If the jail square is missing from the board.                     |

• Source code in magnate/games.py

```
@staticmethod
def _square_chosen_logic(game: Game, user: CustomUser, action: Action) -> Response:
 """
 Handles the user's final destination selection.
 Moves the player, evaluates passing Go, updates walked squares statistics,
 and resets timers.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The current user.
 action (ActionMoveTo): Action indicating the chosen square.

 Returns:
 Response: A ResponseChooseSquare object including the path taken.

 Raises:
 MaliciousUserInputAction: If the action type is incorrect.
 MaliciousUserInput: If the chosen square is not in the allowed `possible_destinations`.
 GameDesignError: If the jail square is missing from the board.
 """

 GameManager._cancel_all_timers(game)
```

```

response: ResponseChooseSquare = ResponseChooseSquare()

if not isinstance(action, ActionMoveTo):
 raise MaliciousUserInputAction(game, user, action)

square = action.square

if str(square.custom_id) not in game.possible_destinations:
 raise MaliciousUserInput(user, "tried to move to an illegal square")

current_pos_id = game.positions[str(user.pk)]
current_pos_square = _get_square_by_custom_id(current_pos_id).get_real_instance()

steps = game.possible_destinations.get(str(square.custom_id))

if steps == 0: #triples
 move_result = {
 "jailed": isinstance(square.get_real_instance(), GoToJailSquare),
 "passed_go": False,
 "path": [current_pos_id, square.custom_id]
 }
else:
 move_result = _move_player_logic(current_pos_square, steps)

response.path = move_result["path"]

if move_result["jailed"]:
 # landing in go to jail; update state to jail the player.
 jail = JailSquare.objects.first()
 if jail is None:
 raise GameDesignError('no jail in game')

 game.positions[str(user.pk)] = int(jail.custom_id)
 game.jail_remaining_turns[str(user.pk)] = 3
 game.phase = GameManager.LIQUIDATION
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.times_in_jail += 1
 stats.save()
else:
 game.positions[str(user.pk)] = int(square.custom_id)
 square = _get_square_by_custom_id(square.custom_id)
 _apply_square_arrival(game, user, response, square, move_result["passed_go"])

stats = PlayerGameStatistic.objects.get(user=user, game=game)
stats.walked_squares += steps
stats.save()

game.possible_destinations = dict()
game.save()

GameManager._set_next_phase_timer(game, user)
return response

```

`_choose_fantasy_logic(game, user, action)` staticmethod

Applies a pre-existing fantasy event or randomly generates a new one. Determines the next game phase and updates timers.

**Parameters:**

| Name   | Type                             | Description                            | Default         |
|--------|----------------------------------|----------------------------------------|-----------------|
| game   | <a href="#">Game</a>             | The current game instance.             | <i>required</i> |
| user   | <a href="#">CustomUser</a>       | The active user making the choice.     | <i>required</i> |
| action | <a href="#">ActionChooseCard</a> | The payload of the card choice action. | <i>required</i> |

**Returns:**

| Name     | Type                     | Description                                                         |
|----------|--------------------------|---------------------------------------------------------------------|
| Response | <a href="#">Response</a> | A ResponseChooseFantasy object containing the fantasy event result. |

**• Source code in** [magnate/games.py](#)

```
@staticmethod
def _choose_fantasy_logic(game: Game, user: CustomUser, action: ActionChooseCard) -> Response:
 """
 Applies a pre-existing fantasy event or randomly generates a new one.
 Determines the next game phase and updates timers.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The active user making the choice.
 action (ActionChooseCard): The payload of the card choice action.

 Returns:
 Response: A ResponseChooseFantasy object containing the fantasy event result.
 """

 GameManager._cancel_all_timers(game)

 response = ResponseChooseFantasy()
 fantasy_event = game.fantasy_event
 generate = not action.chosen_revealed_card
 new_fantasy = None

 if not generate:
 card_cost = fantasy_event.card_cost
 if game.money[str(user.pk)] < card_cost:
 raise MaliciousUserInput(user, "not enough money to pay for the revealed card")

 game.money[str(user.pk)] -= card_cost
 game.parking_money += card_cost
 game.money = dict(game.money) # to update
 game.save(update_fields=['money', 'parking_money'])
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 stats.lost_money += card_cost
 stats.save()

 fantasy_result = apply_fantasy_event(game, user, fantasy_event)
 fantasy_result.save()
 response.fantasy_result = fantasy_result
 game.fantasy_event = None
 game.save()

 else:
 new_fantasy = FantasyEventFactory.generate()
 new_fantasy.save()
```

```
 fantasy_result = apply_fantasy_event(game, user, new_fantasy)
 fantasy_result.save()
 response.fantasy_result = fantasy_result
 game.fantasy_event = None
 game.save()

 if game.streak == 0:
 if game.money[str(user.pk)] < 0:
 game.phase = GameManager.LIQUIDATION
 else:
 game.phase = GameManager.BUSINESS
 else:
 game.phase = GameManager.ROLL_THE_DICES

 if game.phase == GameManager.BUSINESS or game.phase == GameManager.LIQUIDATION:
 GameManager._set_next_phase_timer(game, user)
 elif game.phase == GameManager.ROLL_THE_DICES:
 GameManager._set_kick_out_timer(game, user)
 pass

 game.save()
 return response
```

`_management_logic(game, user, action)` `staticmethod`

Management phase logic (purchases, dropping a purchase, taking a tram). Includes handling purchases for PropertySquare, ServerSquare, and BridgeSquare, updating lost money statistics and timers.

Parameters:

| Name                | Type                       | Description                                                              | Default         |
|---------------------|----------------------------|--------------------------------------------------------------------------|-----------------|
| <code>game</code>   | <a href="#">Game</a>       | The current game instance.                                               | <i>required</i> |
| <code>user</code>   | <a href="#">CustomUser</a> | The acting user.                                                         | <i>required</i> |
| <code>action</code> | <a href="#">Action</a>     | The payload of the action taken (e.g., ActionBuySquare, ActionTakeTram). | <i>required</i> |

Returns:

| Name                  | Type                     | Description        |
|-----------------------|--------------------------|--------------------|
| <code>Response</code> | <a href="#">Response</a> | Standard response. |

Raises:

| Type                                     | Description                                                           |
|------------------------------------------|-----------------------------------------------------------------------|
| <a href="#">MaliciousUserInputAction</a> | If the action does not fit the phase, context, or square type.        |
| <a href="#">MaliciousUserInput</a>       | If the user tries to take a tram to an invalid square or lacks funds. |

• Source code in `magnate/games.py`

```
@staticmethod
def _management_logic(game: Game, user: CustomUser, action: Action) -> Response:
 """
 Management phase logic (purchases, dropping a purchase, taking a tram).
 Includes handling purchases for PropertySquare, ServerSquare, and BridgeSquare,
 updating lost money statistics and timers.

 Args:
```

game (Game): The current game instance.  
 user (CustomUser): The acting user.  
 action (Action): The payload of the action taken (e.g., ActionBuySquare, ActionTakeTram).

Returns:

Response: Standard response.

Raises:

MaliciousUserInputAction: If the action does not fit the phase, context, or square type.  
 MaliciousUserInput: If the user tries to take a tram to an invalid square or lacks funds.

"""

GameManager.\_cancel\_all\_timers(game)

current\_square = \_get\_user\_square(game, user).get\_real\_instance()

prop\_rel = \_get\_relationship(game, current\_square)

if isinstance(action, ActionBuySquare):

if not isinstance(current\_square, (PropertySquare, ServerSquare, BridgeSquare)):  
 raise MaliciousUserInputAction(game, user, action)

\_handle\_property\_acquisition(game, user, current\_square)  
 stats = PlayerGameStatistic.objects.get(user=user, game=game)  
 stats.lost\_money += current\_square.buy\_price  
 stats.save()  
 game.money[str(user.pk)] -= current\_square.buy\_price

elif isinstance(action, ActionDropPurchase):

if isinstance(action.square, (PropertySquare, ServerSquare, BridgeSquare)):  
 return GameManager.\_initiate\_auction(game, action.square, user)

else:

raise MaliciousUserInputAction(game, user, action)

elif isinstance(action, ActionTakeTram):

if isinstance(current\_square, TramSquare):  
 square = action.square  
 tram\_squares = TramSquare.objects.filter()  
 tram\_square\_actual\_id = game.positions[str(user.pk)]  
 tram\_squares\_extern\_ids = [s.custom\_id for s in tram\_squares]

if square.custom\_id == tram\_square\_actual\_id: # case stay in the same square, free  
 pass

elif square.custom\_id in tram\_squares\_extern\_ids: # Move to another square

if game.money[str(user.pk)] < square.buy\_price:  
 raise MaliciousUserInput(user, "does not have enough money to take tram")  
 game.money[str(user.pk)] -= square.buy\_price  
 stats = PlayerGameStatistic.objects.get(user=user, game=game)  
 stats.lost\_money += square.buy\_price  
 stats.save()  
 game.positions[str(user.pk)] = int(square.custom\_id)

else:

raise MaliciousUserInput(user, "tried to take a tram to a non tram square")

else:

raise MaliciousUserInputAction(game, user, action)

elif isinstance(action, ActionNextPhase):

# TODO: Remove?

pass

else:

raise MaliciousUserInputAction(game, user, action)

if game.phase == GameManager.MANAGEMENT:

```
 if game.streak == 0:
 game.phase = GameManager.BUSINESS
 else:
 game.phase = GameManager.ROLL_THE_DICES

 if game.phase == GameManager.BUSINESS:
 GameManager._set_next_phase_timer(game, user)
 elif game.phase == GameManager.ROLL_THE_DICES:
 GameManager._set_kick_out_timer(game, user)
 pass

 game.save()
 return Response()
```

`_business_logic(game, user, action)``staticmethod`

Unifies the business and liquidation phases. Handles building, demolishing, trading, mortgaging, and proceeding to the next turn.

Parameters:

| Name                | Type                    | Description                      | Default         |
|---------------------|-------------------------|----------------------------------|-----------------|
| <code>game</code>   | <code>Game</code>       | The current game instance.       | <i>required</i> |
| <code>user</code>   | <code>CustomUser</code> | The active user.                 | <i>required</i> |
| <code>action</code> | <code>Action</code>     | The payload of the action taken. | <i>required</i> |

Returns:

| Name                  | Type                  | Description        |
|-----------------------|-----------------------|--------------------|
| <code>Response</code> | <code>Response</code> | Standard response. |

Raises:

| Type                            | Description                                                      |
|---------------------------------|------------------------------------------------------------------|
| <code>MaliciousUserInput</code> | If restrictions (e.g., negative balance on next turn) are unmet. |

• Source code in `magnate/games.py`

```
@staticmethod
def _business_logic(game: Game, user: CustomUser, action: Action) -> Response:
 """
 Unifies the business and liquidation phases. Handles building, demolishing,
 trading, mortgaging, and proceeding to the next turn.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The active user.
 action (Action): The payload of the action taken.

 Returns:
 Response: Standard response.

 Raises:
 MaliciousUserInput: If restrictions (e.g., negative balance on next turn) are unmet.
 """
```

```

GameManager._cancel_all_timers(game)
Logic for the business phase where players can build houses, trade, etc.
if isinstance(action, ActionBuild):
 # Check owner
 building_square = action.square
 relationship = _build_square(game, user, building_square, action.houses, False)

elif isinstance(action, ActionDemolish):
 # Similiar to build
 demolition_square = action.square
 relationship = _demolish_square(game, user, demolition_square, action.houses, False)

elif isinstance(action, ActionTradeProposal):
 relationship = GameManager._propose_trade(game, user, action)
 return Response()

elif isinstance(action, ActionMortgageSet):
 relationship = _set_mortgage(game, user, action.square, False)

elif isinstance(action, ActionMortgageUnset):
 relationship = _unset_mortgage(game, user, action.square, False)
elif isinstance(action, ActionNextPhase):
 current_money = game.money[str(user.pk)]

 if game.phase == GameManager.BUSINESS:
 if current_money >= 0:
 GameManager._next_turn(game, user)
 else:
 game.phase = GameManager.LIQUIDATION
 game.save()

 elif game.phase == GameManager.LIQUIDATION:
 if current_money >= 0:
 GameManager._next_turn(game, user)
 else:
 raise MaliciousUserInput(user, "Cannot end in NEGATIVE")
 return Response() # timers set in next turn

GameManager._set_next_phase_timer(game, user)
return Response()

```

`_answer_trade_proposal_logic(game, user, action)` `staticmethod`

Processes the response (accept/reject) to a trade proposal. Transfers properties and money ensuring no player goes into negative balance, updates trade and money statistics for both players, and returns the turn to the proposer.

#### Parameters:

| Name   | Type                              | Description                                                         | Default         |
|--------|-----------------------------------|---------------------------------------------------------------------|-----------------|
| game   | <a href="#">Game</a>              | The current game instance.                                          | <i>required</i> |
| user   | <a href="#">CustomUser</a>        | The targeted user responding to the offer.                          | <i>required</i> |
| action | <a href="#">ActionTradeAnswer</a> | The user's response payload (contains <code>choose</code> boolean). | <i>required</i> |



**Returns:**

| Name | Type                     | Description        |
|------|--------------------------|--------------------|
|      | <a href="#">Response</a> | Standard response. |

**Raises:**

| Type                                     | Description                                                                                                       |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <a href="#">GameLogicError</a>           | If there is no active proposal in the game state.                                                                 |
| <a href="#">MaliciousUserInput</a>       | If an unauthorized user attempts to answer, or if the trade results in a negative money balance for either party. |
| <a href="#">MaliciousUserInputAction</a> | If an invalid action type is passed.                                                                              |

**• Source code in** [magnate/games.py](#)

```
@staticmethod
def _answer_trade_proposal_logic(game: Game, user: CustomUser, action: Action) -> Response:
 """
 Processes the response (accept/reject) to a trade proposal.
 Transfers properties and money ensuring no player goes into negative balance,
 updates trade and money statistics for both players, and returns the turn
 to the proposer.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The targeted user responding to the offer.
 action (ActionTradeAnswer): The user's response payload (contains `choose` boolean).

 Returns:
 Response: Standard response.

 Raises:
 GameLogicError: If there is no active proposal in the game state.
 MaliciousUserInput: If an unauthorized user attempts to answer,
 or if the trade results in a negative money balance for either party.
 MaliciousUserInputAction: If an invalid action type is passed.
 """
 if isinstance(action, ActionTradeAnswer):
 offer = game.proposal

 if offer is None:
 raise GameLogicError("there should be an active trade proposal")

 if user.pk != offer.destination_user.pk:
 raise MaliciousUserInput(user, f"cannot accept proposal {offer}")

 offering = offer.player

 offered_money = offer.offered_money
 asked_money = offer.asked_money
 offered_properties = offer.offered_properties
 asked_properties = offer.asked_properties

 if action.choose:
 # Money logic (ensure no player goes to negative)
 money_diff = offered_money - asked_money
 acceptor_money = game.money[str(user.pk)] + money_diff
 offering_money = game.money[str(offering.pk)] - money_diff
```

```

 if acceptor_money < 0:
 raise MaliciousUserInput(user, f"cannot go to negative in trade")
 if offering_money < 0:
 raise MaliciousUserInput(offering, f"cannot go to negative in trade")

 for relationship in offered_properties.all():
 _handle_property_trade(game, user, relationship.square)

 for relationship in asked_properties.all():
 _handle_property_trade(game, offering, relationship.square)

 game.money[str(offering.pk)] = offering_money
 stats = PlayerGameStatistic.objects.get(user=offering, game=game)
 if money_diff > 0:
 stats.lost_money += abs(money_diff)
 else:
 stats.won_money += abs(money_diff)
 stats.num_trades += 1
 stats.save()

 game.money[str(user.pk)] = acceptor_money
 stats = PlayerGameStatistic.objects.get(user=user, game=game)
 if money_diff > 0:
 stats.won_money += abs(money_diff)
 else:
 stats.lost_money += abs(money_diff)
 stats.num_trades += 1
 stats.save()

 game.save()

 game.phase = GameManager.BUSINESS
 game.active_phase_player = offering
 game.proposal = None
 game.save()
 GameManager._set_next_phase_timer(game, offering)

 return Response()
else:
 raise MaliciousUserInputAction(game, user, action)

```

`_initiate_auction(game, square, player)` staticmethod

Transitions the game into an AUCTION phase for a dropped property.

#### Parameters:

| Name   | Type                       | Description                               | Default         |
|--------|----------------------------|-------------------------------------------|-----------------|
| game   | <a href="#">Game</a>       | The current game instance.                | <i>required</i> |
| square | <a href="#">BaseSquare</a> | The property square going up for auction. | <i>required</i> |

#### Returns:

| Name     | Type                     | Description        |
|----------|--------------------------|--------------------|
| Response | <a href="#">Response</a> | Standard response. |

• Source code in `magnate/games.py`

```
@staticmethod
def _initiate_auction(game: Game, square: BaseSquare, player: CustomUser) -> Response:
 """
 Transitions the game into an AUCTION phase for a dropped property.

 Args:
 game (Game): The current game instance.
 square (BaseSquare): The property square going up for auction.

 Returns:
 Response: Standard response.
 """
 GameManager._cancel_all_timers(game)

 ActionDropPurchase.objects.get_or_create(game=game, player=player, square=square)

 game.next_phase_task_id = None
 game.phase = GameManager.AUCTION

 auction = Auction.objects.create(game=game, square=square, is_active=True, bids = {})
 game.current_auction = auction
 game.save()

 # TODO: Remove magic number
 GameManager._set_auction_timer(game)
 # TODO: Remove in production
 game.refresh_from_db()

 return Response()
```

`_bid_property_auction_logic(game, user, action)` `staticmethod`

Registers a player's bid in an active auction. Verifies that the player has not previously dropped the purchase of that property. Automatically ends the auction if all eligible players have bid.

**Parameters:**

| Name                | Type                    | Description                         | Default         |
|---------------------|-------------------------|-------------------------------------|-----------------|
| <code>game</code>   | <code>Game</code>       | The current game instance.          | <i>required</i> |
| <code>user</code>   | <code>CustomUser</code> | The user placing the bid.           | <i>required</i> |
| <code>action</code> | <code>ActionBid</code>  | The action carrying the bid amount. | <i>required</i> |

**Returns:**

| Name                  | Type                  | Description                                                                  |
|-----------------------|-----------------------|------------------------------------------------------------------------------|
| <code>Response</code> | <code>Response</code> | Standard response, or the resolved auction response if all players have bid. |

**Raises:**

| Type                                     | Description                                                                                                 |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <a href="#">GameLogicError</a>           | If there is no active auction found.                                                                        |
| <a href="#">MaliciousUserInputAction</a> | If an invalid action type is passed.                                                                        |
| <a href="#">MaliciousUserInput</a>       | If the user bids twice, triggered the auction drop themselves, or if the bid exceeds their current balance. |

• Source code in `magnate/games.py`

```
@staticmethod
def _bid_property_auction_logic(game: Game, user: CustomUser, action: Action) -> Response:
 """
 Registers a player's bid in an active auction.
 Verifies that the player has not previously dropped the purchase of that property.
 Automatically ends the auction if all eligible players have bid.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user placing the bid.
 action (ActionBid): The action carrying the bid amount.

 Returns:
 Response: Standard response, or the resolved auction response if all players have bid.

 Raises:
 GameLogicError: If there is no active auction found.
 MaliciousUserInputAction: If an invalid action type is passed.
 MaliciousUserInput: If the user bids twice, triggered the auction drop themselves,
 or if the bid exceeds their current balance.
 """
 if not isinstance(action, ActionBid):
 raise MaliciousUserInputAction(game, user, action)

 auction = game.current_auction
 if not auction:
 raise GameLogicError("No active auction")

 bids = auction.bids
 # user has not bid yet
 if bids.get(str(user.pk)):
 raise MaliciousUserInput(user, "User already placed a bid in this auction")

 # user who started the bid cant bid
 dropped = ActionDropPurchase.objects.filter(game=game, player=user, square=auction.square).exists()
 if dropped:
 raise MaliciousUserInput(user, "cannot bid in an auction they triggered")

 # user has enough money -> compulsory for auctions
 amount = action.amount
 if amount > game.money[str(user.pk)]:
 raise MaliciousUserInput(user, "Bid amount exceeds current balance")

 bids[str(user.pk)] = amount
 auction.bids = bids
 auction.save()
```

```
game.save()

eligible_players_count = game.players.exclude(
 pk__in=ActionDropPurchase.objects.filter(game=game, square=auction.square).values('player')
).count()

if len(bids) >= eligible_players_count:
 # all bidded
 GameManager._cancel_auction_timer(game)
 return GameManager._end_auction(game) #type:ignore

return Response()
```

`_end_auction(game)` `staticmethod`

Ends an active auction, resolves the winner based on the highest bid, handles ties, and transitions the game state back to BUSINESS.

Parameters:

| Name | Type                 | Description                | Default         |
|------|----------------------|----------------------------|-----------------|
| game | <a href="#">Game</a> | The current game instance. | <i>required</i> |

Returns:

| Name    | Type                            | Description                   |
|---------|---------------------------------|-------------------------------|
| Auction | <a href="#">Response</a>   None | The finalized auction object. |

Raises:

| Type                           | Description                                            |
|--------------------------------|--------------------------------------------------------|
| <a href="#">GameLogicError</a> | If not in AUCTION phase or state is missing square ID. |

• Source code in `magnate/games.py`

```
@staticmethod
def _end_auction(game: Game) -> Response | None:
 """
 Ends an active auction, resolves the winner based on the highest bid,
 handles ties, and transitions the game state back to BUSINESS.

 Args:
 game (Game): The current game instance.

 Returns:
 Auction: The finalized auction object.

 Raises:
 GameLogicError: If not in AUCTION phase or state is missing square ID.
 """
 ## call this with a timer -> end of the auction

 if game.phase == GameManager.END_GAME:
 return None # Inevitable if the auction callback is triggered after the game has ended, just ignore it

 if game.phase != GameManager.AUCTION:
```

```

 raise GameLogicError("Tried to end auction but game is not in auction phase")

 auction = game.current_auction
 if not auction:
 raise GameLogicError("Game is in AUCTION phase but no auction object found")

 square = auction.square.get_real_instance()
 bids = auction.bids

 # no one bid
 if not bids:
 if game.streak == 0:
 game.phase = GameManager.BUSINESS
 else:
 game.phase = GameManager.ROLL_THE_DICES

 auction.is_active = False
 auction.winner = None
 auction.final_amount = 0
 auction.is_tie = False
 auction.save()

 game.current_auction = None
 game.save()

 if game.phase == GameManager.BUSINESS:
 GameManager._set_next_phase_timer(game, game.active_turn_player)
 elif game.phase == GameManager.ROLL_THE_DICES:
 GameManager._set_kick_out_timer(game, game.active_turn_player)
 pass

 game.save()
 return ResponseAuction(auction=auction)

 max_bid_amount = max(bids.values())
 winners = [int(uid) for uid, amt in bids.items() if amt == max_bid_amount]

 if len(winners) > 1 or max_bid_amount == 0:
 if game.streak == 0:
 game.phase = GameManager.BUSINESS
 else:
 game.phase = GameManager.ROLL_THE_DICES

 auction.is_active = False
 auction.winner = None
 auction.final_amount = max_bid_amount

 auction.is_tie = len(winners) > 1
 auction.save()

 game.current_auction = None
 game.save()

 if game.phase == GameManager.BUSINESS:
 GameManager._set_next_phase_timer(game, game.active_turn_player)
 elif game.phase == GameManager.ROLL_THE_DICES:
 GameManager._set_kick_out_timer(game, game.active_turn_player)
 pass

```

```

 game.save()
 return ResponseAuction(auction=auction)

winner_id = winners[0]
if str(winner_id) not in game.money: # surrenders mid auction -> no winner
 game.phase = GameManager.BUSINESS if game.streak == 0 else GameManager.ROLL_THE_DICES
 auction.is_active = False
 auction.save()

 game.current_auction = None
 game.save()

 if game.phase == GameManager.BUSINESS:
 GameManager._set_next_phase_timer(game, game.active_turn_player)
 elif game.phase == GameManager.ROLL_THE_DICES:
 GameManager._set_kick_out_timer(game, game.active_turn_player)
 pass

 game.save()
 return ResponseAuction(auction=auction)

winner = CustomUser.objects.get(pk=winner_id)
highest_bid = max_bid_amount

game.money[str(winner.pk)] -= highest_bid
stats = PlayerGameStatistic.objects.get(user=winner, game=game)
stats.lost_money += highest_bid
stats.save()

_handle_property_acquisition(game, winner, square)

auction.winner = winner
auction.final_amount = highest_bid
auction.is_active = False
auction.is_tie = False
auction.save()

if game.streak == 0:
 game.phase = GameManager.BUSINESS
else:
 game.phase = GameManager.ROLL_THE_DICES

game.current_auction = None
game.save()

if game.phase == GameManager.BUSINESS:
 GameManager._set_next_phase_timer(game, game.active_turn_player)
elif game.phase == GameManager.ROLL_THE_DICES:
 GameManager._set_kick_out_timer(game, game.active_turn_player)
 pass

game.save()

return ResponseAuction(auction=auction)

```

`_next_turn(game, user)` `classmethod`

Advances the game to the next player in the established order. Resets bot flags and manages timers for the new turn.

#### Parameters:

| Name              | Type                    | Description                                   | Default         |
|-------------------|-------------------------|-----------------------------------------------|-----------------|
| <code>game</code> | <code>Game</code>       | The current game instance.                    | <i>required</i> |
| <code>user</code> | <code>CustomUser</code> | The current active user whose turn is ending. | <i>required</i> |

#### Returns:

#### Type Description

`None` `None`

#### Raises:

#### Type Description

`GameLogicError` If the current or next player cannot be identified in the player list.

#### • Source code in `magnate/games.py`

```
@classmethod
def _next_turn(cls, game: Game, user: CustomUser) -> None:
 """
 Advances the game to the next player in the established order.
 Resets bot flags and manages timers for the new turn.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The current active user whose turn is ending.

 Returns:
 None

 Raises:
 GameLogicError: If the current or next player cannot be identified in the player list.
 """

 players_list = list(game.players.all())
 num_players = len(players_list)
 current_index = -1
 current_player_id = -1
 for p in players_list:
 if p.pk == game.active_turn_player.pk:
 current_player_id = p.pk
 current_index = game.ordered_players.index(current_player_id)
 break

 if current_index == -1:
 raise GameLogicError('current player not found')

 next_index = (current_index + 1) % num_players
 if next_index <= current_index:
 game.current_round += 1

 # The next active user is for both: phase and turn
```



```
next_player = game.players.filter(pk=game.ordered_players[next_index]).first()
if next_player is None:
 raise GameLogicError('next player is None')

if game.current_round > game.max_rounds:
 game.phase = GameManager.END_GAME
 GameManager._cancel_all_timers(game)
 if game.current_auction:
 game.current_auction.is_active = False
 game.current_auction.save()
 game.current_auction = None
 game.save()

#cls._end_game_logic(game, next_player) #####
return

game.active_phase_player = next_player
game.active_turn_player = next_player
game.phase = GameManager.ROLL_THE_DICES
game.current_turn += 1
game.save()

bot_next = Bot.objects.filter(pk=next_player.pk).first()
if bot_next:
 bot_next.has_proposed_trade = False
 bot_next.save()

GameManager._cancel_all_timers(game)

GameManager._set_kick_out_timer(game, next_player)
pass

game.save()
```

`_propose_trade(game, user, action)` `classmethod`

Validates and initiates a trade proposal between players. Checks that there are no built houses in the involved groups. Updates bot states if applicable and changes the phase to PROPOSAL\_ACCEPTANCE.

Parameters:

| Name   | Type                             | Description                                               | Default         |
|--------|----------------------------------|-----------------------------------------------------------|-----------------|
| game   | <code>Game</code>                | The current game instance.                                | <i>required</i> |
| user   | <code>CustomUser</code>          | The user proposing the trade.                             | <i>required</i> |
| action | <code>ActionTradeProposal</code> | The payload detailing offered/asked money and properties. | <i>required</i> |

Returns:

Type Description

|                   |      |
|-------------------|------|
| <code>None</code> | None |
|-------------------|------|

**Raises:**

| Type                               | Description                                                                                                                                                                                 |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">MaliciousUserInput</a> | If the action involves negative money, references an invalid destination player, attempts to trade unowned properties, or tries to trade properties belonging to a group with built houses. |

- Source code in `magnate/games.py`

```
@classmethod
def _propose_trade(cls, game: Game, user: CustomUser, action: ActionTradeProposal) -> None:
 """
 Validates and initiates a trade proposal between players.
 Checks that there are no built houses in the involved groups.
 Updates bot states if applicable and changes the phase to PROPOSAL_ACCEPTANCE.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user proposing the trade.
 action (ActionTradeProposal): The payload detailing offered/asked money and properties.

 Returns:
 None

 Raises:
 MaliciousUserInput: If the action involves negative money, references an invalid
 destination player, attempts to trade unowned properties, or tries
 to trade properties belonging to a group with built houses.
 """
 if action.player.pk != user.pk or action.offered_money < 0 or action.asked_money < 0:
 raise MaliciousUserInput(user, "cannot do operation")
 if action.destination_user.pk not in game.ordered_players:
 # FIXME: Change to internal order so that it handles player change to AI
 raise MaliciousUserInput(user, "referenced a player that is not in game")

 asked_properties_list = action.asked_properties.all()
 asked_count = PropertyRelationship.objects.filter(
 game=game,
 owner=action.destination_user,
 id__in=action.asked_properties.all()
).count()

 if asked_count != asked_properties_list.count():
 raise MaliciousUserInput(user, "destination does not have enough properties")

 offered_properties_list = action.offered_properties.all()
 offered_count = PropertyRelationship.objects.filter(
 game=game,
 owner=action.player,
 id__in=offered_properties_list
).count()

 if offered_count != offered_properties_list.count():
 raise MaliciousUserInput(user, f"offer does not have enough properties: player owns {offered_count} but tried")

 all_trade_properties = list(asked_properties_list) + list(offered_properties_list)
 for rel in all_trade_properties:
 real_sq = rel.square.get_real_instance()
 if isinstance(real_sq, PropertySquare):
 group_has_houses = PropertyRelationship.objects.filter(
```

```
 game=game,
 square__propertysquare__group=real_sq.group,
 houses__gt=0
).exists()
 if group_has_houses:
 raise MaliciousUserInput(user, "cannot trade properties from a group with constructions")

bot = Bot.objects.filter(pk=user.pk).first()
if bot:
 bot.has_proposed_trade = True
 bot.save()

game.phase = GameManager.PROPOSAL_ACCEPTANCE
game.active_phase_player = action.destination_user
game.proposal = action # type: ignore
game.save()
GameManager._set_next_phase_timer(game, action.destination_user)
```

`_bankrupt_player(game, user)`

classmethod

Eliminates a player from the game due to bankruptcy, clearing their properties and state. If only one player remains, ends the game. Otherwise, passes the turn to the next player.

Parameters:

| Name | Type                    | Description                | Default         |
|------|-------------------------|----------------------------|-----------------|
| game | <code>Game</code>       | The current game instance. | <i>required</i> |
| user | <code>CustomUser</code> | The user going bankrupt.   | <i>required</i> |

Returns:

Type Description

None

• Source code in `magnate/games.py`

```
@classmethod
def _bankrupt_player(cls, game: Game, user: CustomUser):
 """
 Eliminates a player from the game due to bankruptcy, clearing their properties
 and state. If only one player remains, ends the game. Otherwise, passes the turn
 to the next player.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user going bankrupt.

 Returns:
 None
 """
 try:
 current_idx = game.ordered_players.index(user.pk)
 next_pk = game.ordered_players[(current_idx + 1) % len(game.ordered_players)]
 next_player = CustomUser.objects.get(pk=next_pk)
 except ValueError:
```

```

 next_player = None

 if user.pk in game.ordered_players:
 game.ordered_players.remove(user.pk)

 game.players.remove(user)
 game.money.pop(str(user.pk), None)
 game.positions.pop(str(user.pk), None)
 game.jail_remaining_turns.pop(str(user.pk), None)

 PropertyRelationship.objects.filter(game=game, owner=user).delete()
 user.active_game = None
 user.save()

 if game.players.count() == 1:
 game.phase = GameManager.END_GAME
 GameManager._cancel_all_timers(game)

 if game.current_auction:
 auction = game.current_auction
 auction.is_active = False
 auction.save()
 game.current_auction = None

 game.save()
 return #endgame logic called afterwards

 from .models import Bot
 bots_in_game = Bot.objects.filter(id__in=game.players.values_list('id', flat=True))
 if game.players.count() == bots_in_game.count(): #solo quedan bots
 game.phase = GameManager.END_GAME
 GameManager._cancel_all_timers(game)

 if game.current_auction:
 auction = game.current_auction
 auction.is_active = False
 auction.save()
 game.current_auction = None

 game.save()
 return

 if game.active_turn_player.pk == user.pk and next_player:
 game.active_turn_player = next_player
 game.active_phase_player = next_player
 game.phase = GameManager.ROLL_THE_DICES
 game.streak = 0

 GameManager._cancel_all_timers(game)

 # new task
 GameManager._set_kick_out_timer(game, next_player)

 game.save()

```

`_apply_end_bonuses(game, num_bonuses=3)` classmethod

Selects random bonus categories and awards extra money to players based on their game statistics.

#### Parameters:

| Name                     | Type                 | Description                                                | Default         |
|--------------------------|----------------------|------------------------------------------------------------|-----------------|
| <code>game</code>        | <a href="#">Game</a> | The current game instance.                                 | <i>required</i> |
| <code>num_bonuses</code> | <code>int</code>     | The number of categories to pick for bonuses (default: 3). | <code>3</code>  |

#### Returns:

| Name                       | Type                          | Description                                                      |
|----------------------------|-------------------------------|------------------------------------------------------------------|
| <code>ResponseBonus</code> | <a href="#">ResponseBonus</a> | An object mapping the chosen bonus categories and their winners. |

#### • Source code in `magnate/games.py`

```
@classmethod
def _apply_end_bonuses(cls, game: Game, num_bonuses: int = 3) -> ResponseBonus:
 """
 Selects random bonus categories and awards extra money to players based on
 their game statistics.

 Args:
 game (Game): The current game instance.
 num_bonuses (int): The number of categories to pick for bonuses (default: 3).

 Returns:
 ResponseBonus: An object mapping the chosen bonus categories and their winners.
 """

 active_players = game.players.values_list('pk', flat=True)
 stats = PlayerGameStatistic.objects.filter(game=game, user__pk__in=active_players)

 valid_categories_data = []
 for category in BonusCategory.objects.all():
 field = category.stat_field
 max_value = stats.aggregate(Max(field)).get(f'{field}__max')
 if max_value and max_value > 0:
 valid_categories_data.append((category, max_value))

 chosen_data = random.sample(valid_categories_data, min(num_bonuses, len(valid_categories_data)))

 response = ResponseBonus()
 bonuses = {}

 for category, max_value in chosen_data:
 field = category.stat_field
 winners = list(stats.filter(**{field: max_value}).values_list('user__pk', flat=True))

 for pk in winners:
 game.money[str(pk)] = game.money.get(str(pk), 0) + category.bonus_amount

 bonuses[field] = {
 'bonus_amount': category.bonus_amount,
 'winners': winners
 }

 response.bonuses = bonuses
```

```
game.save()
return response
```

```
_end_game_logic(game, user, action=None) classmethod
```

Ends the game, applies final bonuses, generates the summary (GameSummary) with the net worth of all participants (including eliminated ones), and removes the bots.

Parameters:

| Name                | Type                    | Description                                             | Default           |
|---------------------|-------------------------|---------------------------------------------------------|-------------------|
| <code>game</code>   | <code>Game</code>       | The current game instance.                              | <i>required</i>   |
| <code>user</code>   | <code>CustomUser</code> | The user associated with triggering the end game phase. | <i>required</i>   |
| <code>action</code> | <code>Action</code>     | The final action payload.                               | <code>None</code> |

Returns:

| Name                  | Type                  | Description                                        |
|-----------------------|-----------------------|----------------------------------------------------|
| <code>Response</code> | <code>Response</code> | The final response containing the applied bonuses. |

Raises:

| Type                        | Description                                      |
|-----------------------------|--------------------------------------------------|
| <code>GameLogicError</code> | If the game has already been marked as finished. |

• Source code in `magnate/games.py`

```
@classmethod
def _end_game_logic(cls, game: Game, user: CustomUser, action: Optional[Action] = None) -> Response:
 """
 Ends the game, applies final bonuses, generates the summary (GameSummary)
 with the net worth of all participants (including eliminated ones), and
 removes the bots.

 Args:
 game (Game): The current game instance.
 user (CustomUser): The user associated with triggering the end game phase.
 action (Action): The final action payload.

 Returns:
 Response: The final response containing the applied bonuses.

 Raises:
 GameLogicError: If the game has already been marked as finished.
 """
 GameManager._cancel_all_timers(game)

 if not game.finished:
 game.finished = True
 response = cls._apply_end_bonuses(game, num_bonuses=3)

 from django.utils import timezone

 # include eliminated users
 all_participants = PlayerGameStatistic.objects.filter(game=game).select_related('user')
 active_players = game.players.all()
```

```

final_money_dict = {}

for stat in all_participants:
 participant = stat.user
 if participant.username in [participant.username for participant in active_players]:
 # not eliminated
 final_money_dict[str(participant.username)] = _calculate_net_worth(game, participant)
 if not game.has_bots:
 participant.points += final_money_dict[str(participant.username)]
 participant.elo += final_money_dict[str(participant.username)]
 participant.save()
 else:
 final_money_dict[str(participant.username)] = 0

bonus_per_player = {}
for field, info in response.bonuses.items():
 bonus_amount = info['bonus_amount']
 for winner_pk in info['winners']:
 key = str(winner_pk)
 bonus_per_player[key] = bonus_per_player.get(key, 0) + bonus_amount

pk_to_username = {str(stat.user.pk): stat.user.username for stat in all_participants}
final_money_dict_pk = {pk: final_money_dict[username] for pk, username in pk_to_username.items()}

last_money = dinero final sin contar los bonuses
last_money_dict = {
 pk: amount - bonus_per_player.get(pk, 0)
 for pk, amount in final_money_dict_pk.items()
}
response.last_money = last_money_dict

winner = None
max_money = -1

for stat in all_participants:
 participant = stat.user

 if not game.has_bots:
 participant.num_played_games += 1

 p_money = final_money_dict.get(str(participant.username), 0)
 if p_money > max_money:
 max_money = p_money
 winner = participant

 participant.save()

if winner:
 if not game.has_bots:
 winner.num_won_games += 1
 winner.save()

if not game.has_bots:
 GameSummary.objects.create(
 game=game,
 start_date=game.datetime,
 end_date=timezone.now(),
 final_money=final_money_dict

```

```
)

 response.save()
 game.bonus_response = response
 game.save()

 from .models import Bot
 bots_in_game = Bot.objects.filter(id__in=game.players.values_list('id', flat=True))
 bots_in_game.delete()

 active_players.update(active_game=None)

 return response
else:
 raise GameLogicError('game was already ended')
```



## 3.2 Exceptions

Game Exceptions Module.

This module defines custom exceptions used throughout the game logic to handle internal errors, board configuration issues, and invalid or malicious actions performed by users.

### 3.2.1 GameLogicError

Bases: Exception

Exception raised for errors in the internal game logic.

This is typically thrown when the game reaches an impossible state or encounters an unexpected condition during normal execution.

Attributes:

| Name    | Type | Description                              |
|---------|------|------------------------------------------|
| message | str  | Explanation of the internal logic error. |

• Source code in `magnate/exceptions.py`

```
class GameLogicError(Exception):
 """
 Exception raised for errors in the internal game logic.

 This is typically thrown when the game reaches an impossible state
 or encounters an unexpected condition during normal execution.

 Attributes:
 message (str): Explanation of the internal logic error.
 """
 def __init__(self, message=''):
 """
 Initializes the GameLogicError.

 Args:
 message (str, optional): Specific details about the logic error. Defaults to ''.

 Returns:
 None
 """
 self.message = "Internal logic error: " + message
 super().__init__(self.message)
```

`__init__(message='')`

Initializes the GameLogicError.

Parameters:

| Name    | Type | Description                                             | Default |
|---------|------|---------------------------------------------------------|---------|
| message | str  | Specific details about the logic error. Defaults to ''. | ''      |

**Returns:****Type Description**

None

• **Source code in** `magnate/exceptions.py`

```
def __init__(self, message=''):
 """
 Initializes the GameLogicError.

 Args:
 message (str, optional): Specific details about the logic error. Defaults to ''.

 Returns:
 None
 """
 self.message = "Internal logic error: " + message
 super().__init__(self.message)
```

**3.2.2** `GameDesignError`Bases: `Exception`

Exception raised for errors in the game's design or board configuration.

This is used when a square lacks required data (like rent prices) or when the board is fundamentally misconfigured (e.g., missing a Jail square).

**Attributes:****Name**   **Type**   **Description**

|         |     |                                  |
|---------|-----|----------------------------------|
| message | str | Explanation of the design error. |
|---------|-----|----------------------------------|

• **Source code in** `magnate/exceptions.py`

```
class GameDesignError(Exception):
 """
 Exception raised for errors in the game's design or board configuration.

 This is used when a square lacks required data (like rent prices) or
 when the board is fundamentally misconfigured (e.g., missing a Jail square).

 Attributes:
 message (str): Explanation of the design error.
 """
 def __init__(self, message=''):
 """
 Initializes the GameDesignError.

 Args:
 message (str, optional): Specific details about the configuration or design error. Defaults to ''.

 Returns:
 None
 """
```

```
self.message = "Internal logic error: " + message
super().__init__(self.message)
```

```
__init__(message='')
```

Initializes the GameDesignError.

#### Parameters:

| Name    | Type | Description                                                               | Default |
|---------|------|---------------------------------------------------------------------------|---------|
| message | str  | Specific details about the configuration or design error. Defaults to ''. | ''      |

#### Returns:

#### Type Description

None

#### • Source code in `magnate/exceptions.py`

```
def __init__(self, message=''):
 """
 Initializes the GameDesignError.

 Args:
 message (str, optional): Specific details about the configuration or design error. Defaults to ''.

 Returns:
 None
 """
 self.message = "Internal logic error: " + message
 super().__init__(self.message)
```

### 3.2.3 `MaliciousUserInput`

Bases: `Exception`

Exception raised when a user provides invalid, unauthorized, or potentially malicious input.

This acts as a security and validation layer, thrown when a user attempts to manipulate assets they do not own, or tries to exploit the game mechanics.

#### Attributes:

#### Name Type Description

|         |     |                                                                         |
|---------|-----|-------------------------------------------------------------------------|
| message | str | Explanation of the malicious input, tagged with the user's primary key. |
|---------|-----|-------------------------------------------------------------------------|

#### • Source code in `magnate/exceptions.py`

```
class MaliciousUserInput(Exception):
 """
 Exception raised when a user provides invalid, unauthorized, or potentially malicious input.

 This acts as a security and validation layer, thrown when a user attempts
```

to manipulate assets they do not own, or tries to exploit the game mechanics.

```
Attributes:
 message (str): Explanation of the malicious input, tagged with the user's primary key.
"""
def __init__(self, user: CustomUser, message=''):
 """
 Initializes the MaliciousUserInput exception.

 Args:
 user (CustomUser): The user who triggered the exception.
 message (str, optional): Specific details about what the user attempted to do. Defaults to ''.

 Returns:
 None
 """
 self.message = f"[{user.pk}] Potentially malicious input: " + message
 super().__init__(self.message)
```

```
__init__(user, message='')
```

Initializes the MaliciousUserInput exception.

Parameters:

| Name    | Type       | Description                                                           | Default  |
|---------|------------|-----------------------------------------------------------------------|----------|
| user    | CustomUser | The user who triggered the exception.                                 | required |
| message | str        | Specific details about what the user attempted to do. Defaults to ''. | ''       |

Returns:

Type Description

None

• Source code in `magnate/exceptions.py`

```
def __init__(self, user: CustomUser, message=''):
 """
 Initializes the MaliciousUserInput exception.

 Args:
 user (CustomUser): The user who triggered the exception.
 message (str, optional): Specific details about what the user attempted to do. Defaults to ''.

 Returns:
 None
 """
 self.message = f"[{user.pk}] Potentially malicious input: " + message
 super().__init__(self.message)
```

3.2.4 `MaliciousUserInputAction`

Bases: `MaliciousUserInput`

Exception raised when a user attempts an action that is not allowed in the current game phase.

This is a specialized form of MaliciousUserInput used heavily in the GameManager to enforce strict phase-based state machine transitions.

Attributes:

Name    Type    Description

|         |     |                                                                           |
|---------|-----|---------------------------------------------------------------------------|
| message | str | Detailed message including the invalid action and the current game phase. |
|---------|-----|---------------------------------------------------------------------------|

• Source code in `magnate/exceptions.py`

```
class MaliciousUserInputAction(MaliciousUserInput):
 """
 Exception raised when a user attempts an action that is not allowed in the current game phase.

 This is a specialized form of MaliciousUserInput used heavily in the GameManager
 to enforce strict phase-based state machine transitions.

 Attributes:
 message (str): Detailed message including the invalid action and the current game phase.
 """
 def __init__(self, game: Game, user: CustomUser, action: Action):
 """
 Initializes the MaliciousUserInputAction exception.

 Args:
 game (Game): The current game instance where the violation occurred.
 user (CustomUser): The user attempting the invalid action.
 action (Action): The forbidden action the user attempted to perform.

 Returns:
 None
 """
 self.message = f"cannot perform action {action} in phase {game.phase}"
 super().__init__(user, self.message)
```

```
__init__(game, user, action)
```

Initializes the MaliciousUserInputAction exception.

Parameters:

| Name   | Type                    | Description                                             | Default         |
|--------|-------------------------|---------------------------------------------------------|-----------------|
| game   | <code>Game</code>       | The current game instance where the violation occurred. | <i>required</i> |
| user   | <code>CustomUser</code> | The user attempting the invalid action.                 | <i>required</i> |
| action | <code>Action</code>     | The forbidden action the user attempted to perform.     | <i>required</i> |

Returns:

Type Description

None

• Source code in `magnate/exceptions.py`

```
def __init__(self, game: Game, user: CustomUser, action: Action):
 """
 Initializes the MaliciousUserInputAction exception.

 Args:
 game (Game): The current game instance where the violation occurred.
 user (CustomUser): The user attempting the invalid action.
 action (Action): The forbidden action the user attempted to perform.

 Returns:
 None
 """
 self.message = f"cannot perform action {action} in phase {game.phase}"
 super().__init__(user, self.message)
```

3.2.5 `InvalidBotLevel`

Bases: `Exception`

Exception raised when an invalid difficulty level is provided for a bot.

• Source code in `magnate/exceptions.py`

```
class InvalidBotLevel(Exception):
 """
 Exception raised when an invalid difficulty level is provided for a bot.
 """
 def __init__(self, game: Game, level: str):
 """
 Initializes the InvalidBotLevel exception.

 Args:
 game (Game): Th current game intance where the violation occurred.
 level (str): The invalid bot level.

 Returns:
 None
 """
 self.message = f"invalid bot level {level} in game {game.pk}"
 super().__init__(self.message)
```

`__init__(game, level)`

Initializes the InvalidBotLevel exception.

Parameters:

| Name               | Type              | Description                                           | Default         |
|--------------------|-------------------|-------------------------------------------------------|-----------------|
| <code>game</code>  | <code>Game</code> | Th current game intance where the violation occurred. | <i>required</i> |
| <code>level</code> | <code>str</code>  | The invalid bot level.                                | <i>required</i> |

**Returns:****Type Description**

None

**• Source code in** `magnate/exceptions.py`

```
def __init__(self, game: Game, level: str):
 """
 Initializes the InvalidBotLevel exception.

 Args:
 game (Game): Th current game intance where the violation occurred.
 level (str): The invalid bot level.

 Returns:
 None
 """
 self.message = f"invalid bot level {level} in game {game.pk}"
 super().__init__(self.message)
```

## 3.3 Agent Heuristics: Probabilities and Dynamic Synergies

### 3.3.1 0. Core Concepts and Constants

#### Required Constants

- **LANDING\_PROB (PROB\_CAER):** 1/54 (Probability of landing on a specific square, excluding jail).
- **TOTAL\_TURNS:** 500 (Used as a baseline to calculate remaining turns).
- **AUCTION\_ROI\_CTE:** 0.75 (Safety margin to ensure ROI in maximum bids).
- **FANTASY\_CTE:** 0.0 (Expected net value of an unknown fantasy card).

#### Core Concepts

- **Expected Visits:** Landing Probability \* Number of Opponents \* Remaining Turns.
- **Dynamic Reserve:** The highest rent currently chargeable by any opponent on the board.
- **Rent Delta:** The gross change in rent income (including monopoly multipliers or station/bridge scaling) when gaining or losing a property.

### 3.3.2 1. Jail

- **EV\_ExitJail:** Sum of EV of unowned buyables - Expected rent paid to others' properties - Bail cost (if applicable).
- **EV\_StayInJail:** -EV\_ExitJail.

### 3.3.3 2. Buying and Special Moves

- **Buyables (Properties/Bridges/Servers):**  $EV = (Rent\ Delta * Expected\ Visits) + Block\ Value - Buy\ Price + Mortgage\ Value\ (Residual\ Value)$
- **Block Value:** Evaluated ONLY if an opponent already owns properties of the same group. Weighted by the number of opponents.
- **Trams:**
- **Take Tram:**  $EV = Uniform\ average\ EV\ of\ the\ next\ 12\ linearly\ connected\ squares - Travel\ cost.$
- **Skip Tram:**  $EV = Uniform\ average\ EV\ of\ the\ next\ 12\ linearly\ connected\ squares\ from\ the\ current\ position.$

### 3.3.4 3. Square Selection

The square with the highest expected value is chosen. \* **Opponent's Property EV:** -Current Rent owed.

### 3.3.5 4. Business and Construction

- **Build:**  $EV = (Projected\ rent\ increase * Expected\ Visits) - Build\ price.$
- **Demolish:**  $EV = Refund\ (Build\ price / 2) - (Rent\ loss * Expected\ Visits).$
- **Unmortgage:**  $EV = (Rent\ Delta\ gained * Expected\ Visits) - Cost\ (Buy\ price / 2).$
- **Mortgage:**  $EV = Cash\ gained\ (Buy\ price / 2) - (Rent\ Delta\ lost * Expected\ Visits).$  (Note: Rent Delta automatically factors in the monopoly multiplier impact).

### 3.3.6 5. Auctions

- **Max ROI Bid:**  $(Square\ Buying\ EV + Buy\ Price) * AUCTION\_ROI\_CTE.$
- **Budget:** Current Money - Dynamic Reserve.
- **Max Bid:** Minimum between Budget and Max ROI Bid.
- **Real Bids:** Distributed across MaxBid or 0 (not to bid).



### 3.3.7 6. Liquidation and Surrender

- **Liquidate:** Handled inherently by EV sorting. Demolishing/mortgaging monopolies generates massive negative EV due to the Rent Delta penalty, forcing the agent to liquidate low-value single properties first.
- **Surrender:** Executed only if there are absolutely no other actions available to raise funds.

### 3.3.8 7. Trades

- **EV\_Trade:**  $\text{Own Net Benefit} - (\text{Rival Net Benefit} / \text{Number of Opponents})$ .
- **Net Benefit:**  $(\text{Money Gained} + \text{EV\_properties\_gained}) - (\text{Money Lost} + \text{EV\_properties\_lost})$ .
- **Propose (Initiative):** Identify target properties and generate random offers prioritizing maximum EV\_trade. Offer value is capped at 80% of the target's buying EV to ensure profitability.
- **Decision:** Accept or propose exclusively if  $\text{EV\_Trade} > 0$ .